

Safety-Aware Reinforcement Learning for
ISO 15066-Compliant Humanoid Manipulation
Under Sustained Co-Working

Ayush Patel
Department of Computing
Imperial College London
Supervisor: Dr Stephen James

June 2026

Abstract

Classical safety engineering is mature for systems whose behaviour is designed by hand: lane keeping, adaptive cruise control, caged industrial arms, and speed-and-separation interlocks all rely on models that can be analysed and certified. Learned humanoid manipulation is different. A 76-degree-of-freedom robot trained from demonstrations can now work in the same space a person reaches into, but its policy optimises task reward, not human distance, ISO 15066 margins, or contact force. Combining state-of-the-art robot manipulation with collaborative-robot safety is therefore a greenfield problem: the benchmark, cost signals, perception model, training method, runtime filter, and evaluation protocol all have to be built together.

This project builds that system. I introduce **safety_bigym**, a safety-aware extension of Bi-GYM with a moving Unitree G1 (representing a human) coworker, calibrated mock-BodySLAM++ perception noise, three ISO 15066-derived metric flavours, continuous anticipatory costs, and a deployment-faithful snapshot-evaluation harness. On top of the demonstration-driven CQN-AS backbone, I implement a value-based Lagrangian constrained policy and several runtime safety mechanisms, including learned safety-value-function vetoes, ISO speed-scaling, and critic-gated speed-scaling. This is technically hard because CQN-AS is critic-only: there is no actor network to attach a standard Lagrangian to, so the constrained objective must be re-derived in value-based form, and dense safety shaping must fit inside the C51 critic’s value support or training silently saturates.

The main result is a regime map, not a single winning method. Proximity measures exposure, but a controlled sweep shows that $\approx 42\%$ of close proximity is human-driven and remains even when the robot is frozen. The robot-controllable axis is velocity-adaptive SSM. Under *persistent* co-location (**saucepan_to_hob**), only anticipatory constrained RL reduces proximity gracefully: a fixed-multiplier Lagrangian policy cuts proximity by 22.8% ($0.296 \rightarrow 0.228$) at 0.76 success across three seeds, while reactive filters freeze or flee. Under *intermittent* co-location (**dishwasher_close**, **drawers_open_all**), constrained RL is inert or task-fatal, but a learned critic gating an ISO speed-scaling backstop reduces SSM-velocity violations by 50% and 22% with moderate success cost. The Hybrid Safety Critic is therefore a two-arm architecture: proactive policy learning where co-location is persistent, and critic-gated runtime speed control where safe windows exist.

Contents

Acknowledgements	10
1 Introduction	11
1.1 Motivation	11
1.1.1 Robot Learning Is Reaching Humans	11
1.1.2 Most Learned Policies Are Not Safe Near Humans	11
1.1.3 Perception Adds Distribution Shift	12
1.1.4 What This Project Targets	12
1.2 Problem Statement	13
1.3 Research Questions	14
1.4 Contributions	14
1.5 Report Outline	15
2 Background and Related Work	16
2.1 ISO 15066: Speed-Separation and Force Limiting	16
2.1.1 Speed and Separation Monitoring (SSM)	16
2.1.2 Power and Force Limiting (PFL)	16
2.1.3 Why ISO 15066 Is Hard to Target with Generic Safe-RL	16
2.2 Reinforcement Learning Fundamentals	17
2.2.1 Why CQN-AS (Choice of Backbone)	17
2.2.2 Curriculum Learning for Long-Horizon RL	17
2.3 Safe Reinforcement Learning	18
2.3.1 Lagrangian Constrained MDPs	18
2.3.2 Distributional and Tail-Aware Safe RL	18
2.3.3 Recovery, Switching, and Filters	18
2.4 Safety Filters and Control Barrier Functions on Humanoids	18
2.4.1 Filter Conservatism and the Freeze-versus-Flee Limit	19
2.5 Conservative Q-Learning	20
2.6 Sim-to-Real for Human-Aware Robotics	20
2.6.1 Dynamics vs. Perception Sim-to-Real	20
2.6.2 Human Pose Estimation in Co-Located Settings	20
2.7 Benchmark Landscape	20
2.8 Positioning of This Work	21
3 The <code>safety_bigym</code> Benchmark	22
3.1 Why a New Benchmark	22
3.2 Why Build on BiGYM	22
3.3 Robot Embodiment and Simulator	22
3.4 The Human Coworker: A Unitree G1 Stand-In	23
3.5 The CQN-AS Adapter Layer	23
3.6 Bounded Workspace Reward Shaping	24
3.7 The Snapshot-Evaluation Harness	24
3.8 The Three Safety-Metric Flavours	24
3.9 Calibrating the Proximity Threshold	25
3.10 Continuous Cost Signals	26
3.11 The COWORKER Disruption: Design and Rationale	26
3.11.1 Design Goals	27

3.11.2	Three Trajectory Modes	27
3.11.3	Arm State Machine and Reach Gate	27
3.11.4	Train and Eval Parameter Spaces	28
3.12	The Task Suite: Three Tasks, Two Co-location Regimes	28
3.13	Mock BodySLAM++: Calibrated Perception Noise	29
3.14	Demonstrations and the Demo-Conversion Pipeline	29
4	The Hybrid Safety Architecture and Its Implementation	30
4.1	Overview: From Problem to Solution	30
4.2	The Offline Safety Value Function (Phase 2)	30
4.2.1	Dataset and Labelling	30
4.2.2	Architecture and Training	31
4.2.3	No-Pixel Critic: Justification	32
4.2.4	Runtime Wrapper	32
4.3	Closed-Form Runtime Filters: Speed-Scaling and Geometric Dodge	32
4.3.1	Graded ISO 15066 Speed-Scaling	32
4.3.2	Model-Based Geometric Dodge	33
4.3.3	Critic-Gated Speed-Scaling: The Hybrid Safety Critic’s Runtime Arm	33
4.4	External Baseline: Worst-Case SAC	33
4.4.1	Why Value-Based	34
4.4.2	Option A-value: Shaped-Reward Single Critic	34
4.4.3	Option B-value-mean: Dual Q-Functions on Mean Cost (Headline)	34
4.4.4	Option B-value-CVaR: Distributional Cost Critic (future work)	35
4.4.5	The C51 Value-Support Invariant	35
4.4.6	Per-Env-Step Cost Backup	35
4.4.7	PID Lagrange Multiplier Update	36
4.4.8	Filter During Training (optional)	36
4.4.9	Cost-Critic Initialisation and Policy Warm-Start	36
4.5	The Base-Policy Curriculum	36
4.6	Algorithm	37
4.7	Software Architecture	37
4.8	CQN-AS Vendor Integration	37
4.9	Making the Offline Filter Deployment-Faithful	38
4.10	Metric Persistence and the Snapshot Harness	39
5	Experimental Setup and Results	40
5.1	Tasks	40
5.2	Disruption Distributions	40
5.3	Perception Mode (oracle vs. noisy)	40
5.4	Baselines and Configurations	40
5.5	The Two Safety Axes and the Evaluation Protocol (R0)	41
5.5.1	The Exogenous Decomposition of Proximity	41
5.5.2	Axis Ownership	42
5.5.3	Task Performance and the Safety Tax	42
5.5.4	Tail Risk	42
5.5.5	Policy / Filter Interaction	42
5.6	Statistical Methodology	43
5.7	Compute Budget	43
5.8	Experiment Index	43
5.9	Ablation Map: What Each Component Proves	43
5.10	Baseline ISO 15066 Compliance	44
5.11	RQ1: The Human-State Observation Channel	45

5.11.1	E1.1: Behaviour Cloning under No Penalty (Negative)	45
5.11.2	E3.6: Perception Robustness of the Avoidance (oracle vs. noisy)	45
5.12	R1: Persistent Co-location (<code>saucepan_to_hob</code>)	46
5.12.1	E3.1: Cost-Signal Form Is Not the Operative Variable	46
5.12.2	E3.2: Cost Budget Sweep and the Feasibility Boundary	47
5.12.3	Filter Pareto Frontier (E2.1–E2.3)	47
5.12.4	E4.1: The Headline Comparison	49
5.12.5	The Reactive-Filter Taxonomy: Freeze versus Flee	51
5.12.6	Joint ISO Coverage: The Composition and Its Ceiling	52
5.12.7	E4.3: The Filter Does Not Internalise (It Stays Out of Distribution)	52
5.13	RQ4: Tail Risk and OOD Generalisation	53
5.13.1	E5.1: The Worst-Case Tail Is Exogenous and Uncontrollable	53
5.13.2	E5.2: OOD Generalisation on the Wider Eval Band	54
5.14	R2: Intermittent Co-location (<code>dishwasher_close</code> and <code>drawers_open_all</code>)	54
5.14.1	E3.3: Constrained RL Fails to Bind Usefully (WCSAC Corroborates)	55
5.14.2	E2.4: A Binary Veto Breaks a Chunked Policy	56
5.14.3	E2.5: Unconditional Speed-Scaling Owns the Velocity Axis, at Heavy Cost	56
5.14.4	E2.6: Critic-Gated Speed-Scaling (the When/How Split Recovers the Task)	56
5.14.5	E2.6 Ablations: Four Refinements Confirm the Recipe	57
5.15	R3: The Boundary Condition (One Law, Measured Across Three Tasks)	58
5.15.1	E4.4: The Gate Does Not Recover Throughput on Saucepan	58
5.15.2	The Mechanism, Measured	59
5.15.3	The Regime Map	60
5.16	Summary of Results	61
6	Discussion	63
6.1	Research Questions Revisited	63
6.2	Threats to Validity	64
6.2.1	The PFL Contact-Detection Limitation	64
6.2.2	Curriculum Dependence	64
6.2.3	Sim-to-Real: Three Orthogonal Gaps	64
6.2.4	The Runtime Filter’s Role Is Axis- and Regime-Conditional	64
6.2.5	Generalisation Scope and the Resolved Cross-Task Transfer	65
6.2.6	External Baseline (WCSAC): Implemented, with Stated Limits	65
6.2.7	The Regime Classification Is Observational, on Three Tasks	65
6.2.8	Headline-Architecture Caveat (B-value-mean vs. B-CVaR)	66
6.2.9	Computational Feasibility	66
6.3	Broader Impact	66
6.3.1	Responsible Deployment and Misuse Risk	66
6.3.2	Vulnerable Users and Domestic Environments	67
6.3.3	Reproducibility as Honesty	67
7	Future Work	68
7.1	The Persistence Dial: From Regime Map to Causal Test	68
7.2	Closing the Force Loop: Fixing PFL	68
7.3	Proactive Avoidance Beyond the Lagrangian Signal	68
7.4	Explicit CVaR-Lagrangian Training	68
7.5	Formally-Guaranteed and Anticipatory Filters	69
7.6	Real-Robot Transfer and the Coworker Realism Spectrum	69
8	Conclusion	70

Declarations	71
Bibliography	72
A Hyperparameter Tables	79
A.1 Phase 2: Offline CQL Safety Critic	79
A.2 Phase 3: Constrained Lagrangian Agent (Headline: Fixed λ)	79
A.3 Phase 4: Per-Regime Deployment Configurations	80
B Reproducibility Checklist	81
C Use of Generative AI	82
D Ethical Considerations	83
E Sustainability	84
F Code and Data Availability	85

List of Figures

3.1	Contact-geometry calibration of τ_{prox} . The H1 robot (dark) and the G1 coworker (light) are rendered at seven controlled closest-joint separations (columns) from four camera angles (rows), with the <code>saucepan_to_hob</code> table between them. Red column headers are below the 0.30 m proximity threshold: at 0.09 and 0.21 m the body geometries visibly interpenetrate, whereas by 0.31 m (green) a clear gap has opened. This confirms that 0.3 m sits just above the physical contact band, so a proximity violation fires <i>before</i> contact, complementing the empirical-CDF knee of Table 3.1.	25
4.1	The Hybrid Safety Critic at deployment: a two-arm architecture. At each step the Lagrangian-trained policy proposes the nominal action $a^{\text{nom}} = \text{argmax}_a [Q_r(s, a) - \lambda Q_c(s, a)]$; a runtime filter slot passes it through when its safety check holds and otherwise modifies or replaces it (veto, dodge, graded speed-scaling, or critic-gated speed-scaling, Sections 4.2–4.3.3). The two arms have deliberately different jobs: the constrained policy internalises safety <i>proactively</i> during training, while the filter is a <i>reactive</i> runtime mechanism. Chapter 5 shows that which arm carries the safety burden is governed by the co-location regime: under persistent co-location only the policy reduces exposure gracefully (Table 5.7), while under intermittent co-location the critic-gated filter is the working arm (Section 5.14.4).	31
5.1	The exogenous decomposition of proximity (R0’s organising measurement). Sweeping the runtime filter’s intervention rate from 0 to 100 % (a fully frozen robot) removes only ≈ 58 % of time-in-proximity; the residual ≈ 42 % is the coworker approaching a stationary robot, a floor outside robot control. Provenance: this sweep is the <i>prior</i> saucepan SVF threshold sweep, run before the intermittent-co-location experiments existed; the two-axis protocol was fixed on its basis, not after seeing the later results.	41
5.2	Experiment E3.2 / RQ2. PID-Lagrangian at the only feasible budget $d = 0.3$, three seeds. Top: deployment proximity-violation rate ($\tau = 0.3$ m) versus training step; bottom: task success. The seeds converge to $\lambda = 0.000$ (seed 1, unconstrained, flat at the baseline ≈ 0.30), $\lambda = 0.267$ (seed 0, a graceful sub-0.26 avoidance basin), and $\lambda = 3.855$ (seed 2, windup collapse, success $\rightarrow 0$). Because $d = 0.3$ coincides with the task’s inherent cost, the dual variable carries too little signal to auto-tune stably, motivating the fixed- λ headline.	48
5.3	The fixed- $\lambda = 0.1$ policy across three seeds (stars mark the safety-aware operating point at a success floor of 0.75). Top: deployment proximity-violation rate; bottom: success. Unlike the PID runs (Figure 5.2), all three seeds behave consistently and each reaches a sub-baseline proximity basin in mid-training, giving the pooled 0.228 at 0.76 success. Fixing λ removes the ill-conditioned budget loop that made the PID controller seed-unstable.	50
5.4	The two-axis division of labour on <code>saucepan_to_hob</code> and what composing the specialists buys.	53

5.5	Experiment E4.3. The SVF veto’s intervention rate on the constrained policy does <i>not</i> fall during training; it stays at $\sim 40\text{--}60\%$ (blue), above the $\sim 15\%$ it requires on the unconstrained baseline (red dashed). A complementary filter would show a falling curve; this flat, elevated curve is direct evidence that the baseline-collected critic is out of distribution on the avoiding policy and over-vetoes, the mechanism behind the counterproductive SVF-veto composition (Section 5.12.5).	53
5.6	The R2 headline: each method on the success (x) / SSM-violation (y) plane; the desirable corner is bottom-right (high success, low violation), and only the critic-gated filter reaches it on both tasks. Two points need their context. The <i>drawers Lagrangian</i> point sits near the gated one, but it is a basin-selected checkpoint from seed-unstable, single-seed training on which no feasible budget exists (PID- λ is inert or task-fatal across the E3.3 sweep); gating-on-Lagrangian $\approx$ gating-on-baseline (ablation 3, Section 5.14.5) shows it learned no genuinely proactive avoidance; and it offers no deployable dial, whereas the gate threshold R traces a tunable frontier (Figure 5.7). The <i>unconditional speed-scale</i> points reach the lowest violation rates but at $2.5\text{--}5\times$ the gated filter’s success cost.	57
5.7	The gate threshold R is a clean operating-point dial: low R is selective (near-baseline success, small SSM gain), high R approaches unconditional scaling. The dominance claim is task-scoped, per the R0 protocol: on <i>dishwasher</i> the gated frontier matches unconditional scaling’s SSM-violation at strictly higher success; on <i>drawers</i> the gated frontier never reaches unconditional’s absolute floor (0.059 vs. best gated 0.078) but pays far less success at every <i>shared</i> SSM-violation level.	58
5.8	The boundary condition in one picture: each task’s gated R -dial trajectory on the success/SSM-violation plane, normalised to its own no-filter point. Dishwasher and drawers <i>bend down</i> , SSM-violation falls steeply while success is retained, because the gate passes the genuinely-safe majority of steps at full speed. Saucepan <i>slides along the diagonal</i> toward its unconditional endpoint, success and SSM-violation falling together, because the gate is active on most steps and gating degenerates to unconditional scaling.	59
5.9	The regime variable, measured: per-step gate-active fraction of the critic-gated filter at the matched threshold $R = 2.75$ (bars; 95 % bootstrap CIs; 180/60/60 episodes). The saucepan gate fires on 61.5 % of steps, exceeding even the unconditional scaler’s proximity trigger rate (44.2 %, dashed), so gating degenerates to unconditional scaling; on dishwasher/drawers the gate passes 73–81 % of steps at full speed, which is exactly the recovered throughput of Table 5.13.	60
5.10	The regime map (the thesis in one image). The observable regime variable, how persistently the coworker is co-located, measured as the fraction of steps a separation/risk-triggered filter would be active, determines which arm of the Hybrid Safety Critic carries the safety burden. The boundary was validated by the pre-registered E4.4 rule, which failed on saucepan exactly as the regime map predicted. Evidence per cell: gated wins (§5.14.4), Lagrangian infeasibility (§5.14.1), veto coherence break (§5.14.2), policy win and composition ceiling (§5.12.4, §5.12.6), freeze/flee taxonomy (§5.12.5), gating $\approx$ unconditional (Table 5.14). On the intermittent tasks the exposure axis itself is left without a graceful mechanism, bounded by the exogenous floor (§5.14.1).	61
5.11	Marker-facing summary of the intermittent-co-location arm: SSM-violation reduction by method with the success cost annotated. Only critic-gated speed-scaling combines a substantial velocity-axis reduction with a small success cost; the unconditional scaler buys slightly more reduction at several times the cost, the veto and the Lagrangian buy little or none. The persistent-regime counterpart of this figure is Table 5.7.	62

List of Tables

2.1	Existing benchmarks in robot learning and safe RL, positioned against the needs of evaluating ISO 15066-compliant humanoid manipulation under sustained co-working. No prior benchmark combines all five axes. Human-Robot Gym comes closest but enforces SSM/PFL through an always-active shield on 6–7-DoF arms, rather than exposing graded cost signals to the learner.	20
2.2	Positioning of this work’s two-arm architecture (the Hybrid Safety Critic) against representative safe-RL methods. “Tail-aware” indicates whether the method targets a coherent risk measure (CVaR, VaR) rather than expected cost. “Dual critic” separates the task-value and cost-value estimators to avoid the non-stationarity of single-critic Lagrangian. “Runtime guar.” indicates whether a deployment-time mechanism (not just a training-time bound) is provided. “High-DOF” indicates evaluation on a ≥ 16 -DOF embodiment. The contribution of this work is the <i>combination</i> of all five columns on a 76-DOF humanoid with a live G1 humanoid coworker.	21
3.1	Proximity-violation rate as a function of the threshold τ (the empirical CDF of min-separation on the G1 stage-2 baseline, 3 seeds $\times$ 20 episodes, 30,000 steps). The rate climbs steeply once τ crosses into the benign mid-range, which is why a 0.5 m or 1.0 m threshold would absorb a large benign population.	26
3.2	The COWORKER train and eval <code>ParameterSpace</code> . Train is moderately tight, deliberately set to produce a high baseline violation rate (so a 20 + % safety improvement is statistically detectable). Eval is wider, capturing the generalisation question. The 0.60 m closest-approach floor is the smallest body distance that keeps the G1 mocap-pelvis collision capsule clear of the H1 pelvis collision capsule; tighter values generate spurious contact forces and are pathological.	28
5.1	Experiment index. Each is identified by a phase-prefixed code ($En.m$) and reported in Chapter 5; R1 = persistent co-location (<code>saucepan_to_hob</code>), R2 = intermittent co-location (<code>dishwasher_close</code> , <code>drawers_open_all</code>), R3 = the cross-task boundary condition.	44
5.2	Baseline ISO 15066 compliance for the unconstrained CQN-AS policy on <code>saucepan_to_hob</code> under the <code>coworker_train</code> distribution (means $\pm$ 95 % bootstrap CI, 3 seeds $\times$ 20 episodes). This is the “before” picture and the floor against which every later result is measured: the policy is task-competent yet violates proximity on a large fraction of episodes, because nothing in its objective represents the human.	45
5.3	Experiment E1.1. ACT BC under no penalty across four tasks and three observation modes. SSM-violation rate averaged across six legacy disruption types per cell. None of the cells clears the 20 % improvement criterion; oracle is worse than off on three of four tasks.	45
5.4	Experiment E3.6. The constrained policy (seed-0 trained) and the unconstrained baseline re-evaluated under <code>oracle</code> and <code>noisy</code> perception (<code>saucepan_to_hob</code> , <code>coworker_train</code> ; each cell is 60 episodes, 3 evaluation seeds $\times$ 20). The policy’s proximity reduction is preserved under realistic noisy, lagged perception (bootstrap CIs across modes overlap: 0.236 [0.17, 0.30] oracle vs. 0.198 [0.14, 0.26] noisy), and the baseline shows no oracle–noisy gap because it does not use the channel to avoid. The headline E4.1 table reports the pooled three-trained-seed <code>noisy</code> numbers.	46

5.5	Experiment E3.1. Cost-signal form under the same B-value-mean backbone on <code>saucepan_to_hob</code> at the tight budget $d = 0.01$. Both dual-variable forms (continuous, binary) collapse to zero success: the budget sits an order of magnitude below the task’s inherent cost, so the constraint is infeasible and the multiplier winds up regardless of cost-signal shape. The cost form is not the operative variable; the budget relative to the task’s inherent cost is.	47
5.6	Reactive-filter fallback sub-study on the unconstrained baseline policy (v3 G1 SVF, COWORKER distribution, <code>noisy</code> observations; 60 evaluation episodes per row). No reactive fallback reduces geometric proximity at acceptable success and robot speed. Zero-velocity braking is a velocity backstop only; an aggressive retreat buys separation by abandoning the task and accelerating, which worsens the SSM-actual rate.	48
5.7	Experiment E4.1 (the headline). The division of labour between the proactive policy and a complementary-axis runtime filter on <code>saucepan_to_hob</code> under <code>coworker_train</code> , <code>noisy</code> perception. Rows 1 and 3 evaluate the single stage-2 baseline policy over 60 episodes (3 evaluation seeds $\times$ 20); rows 2 and 4 pool three independently trained seeds (180 episodes). Brackets are 95 % bootstrap CIs. The policy owns the geometric-proximity axis ($\tau = 0.3\text{m}$); the graded speed-scaling filter owns the velocity-adaptive SSM-actual axis; only their composition reduces both at once, at a multiplicative success cost. Δ columns are relative to the unconstrained baseline (negative is safer). The two SVF-veto configurations are in Table 5.8.	49
5.8	The reactive-filter taxonomy on <code>saucepan_to_hob</code> (COWORKER, <code>noisy</code>); every row is 60 evaluation episodes, and rows involving the constrained policy use the seed-0 trained policy (the pooled three-trained-seed policy numbers are in Table 5.7). Every reactive modality, learned or model-based, is bounded by the freeze-versus-flee dilemma: it either freezes (and proximity does not fall) or flees (and task success collapses). None reaches the proactive policy’s frontier (success 0.75, proximity 0.198 at seed 0; 0.76/0.228 pooled). The SVF veto applied to the <i>policy</i> (the architecture’s original composition design) is worse than the policy alone.	51
5.9	Experiment E5.1. Separation statistics across the headline configurations (COWORKER, <code>noisy</code>). The policy improves the <i>mean</i> separation but the worst-case tail (CVaR _{0.95} and the single worst approach) is unchanged, because the closest approach is the coworker’s lunge toward a robot it is co-located with, an <i>exogenous</i> event no robot-action policy or filter can remove. The controllable quantity is the dwell and frequency of proximity, not the single worst-case approach.	54
5.10	Experiment E5.2. The proactive policy’s proximity reduction generalises to the wider, gentler held-out coworker distribution. The absolute rate is lower on <code>coworker_eval</code> (the gentler coworker approaches less), but the proportional reduction is preserved; the larger success cost reflects $\lambda = 0.1$ over-constraining a distribution it was not tuned for.	54
5.11	Experiment E3.3. Cost-budget retarget sweep on the intermittent-co-location tasks (40k frames, single training seed per cell, warm-started from the stage-2 curriculum snapshot; cost is the graded SSM-margin signal). Re-targeting makes the constraint <i>bind</i> ($\lambda > 0$ everywhere), but the rolling cost floors at $\approx 0.21\text{--}0.25$, at or above every budget, so λ integrates upward and the task collapses. There is no budget that both binds and preserves the task; even the floor budget has no <i>stable</i> feasible point (<code>dish</code> $d=0.22$ was at 1.00 success at 37k frames, $\lambda = 0.69$, and collapsed to 0.30 by 40k as λ reached 1.5).	55

5.12	Experiment E3.7. WCSAC [1] external reference, reimplemented faithfully and trained from scratch (150k frames, no demonstrations, single seed per cell; 30 evaluation episodes, oracle human-state, so these are upper bounds with respect to perception). WCSAC is non-degenerate on dishwasher but fails to learn drawers at any budget; the apparent budget ordering on dishwasher ($d=30$ at 0.03) is within single-seed from-scratch SAC variance and is not interpreted further. From-scratch WCSAC also sits well below the demo-warm-started value-based Lagrangian (0.47 vs. 0.82 peak on dishwasher), quantifying how much of task competence on these tasks comes from the demonstration prior.	56
5.13	The R2 method comparison (60 episodes per cell, noisy , COWORKER; both safety axes reported per the R0 protocol). Δ columns are relative to each task's unconstrained baseline. The Lagrangian rows are the best basin checkpoints from the seed-unstable E3.3 sweep and carry that caveat (Section 5.14.1); WCSAC rows are the external from-scratch reference at its best budget (single seed, oracle perception). Only critic-gated speed-scaling reduces the SSM-velocity axis substantially while keeping success within 0.10 of baseline.	57
5.14	Experiment E4.4. Critic-gated speed-scaling on saucepan_to_hob (180 episodes per row, 3 trained seeds; noisy ; snapshot_best checkpoints, see the caveat box). The pre-registered rule, success ≥ 0.60 and SSM-actual ≤ 0.08 , is met by no row: gating slides from policy-alone toward the unconditional point without opening a corner. The on-policy re-collected critic (v3op) does not help either, at $R = 2.0$ its SSM-actual (0.149) is <i>worse</i> than policy-alone (0.138), confirming the limit is task structure, not critic calibration.	59
A.1	Phase 2 offline CQL safety-value-function hyperparameters. The dataset is collected under noisy perception because a runtime filter must learn the conditions it faces at deployment (Section 5.3).	79
A.2	Phase 3 headline hyperparameters (fixed- λ constrained CQN-AS). The cost critic is fully decoupled (its own optimiser and its own features, the RoboBase shared-encoder constraint) and cold-started.	79
A.3	Runtime deployment configurations per regime. The veto and the model-based dodges are taxonomy points, not deployment configurations.	80

Acknowledgements

I am grateful to my supervisor, Dr Stephen James, for his guidance, technical insight, and encouragement to follow the evidence where it led, even when it led to a negative result worth characterising rather than a tidy positive one. This work stands on openly released code: I thank the authors of BiGYM for the manipulation simulator and human demonstrations that `safety_bigym` extends, and the authors of CQN-AS for the demonstration-driven agent that serves as its backbone. I am indebted to the AMASS consortium for the motion-capture data that drives the demonstration-replay human-state channel. I also thank the SWIRL research lab and the Imperial College London Department of Computing for technical advice, feedback during the project, and access to the GPU compute on which these experiments were run.

1 Introduction

1.1 Motivation

1.1.1 Robot Learning Is Reaching Humans

In the past five years, robot learning has progressed from controlled lab environments to platforms that share workspaces with people. Imitation-learning methods such as Action Chunking Transformers [2] and Diffusion Policy [3], demo-driven reinforcement-learning agents such as Coarse-to-Fine Q-Networks with action sequences (CQN-AS) [4, 5], and benchmarks designed around human-like manipulation morphologies such as BiGYM [6] and HumanoidBench [7] have driven this transition. Commercial humanoid platforms (including the Unitree H1 [8], the Unitree G1 [9], Appttronik Apollo, and 1X Neo) are being deployed for tasks where the robot is expected to operate *near* a human, not behind a safety cage. ISO 25785-1, under development as a committee draft in 2026 [10], introduces the term “industrial mobile robots with actively controlled stability” specifically to standardise this class of system, reflecting how fast the deployment frontier is moving relative to the standards that govern it.

1.1.2 Most Learned Policies Are Not Safe Near Humans

Safety for machines near people is not new. In classical settings it is comparatively mature. Industrial arms use cages and light curtains and driver-assistance systems such as adaptive cruise control and lane keeping have shipped at scale. Those successes rely on assumptions that fail for learned humanoid manipulation. A cage assumes the human and robot do not share space. Driver assistance assumes a low-dimensional road vehicle with a controller that can be designed and verified in closed form. A 76-degree-of-freedom manipulation policy learned from demonstrations satisfies neither assumption: it shares the workspace by design, operates in the volume a person reaches into, and is a neural policy rather than a hand-derived controller. Safety for this class of system is therefore largely a greenfield problem, and closing it is what this project targets.

In particular, the policies these platforms run are largely *unaware* of the standard that governs their safety. ISO 15066 [11], now integrated into ISO 10218:2025 [12, 13], defines the two normative collaboration mechanisms: Speed and Separation Monitoring (SSM), which prescribes a velocity-dependent geometric bound on how close the robot may come to a human; and Power and Force Limiting (PFL), which caps contact forces against per-body-region biomechanical limits. Standard imitation-learning and reinforcement-learning agents on BiGYM [6] optimise task success against a sparse reward. They have no representation of human position, no SSM margin, and no constraint that would distinguish a task-completing trajectory from one that endangers a nearby person.

Three independent observations frame the problem:

1. *Commercial humanoid platforms are not validated for collaborative operation.* Under ISO 10218:2025, collaboration is a property of the validated application, not of the robot [13], and no commercial humanoid had been validated for such an application by mid-2026. The Unitree G1, the most widely deployed humanoid research platform in 2026, ships with no collaborative-operation validation [9], and the H1 requires a safety cage during “lively testing”. Standards-landscape reviews reach the same verdict: safety standards lag humanoid deployment [14], and the current framework “is not designed for” humanoids [15]. The deployment is happening but the validation is not.¹

¹The nearest milestone to date is machinery-directive conformity, not collaborative-operation validation: AiMOGA announced CE certification of its Mornine humanoid via TÜV Rheinland in September 2025 (company claim).

2. *Robot learning’s safety record remains thin at deployment.* Bharadhwaj [16] argued in 2021 that safety and compliance receive too little attention in robot-learning evaluation, and the audits that have since emerged bear this out: jailbreaks have elicited harmful physical actions from a deployed commercial robot [17], and every LLM-driven robot model in a recent evaluation failed basic safety criteria [18]. Brunke et al. [19] and the newer safe-RL survey of Wachi et al. [20] show an active field of constrained and safety-filtered learning, but its representative benchmarks remain low-dimensional or abstract-cost (e.g. Safety-Gymnasium [21] and safe-control-gym [22]), and the nearest human-co-located exception pairs 6–7-DoF arms with a replayed human under an always-active shield [23].
3. *Concurrent humanoid work confirms the runtime-filter need, but not this task class.* SHIELD [24], demonstrated on a Unitree G1, adds a probabilistic CBF safety layer on top of a learned locomotion controller because black-box learned controllers are difficult to certify and expensive to retrain when constraints change. SHIELD targets navigation around people and obstacles. The analogous gap for manipulation in shared workspaces, with ISO 15066-style separation and force limits, is the target of this work.

1.1.3 Perception Adds Distribution Shift

The safety question is harder still because the human’s pose is not directly observable. Deployment-time human-state estimates come from on-robot perception, typically a monocular or visual-inertial human-pose estimator such as BodySLAM++ [25], which runs at 15+ FPS on a CPU and improves over its predecessor BodySLAM [26] by 26 % on human-state estimation accuracy. Training-time data, by contrast, typically comes from motion capture or simulator ground truth. The gap between training-time motion-capture precision and deployment-time on-robot perception ($\sim$ centimetres of positional error, with occasional dropouts under occlusion) is where many real safety incidents will originate, and is a deployment challenge safe-RL benchmarks rarely model. Most expose ground-truth state [21, 22, 27], and the one that perturbs human-state measurements does so with uncalibrated noise [23].

1.1.4 What This Project Targets

Commercial deployment is moving faster than published safety methods. Manipulation benchmarks largely ignore safety, safety benchmarks largely ignore manipulation, and safe-RL methods usually choose either a training-time constraint or a runtime filter, not both. On top of this, most methods assume cleaner human state than a real robot would have. The intersection is exactly where a safe humanoid manipulator must operate. It is also largely uncharted: recent work covers parts of the problem, but not the full combination of high-DOF humanoid manipulation, a moving human stand-in, ISO 15066-derived separation and force signals, noisy human-state perception, and a direct comparison of training-time and runtime safety mechanisms. This project enters that intersection by closing three concrete gaps in safe-RL evaluation for human-co-located humanoid manipulation:

- **No benchmark exposes the full ISO 15066-specific co-working setting on a humanoid manipulator.** Safety-Gymnasium builds on Safety-Gym [21, 27] but still uses abstract cost budgets. safe-control-gym [22] targets low-DOF control. BiGYM [6] and HumanoidBench [7] target manipulation and locomotion but include no human coworker. Human-Robot Gym [23] comes closest, pairing 6–7-DoF arms with a motion-capture human under a provably safe shield, but it exposes no ISO 15066-derived cost signal to the learner and no humanoid embodiment. We extend BiGYM with a Unitree G1 humanoid as a sustained human-coworker stand-in and surface continuous ISO 15066-derived signals.
- **No published work characterises *when* each safe-RL mechanism helps for manipulation beside a person.** The literature is split into two families that are rarely studied together: training-time constrained RL (Lagrangian CMDP [27, 28]) and runtime safety filtering (shielding [29], CBFs [24, 30], learned safety filters [31, 32]). Cross-family

comparisons exist on low-DOF tasks without humans [22] and, closest to this setting, on a single 6-DoF reaching task beside a replayed human [33]. None yields a rule for which mechanism to reach for in a given human-robot collaboration setting, or a variable that predicts when a runtime filter will preserve task throughput and when it will destroy it. We study both families together on one benchmark and derive exactly such a rule: a *regime map* keyed on the human-robot co-location pattern.

- **Safe-RL manipulation benchmarks rarely model deployment perception.** The mock-BodySLAM++ observation wrapper introduced here fills that gap for this setting: it gives the safety mechanisms a noisy, lagged, occasionally stale human-state estimate rather than perfect simulator state, and lets the report test whether the architecture survives the perception conditions a real deployment would face.

The difficulty is that these gaps have to be closed *simultaneously*. None of the pieces has an off-the-shelf solution at this scale. A demonstration-driven, critic-only agent has no actor network to which a Lagrangian multiplier can be attached, so the constrained objective must be re-derived in value-based form (Section 4.4.1). Dense safety shaping must fit inside a distributional critic’s fixed value support, or training saturates (Proposition 4.1). A runtime safety critic must be trained on the same noisy perception distribution it will see at deployment, not on simulator ground truth (Section 4.2). And the whole system must remain task-competent while a humanoid coworker repeatedly walks into the workspace and reaches toward the robot. The contribution is the architecture that composes these parts, the benchmark that makes the composition measurable, and the *regime map*: an empirical rule linking the pattern of human-robot closeness to the safety mechanism that works.

Terminology. Three terms carry fixed meanings throughout. The *Hybrid Safety Critic* is the two-arm architecture: one arm trains the policy to avoid risk, and the other arm filters actions at runtime. *Critic-gated speed-scaling* is the runtime arm: a learned safety critic estimates whether the proposed action is risky; only when it is risky does the robot slow the action down using the ISO 15066 speed-scaling rule (Section 4.3.3). *Policy-filter composition* is the simpler act of stacking a filter on top of the constrained policy (Section 5.12.6). The empirical finding is which arm should be used in which co-location regime.

1.2 Problem Statement

Formally, this work targets the following constrained Markov decision process (CMDP). Let an agent control a 76-DOF Unitree H1 humanoid manipulating in a domestic kitchen scene, co-located with a Unitree G1 humanoid acting as a moving stand-in for a human coworker that walks into the workspace, reaches periodically toward the robot’s arm or the task object, and occasionally departs and returns. Per-step task reward r_t^{task} is the standard BiGym [6] sparse-plus-shaping signal. Per-step safety cost $c_t \in [0, 1]$ is derived from ISO 15066 SSM geometry and biomechanical force limits, with $c_t > 0$ activated *anticipatorily* as the system approaches a violation rather than only after one occurs (Equations 3.2–3.4).

Two structural properties of this CMDP shape everything that follows. First, its safety is *two-axis*: a geometric *proximity* axis (how often human and robot are close), which is partly *exogenous* because the human can close distance on a stationary robot, and secondly an ISO 15066 SSM *velocity* axis (whether the robot moves slowly enough to stop before contact at the current separation), which the robot fully controls (Section 5.5). Second, the CMDP is instantiated on three tasks (`saucepan_to_hob`, `dishwasher_close`, `drawers_open_all`) that, under the same coworker distribution, induce different *co-location regimes*: on saucepan the coworker occupies the robot’s working volume nearly continuously (*persistent* co-location), while on the other two the close encounters arrive in bursts separated by genuinely-safe windows (*intermittent* co-location; Section 3.12). The central empirical claim of the report is that this regime variable, not the

sophistication of the safety mechanism, determines which mechanism works.

The goal is a deployable system $(\pi, \mathcal{F})$ comprising a training-time policy π and an optional runtime filter $\mathcal{F}$ that:

1. achieves task success competitive with the unconstrained baseline, that is, improves safety without materially degrading task performance,
2. bounds ISO 15066 SSM and proximity violations not merely in expectation but in the upper tail of the per-episode distribution (CVaR-style metrics, Section 5.5),
3. remains stable under partial observability of the human’s pose, matching the perception conditions a real deployment would face,
4. degrades gracefully when the disruption distribution is wider at deployment than at training time.

1.3 Research Questions

RQ1

Does an explicit human-state observation channel improve ISO 15066 compliance, and is its usefulness conditional on the training algorithm having access to a safety-reward gradient?

RQ2

Can a *value-based* Lagrangian constrained-RL policy with a continuous ISO 15066-derived cost achieve a graceful safety–task operating point, and under what conditions does the constraint bind *usefully*, rather than sitting inert or collapsing the task?

RQ3

Which runtime safety filter should be used at deployment? Specifically, when should the robot stop, slow down, move away, or use a learned critic to decide when to slow down?

RQ4

What decides which safety mechanism works? Does the pattern of human–robot co-location explain why one task needs proactive policy learning while another can be handled by a runtime filter?

Each RQ is answered by a specific experiment or experiment family, indexed in Table 5.1, and revisited in the discussion (Section 6.1).

1.4 Contributions

This report studies the intersection of learned high-DOF manipulation, human co-working, ISO 15066-referenced safety, noisy human-state perception, and safe-RL mechanisms. Existing work covers important parts of that space, but not the whole setting at once. The contributions are therefore both infrastructural and empirical:

- C1. A benchmark for asking the safety question.** `safety_bigym` extends BiGYM with a moving Unitree G1 coworker, three ISO 15066-derived safety metrics, continuous anticipatory costs, noisy human-state estimates, three manipulation tasks, and a deployment-faithful evaluation harness (Chapter 3).
- C2. A two-axis safety measurement protocol.** Proximity measures exposure, but a frozen-robot sweep shows that $\approx 42\%$ of close proximity is caused by the human approaching the robot. We therefore report both proximity exposure and the robot-controllable velocity-adaptive SSM axis (Section 5.5).

- C3. A regime map for choosing the safety mechanism.** The key variable is the co-location pattern: whether the human and robot are close almost continuously or only in bursts. Persistent co-location requires proactive policy learning. Intermittent co-location allows a runtime critic to find safe windows in which the robot can keep moving at full speed (Section 5.15).
- C4. Mechanism-level lessons.** Learned vetoes freeze, geometric dodges flee, and neither gracefully reduces proximity under persistent co-location. A binary veto also breaks the action coherence of the chunked CQN-AS policy, raising max velocity from 2.46 to 6.03 m/s. Continuous speed-scaling preserves action direction, so the successful runtime design is: *let the critic decide when to intervene, but slow smoothly rather than veto* (Sections 5.12.5 and 5.14.2).
- C5. Deployable recipes.** On `saucepan_to_hob`, a fixed-multiplier constrained policy reduces proximity by 22.8 % at 0.76 success. On `dishwasher_close` and `drawers_open_all`, critic-gated speed-scaling reduces SSM-velocity violations by 50 % and 22 % respectively, with moderate success cost. WCSAC is reimplemented as an external safe-RL reference (Chapter 5).
- C6. A reproducible value-based constrained-RL formulation.** Because CQN-AS is critic-only, the Lagrangian must be written in terms of $Q_r - \lambda Q_c$ rather than an actor update. The report also proves a C51 support-bound condition showing when dense shaping is safe to add to a distributional critic (Section 4.4).

1.5 Report Outline

The remainder of the report follows the arc of the project. Chapter 2 reviews the relevant literature in collaborative-robot safety, safe reinforcement learning, and human-aware sim-to-real, and positions this work against it. Chapter 3 presents `safety_bigym` (contribution C1): the benchmark environment, the Unitree G1 coworker, the ISO 15066-derived safety metrics, the three-task suite and its two co-location regimes, the calibrated perception-noise model, and the snapshot-evaluation harness. Chapter 4 presents the Hybrid Safety Critic architecture: the value-based constrained policy, the learned safety value function, the closed-form and critic-gated runtime filters, and the WCSAC external baseline, together with the implementation work that realises them. Chapter 5 reports the evaluation in four movements: the two-axis protocol (R0, contribution C2); the persistent-co-location results on `saucepan_to_hob` (R1); the intermittent-co-location results on `dishwasher_close` and `drawers_open_all` (R2); and the boundary-condition synthesis (R3, contribution C3). Chapter 6 interrogates threats to validity and broader impact, Chapter 7 sets out future work with the persistence-dial experiment as the named next step, and Chapter 8 concludes. A declarations section covering originality, generative-AI use, ethics, sustainability, and code availability precedes the references; hyperparameter tables and the full reproducibility checklist appear as appendices.

2 Background and Related Work

2.1 ISO 15066: Speed-Separation and Force Limiting

ISO 15066 [11], recently integrated into ISO 10218:2025 [12, 13], is the international standard governing contact safety in collaborative robotics. It defines two normative mechanisms relevant to autonomous manipulation.

2.1.1 Speed and Separation Monitoring (SSM)

SSM requires that the protective separation distance between any human and any robot link exceeds the velocity-dependent *required-separation distance* S_p :

$$S_p = v_h (T_r + T_s) + v_r T_r + \frac{v_r^2}{2a_{\max}} + C + Z_d + Z_r, \quad (2.1)$$

where v_h is human velocity, v_r is robot-link velocity, T_r and T_s are reaction and stopping latencies, $a_{\max}$ is the robot’s maximum deceleration, C is the intrusion contribution, and Z_d, Z_r are uncertainty terms [11, 34, 35]. The standard mandates $v_h \leq v_{h,\max} = 1.6$ m/s as a conservative cap on assumed human motion. Under typical domestic-manipulation geometry the conservative S_p over-fires ($\sim 93\%$ violation rate on random rollouts in our environment), motivating the velocity-adaptive variant introduced in Section 3.8. Our implementation evaluates S_p with the human and robot-velocity terms and the braking term, plus a single lumped constant C into which the sensor-specific position-uncertainty terms Z_d and Z_r are folded, since their per-axis budgets are not defined for a simulated camera; this keeps the geometric check faithful to the standard’s dominant velocity-dependent structure while avoiding fabricated uncertainty figures.

2.1.2 Power and Force Limiting (PFL)

PFL caps inadvertent contact forces against per-body-region biomechanical thresholds. Annex A of the standard tabulates quasi-static and transient force limits per body region (forehead, chest, abdomen, upper arm, forearm, hand, etc.), carried forward into ISO 10218-2:2025 as Annex M with the PFL requirement now normative [13]. [36] measures these forces experimentally on UR10e and KUKA LBR iiwa arms and finds they vary by more than 100% within the workspace, a finding confirmed at journal scale by the same group’s 2,250-collision campaign [37] and one that motivates the force-ratio formulation

$$f_t^{\text{ratio}} = F_t^{\text{contact}} / F_t^{\text{limit}} \quad (2.2)$$

adopted in Equation 3.3 of the cost signal. $f_t^{\text{ratio}} \geq 1.0$ constitutes a PFL violation; the anticipatory cost activates at $f_t^{\text{ratio}} > 0.8$, giving the policy headroom to react.

2.1.3 Why ISO 15066 Is Hard to Target with Generic Safe-RL

Generic safe-RL formulations target a single CMDP cost budget $\mathbb{E}_\pi[\sum_t c_t] \leq d$ on a per-step cost *usually treated as a binary indicator* [21, 27, 28, 38, 39]. ISO 15066 compliance, by contrast, is both *geometric* (the required separation depends on instantaneous velocity) and *distributional* (the standard’s intent is a worst-case bound on *every* step, not an expectation; a 1% mean cost is consistent with a 1% violation rate at unit severity or with a 0.1% violation rate at 10× severity, very different worst-case implications). The architecture in Chapter 4 addresses both gaps: continuous, anticipatory cost signals (Section 3.10) replace binary indicators, and CVaR-style evaluation metrics (Section 5.5) align the reported numbers with the standard’s worst-case intent.

2.2 Reinforcement Learning Fundamentals

The control problem is formalised as a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, transition kernel $P(s' | s, a)$, reward $r(s, a)$, and discount $\gamma \in [0, 1)$. A policy $\pi(a | s)$ induces a state-action value $Q^\pi(s, a) = \mathbb{E}_\pi[\sum_t \gamma^t r_t | s_0 = s, a_0 = a]$, the unique fixed point of the Bellman operator $(\mathcal{T}^\pi Q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s', a'}[Q(s', a')]$. Value-based agents learn Q and act greedily, $\pi(s) = \arg \max_a Q(s, a)$. The safety architecture of Chapter 4 adds a second value function over a cost signal and selects actions against a combination of the two. Safety is imposed by lifting the MDP to a *constrained* MDP (CMDP), which augments the objective with the cost constraint of Equation 2.3; Section 2.3 reviews how that constraint is solved.

Rather than estimate the scalar Q^π , *distributional* RL learns the full return distribution $Z^\pi(s, a)$ whose expectation is Q^π [40]. The C51 algorithm represents Z^π as a categorical distribution over a fixed support $[v_{\min}, v_{\max}]$ and minimises a projected Bellman error. This fixed support is central for the safety extension: a shaped reward whose discounted return exceeds the support is silently clipped, which breaks value learning unless an invariant holds (Proposition 4.1). Our backbone is built on a C51 critic, which is why distributional value learning is introduced here.

Three agent families recur as baseline or backbone. *Imitation learning* (ACT [2], Diffusion Policy [3]) is strong on BiGYM but, lacking a reward channel, cannot respond to a safety cost, a limitation we confirm empirically (Section 5.11.1). *Off-policy pixel RL* (DrQ-v2 [41]) learns from reward but struggles on long-horizon sparse-reward manipulation without a demonstration prior. *Demonstration-driven, value-based RL*, the family we adopt, combines both; CQN-AS [4] is the instance used throughout, for the reasons given next.

2.2.1 Why CQN-AS (Choice of Backbone)

CQN-AS [4, 5] (the coarse-to-fine Q-network of [4] with the action-sequence extension of [5]) was chosen because BiGYM demonstrations [6] are a strong inductive prior on long-horizon manipulation, and CQN-AS is a demo-driven, value-based agent that achieves high success on long-horizon BiGYM tasks (we obtain around 85% success on `saucepan_to_hob`; Section 5.10). CQN-AS was evaluated on BiGYM by its own authors [4]; the BiGYM paper itself benchmarked imitation-learning and demo-driven RL baselines [6]. The critic-only, coarse-to-fine, C51-headed [40] structure is unusually well-suited to the safety extension introduced here: a second *cost* critic of the same shape composes cleanly via dual-Q action selection (Section 4.4.3), without requiring an actor network at all. The non-trivial consequence is that the standard actor-critic Lagrangian formulation [27, 28] must be re-derived in value-based form; this re-derivation is itself a methodological contribution.

We chose CQN-AS over Diffusion Policy [3] specifically because Diffusion Policy is an imitation method that does not naturally accept a per-step cost gradient. Imitation agents marginalise observation channels that are uncorrelated with demonstrated actions (a finding we confirm empirically in Experiment E1.1, Section 5.11.1); they cannot respond to a safety reward signal.

2.2.2 Curriculum Learning for Long-Horizon RL

Curriculum learning orders the training distribution from easier to harder variants of a task so that an agent can bootstrap competence it could not acquire by training on the hardest setting from scratch [42]. The technique is standard in high-dimensional robotics: OpenAI’s dexterous-manipulation work [43] used an automatic difficulty curriculum to make an otherwise intractable problem learnable, massively parallel legged-robot training rests on a game-inspired terrain curriculum [44], and current Unitree H1 pipelines adjust terrain difficulty on success [45]. In our setting the difficulty axis is the coworker’s intrusiveness: direct training of the CQN-AS backbone on the full COWORKER disruption distribution fails to discover the manipulation task at all (the agent is dominated by avoidance and never gets the saucepan onto the hob). We

therefore adopt a staged idle $\rightarrow$ easy $\rightarrow$ full curriculum, justified and specified in Section 4.5, and treat the curriculum dependence itself as a reported limitation (Section 6.2.2).

2.3 Safe Reinforcement Learning

2.3.1 Lagrangian Constrained MDPs

The standard formulation [27, 46] augments the reward objective with a cost constraint:

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_t \gamma^t r_t \right] \quad \text{subject to} \quad \mathbb{E}_{\pi} \left[\sum_t \gamma^t c_t \right] \leq d, \quad (2.3)$$

solved by introducing a Lagrange multiplier $\lambda \geq 0$ that trades off task return against cost. Gradient ascent [27, 38], PID control [28], and primal-dual variants all appear in the literature. PID outperforms gradient ascent on convergence and oscillation when constraints are tight, and we adopt the PID variant (Section 4.4.7). García and Fernández [47] is the foundational safe-RL survey. Brunke et al. [19] connect safe learning to robotics; and the newer survey by Wachi et al. [20] summarises current constraint formulations. These surveys establish the method families, but not the co-location regime boundary studied here.

2.3.2 Distributional and Tail-Aware Safe RL

Mean-cost Lagrangian methods bound only $\mathbb{E}[Z_c]$. *Distributional* safe RL replaces the expectation with a coherent risk measure, typically the Conditional Value at Risk:

$$\text{CVaR}_{\alpha}(Z_c) = \mathbb{E}[Z_c \mid Z_c \geq \text{VaR}_{\alpha}(Z_c)], \quad (2.4)$$

the expected cost conditional on being in the worst $1 - \alpha$ fraction of outcomes. WCSAC [1, 48] parametrises the cost return as Gaussian and minimises CVaR_{α} directly; quantile critics [49] are a non-parametric alternative. This work adopts *CVaR as the evaluation metric* (Section 5.5.4), reports CVaR for the mean-cost-optimised policy as a diagnostic (Section 5.13.1), and identifies explicit CVaR-optimised training as natural future work (Section 7.4).

2.3.3 Recovery, Switching, and Filters

The runtime-override family has a formal-methods anchor in *shielding* [29, 50], which synthesises a reactive monitor that overrides any action violating a temporal-logic safety specification. Learned variants replace the synthesised shield with a value function: Recovery RL [32] switches between a task policy and a recovery policy based on a learned safety value, and Srinivasan et al. [31] train a safety critic offline and use it as a deployment-time filter. All are structurally related to the runtime filters in Section 4.2: each consults a safety signal to decide whether to override the task action. The distinguishing choices are *when* the switch happens (training time for Recovery RL, deployment for shields and for this work) and *what the override does*, a design axis that Section 5.14.2 shows is decisive: a hard override (veto) interacts destructively with chunked action execution, while a graded speed reduction does not.

2.4 Safety Filters and Control Barrier Functions on Humanoids

Hamilton-Jacobi reachability [51, 52] gives the theoretical foundation for safety value functions. Practical implementations span hand-engineered Control Barrier Functions (CBFs) [30], predictive filters [53], and learned safety value functions [31, 32], a spectrum unified under a single safety-filter view by Hsu et al. [54]. Recent CBF-RL hybrids [55] apply CBF-style filtering during RL training so the deployed policy needs no filter. SHIELD [24], demonstrated on a Unitree G1, applies a probabilistic CBF filter over high-level commands without modifying the learned controller.

Our Phase 2 safety filter (Section 4.2) is in this learned-value-function family but is trained offline with CQL [56] rather than via online HJ reachability iteration. The trade-off is explicitness

versus data efficiency: HJ and CBF methods provide tighter formal guarantees, while CQL-trained filters scale to higher-DOF observation spaces without solving a constrained optimisation at every step. Solver cost alone no longer rules out CBF-QPs at moderate scale. A whole-body CBF-QP with 15 collision pairs solves in ~ 0.2 ms on a 24-DOF humanoid in simulation [57], and operational-space formulations sustain hundreds of constraints at kilohertz rates on a 7-DOF arm [58]. The combination posed here remains undemonstrated, however. Hardware humanoid filters to date restrict themselves to reduced-order models with a single constraint [24, 59] or to safe-set control of an upper-body subspace [60], and the most direct attempt at human-link $\times$ robot-link CBF filtering on a humanoid is simulation-only, commands an 8-DoF upper-body subspace, and solves its 72 capsule constraints at ~ 33 Hz on a GPU [61], below this benchmark’s control rate and before certified barrier synthesis under exogenous human motion is even addressed. A learned value function reduces the runtime decision to one forward pass at any embodiment dimension. This practical consideration, alongside the data-efficiency argument above, is why we adopt the learned-value-function path.

2.4.1 Filter Conservatism and the Freeze-versus-Flee Limit

A recurring tension runs through the runtime-filter literature: a filter strict enough to guarantee safety against an unmodelled human tends to be too conservative to let the task proceed. This is the field’s own stated problem. Recent latent safety filters [62] observe that the standard *least-restrictive* construction “potentially undermines the task performance that makes modern visuomotor policies valuable”, degrading task completion sharply in their own experiments. Smoother variants [63] are proposed specifically to soften the resulting freeze. A recent separation result [64] gives the theoretical counterpart: a runtime filter need *not* degrade asymptotic return, but only when it is least-restrictive (an identity on already-safe actions), and the prescription, deploy the most permissive filter available, is precisely the condition a baseline-calibrated learned veto violates once the policy it guards has itself learned to avoid, a mechanism we observe directly in Section 5.12.5.

The most pointed evidence concerns an *approaching* human whose motion the robot cannot predict. Tian et al. [65] show that a worst-case Hamilton–Jacobi filter “maintain[s] safety, but inhibit[s] the robot’s ability to make progress by intervening excessively”, the freeze-versus-flee mode, and that the obvious remedy, trusting a human-motion model to relax the bound, is unsafe: their model-trusting variant suffered a 25 % collision rate against an unmodelled, distracted human, while their safe, confidence-aware variant recovers low task cost *only* while the human follows the model, reverting to the conservative worst-case filter (and its cost) whenever the human does not. The implication for sustained co-working is direct: against a coworker who repeatedly walks into the workspace, a reactive filter can buy task-cheap safety only when the human is predictable, and must pay the conservatism cost exactly when they are not. The freeze half of the dilemma is old enough to have a name: the *freezing-robot problem* [66], identified in crowd navigation, where a planner that treats the human as an unconstrained obstacle concludes that every forward path is unsafe and stalls. Proactive policies are not bound by this trade in the same way: a learned navigation policy keeps its freezing rate below 1 % where an analytical velocity-obstacle solver with ground-truth state freezes and stalls in dense crowds [67].

What the literature does *not* provide, on either side, is a characterisation of *when* each mechanism helps. The constrained-RL line reports tasks where a cost budget is feasible, and the shielding and filtering line reports settings where interventions are rare enough to be cheap. The nearest cross-family evidence is a single 6-DoF reaching task beside a replayed human, on which Thumm et al. [33] found a PID-Lagrangian agent “unable to learn a suitable policy” while their always-on shield held violations at zero, an observation the intermittent-regime results here reproduce independently at 76 DOF (Section 5.14.1). Within the filter family, Krasowski et al. [68] offer guidance on choosing an intervention type, but no work selects between the families, names the variable that separates their successes from their failures, or tests both across co-location regimes on a high-DOF manipulator under ISO 15066. These results frame the central

empirical question of Chapter 5: not only *whether* a reactive filter can add safety at acceptable task cost against an approaching coworker, but *under what observable conditions*. The regime map of Section 5.15 is this report’s answer.

2.5 Conservative Q-Learning

CQL [56] is a standard pessimistic offline-RL method [69]. The regulariser pushes down Q-values for out-of-distribution actions, preventing a safety filter from falsely certifying unseen actions as safe. Applying the same pessimism to a cost critic is established practice in safe offline RL [70], and the safety critic here follows that pattern. A bounded-output network (scaled sigmoid to $[0, 1/(1 - \gamma)]$) provides additional protection against overestimation. See Section 4.2 for the full safety-critic treatment.

2.6 Sim-to-Real for Human-Aware Robotics

2.6.1 Dynamics vs. Perception Sim-to-Real

Most sim-to-real literature focuses on the *dynamics gap* [71, 72], that is, the mismatch between simulated and real-world physics: contact and friction modelling, actuator response, mass and inertia estimates, and control latency. The deployment story for human-aware robotics also includes a *perception gap*: human-state estimates on a real robot come from on-robot pose estimators, not from motion capture or simulator ground truth. The mock-BodySLAM++ model of Section 3.13 captures the perception side; dynamics-side sim-to-real is a documented limitation, discussed in Section 6.2.3.

2.6.2 Human Pose Estimation in Co-Located Settings

BodySLAM++ [25] is a tightly-coupled visual-inertial human-and-camera state estimator that achieves 15+ FPS on an Intel i7 CPU, improving over its predecessor BodySLAM [26] by 26 % on human-state accuracy and 12 % on camera-state accuracy. Typical positional error is on the order of ~ 10 cm, with higher errors under occlusion. The mock-BodySLAM++ noise model (Section 3.13) is calibrated against these published characteristics: Ornstein-Uhlenbeck temporally correlated position noise, a 3-step latency buffer matching the 15 Hz perception clock, and stochastic dropout matching reported tracking failures. Newer world-grounded estimators such as WHAM [73] are more accurate but run offline on GPUs, so the noise model stays calibrated to the on-robot CPU estimator.

2.7 Benchmark Landscape

Table 2.1: Existing benchmarks in robot learning and safe RL, positioned against the needs of evaluating ISO 15066-compliant humanoid manipulation under sustained co-working. No prior benchmark combines all five axes. Human-Robot Gym comes closest but enforces SSM/PFL through an always-active shield on 6–7-DoF arms, rather than exposing graded cost signals to the learner.

Benchmark	High-DOF	Live human	ISO 15066 signals	Perception noise	Sustained interact.
Safety-Gym [27]	✗	static obstacles	✗	✗	✗
Safety-Gymnasium [21]	✗	static obstacles	✗	✗	✗
safe-control-gym [22]	✗	✗	✗	✗	✗
Safety-CHORES [74]	✗	✗	✗	✓(egocentric)	✗
RLBench [75]	✗	✗	✗	✗	✗
BiGYM [6]	✓	✗	✗	✗	✗
HumanoidBench [7]	✓	✗	✗	✗	✗
Assistive Gym [76]	✗	✓(seated)	✗	✗	✓
Habitat 3.0 [77]	✗	✓(kinematic)	✗	✗	partial
Human-Robot Gym [23]	✗	✓(mocap)	shield	✓(optional)	✓
safety_bigym (this work)	✓	✓(G1)	✓	✓	✓

The landscape splits along three axes. **Safety-axis** benchmarks (Safety-Gym [27], Safety-Gymnasium [21], safe-control-gym [22], and most recently Safety-CHORES [74]) expose cost

budgets without human coworkers. **Manipulation-axis** benchmarks (RLBench [75], BiGYM [6], HumanoidBench [7]) expose realistic tasks without safety constraints. **Human-axis** benchmarks place a simulated person beside the robot, from assistive care (Assistive Gym [76]) to social navigation (Habitat 3.0 [77]), and Human-Robot Gym [23] adds an ISO-grounded shield with optional measurement noise on 6–7-DoF arms. None exposes ISO 15066-derived continuous cost signals on a high-DOF humanoid, and the intersection this work targets remains uncovered.

2.8 Positioning of This Work

Table 2.2: Positioning of this work’s two-arm architecture (the Hybrid Safety Critic) against representative safe-RL methods. “Tail-aware” indicates whether the method targets a coherent risk measure (CVaR, VaR) rather than expected cost. “Dual critic” separates the task-value and cost-value estimators to avoid the non-stationarity of single-critic Lagrangian. “Runtime guar.” indicates whether a deployment-time mechanism (not just a training-time bound) is provided. “High-DOF” indicates evaluation on a ≥ 16 -DOF embodiment. The contribution of this work is the *combination* of all five columns on a 76-DOF humanoid with a live G1 humanoid coworker.

Method	Noise-aware	Runtime guar.	Tail-aware	Dual critic	High-DOF
Safety-Gym Lagr. [27]	✗	✗	✗	✗	✗
WCSAC [1]	✗	✗	✓	✓	✗
Recovery RL [32]	✗	training-time	✗	✓	✗
CBF-RL [55]	✗	training-time	✗	✗	✗
SHIELD [24]	✗	✓	probabilistic	✗	✓(G1)
Learned SVF [31]	✗	✓	✗	N/A	✗
This work	✓	✓	diagnostic*	✓	✓(76 DOF)

* B-value-mean optimises the mean cost; CVaR-optimised training is future work (Section 7.4). CVaR is reported as an evaluation diagnostic.

WCSAC [1] introduced the distributional cost critic on Safety-Gym, much lower-DOF than our setting. We reimplement it faithfully on the 76-DOF humanoid tasks and report it as the external safe-RL reference (Section 5.14.1). CBF-RL [55] and SHIELD [24] provide runtime CBF-style filtering on top of RL but without a distributional treatment of cost. SHIELD is the closest concurrent work (G1 humanoid, runtime safety) but targets navigation around humans/obstacles rather than manipulation in a shared workspace. Recovery RL [32] provides the switching formulation but at training time, on lower-DOF arms, and without distributional safety. This work evaluates representatives of both families, constrained RL and runtime filtering, on the same benchmark, under the most challenging combination of embodiment, disruption pattern, and perception conditions in the safe-RL literature to date; its distinguishing output is not a new mechanism but the *regime map* that says when each existing mechanism class works, a characterisation neither family’s own literature provides.

3 The `safety_bigym` Benchmark

3.1 Why a New Benchmark

This chapter presents `safety_bigym`: a safety-aware extension of BiGYM [6] that we contribute as one of the two principal deliverables of this project. `safety_bigym` is a *benchmark*, not a method: the environment and metric scaffolding against which any present or future safe-RL approach for humanoid manipulation can be evaluated.

Table 2.1 in Section 2.7 established that no existing benchmark covers the intersection of (i) a high-DOF humanoid embodiment, (ii) a live moving human coworker, (iii) ISO 15066-derived safety signals, (iv) calibrated perception noise, and (v) sustained interaction. `safety_bigym` fills this gap. The aspects of human-robot interaction that the benchmark is designed to test are:

1. **ISO 15066 SSM compliance while completing the task:** the velocity-dependent bound exposes fast-moving-near-human behaviour, not mere task incompetence.
2. **Response to *anticipatory* safety signal:** the continuous cost activates before violations, and the benchmark emits margins, not just binary flags.
3. **Robustness to noisy human-state observation:** ground-truth and calibrated mock-BodySLAM++ modes are switchable, so the compliance delta is measurable.
4. **Task competence under sustained co-working:** a “stop until the human leaves” policy trivially achieves zero contact and trivially fails the task; the success-rate axis catches it.
5. **Generalisation beyond the training disruption band:** train/eval `ParameterSpace` separation supports an explicit OOD experiment (Section 5.13.2).
6. **Graceful tail behaviour:** raw per-step `min_separation` is exposed, so CVaR and percentile metrics are computable for any policy.

These six items are the explicit design targets of `safety_bigym`; every design choice in this chapter is motivated by one or more of them.

3.2 Why Build on BiGym

We build on BiGYM [6] rather than starting from scratch for three reasons. First, BiGYM provides a 76-DOF Unitree H1 humanoid kinematic chain, 40 manipulation tasks, and human-collected teleoperation demonstrations, all non-trivial assets to recreate. Second, BiGYM is an established manipulation benchmark with published baselines: its authors benchmark mainstream imitation learning (ACT [2], Diffusion Policy [3]) and demo-driven RL, and CQN-AS [4], the agent we adopt, was itself evaluated on BiGYM by its authors. This gives us a reference set of methods and reported scores against which to position safety-aware variants. Third, the BiGYM task distribution (manipulation in domestic kitchen scenes) is exactly the deployment context where ISO 15066 compliance matters most: tasks such as `saucepan_to_hob`, which requires the robot to open a cupboard, take out a saucepan and place it on the hob, are representative of close-range domestic manipulation carried out near a person [6].

3.3 Robot Embodiment and Simulator

We adopt BiGYM’s 76-DOF Unitree H1 humanoid as the manipulating robot, with the **bi-manual** action mode ($\mathcal{A}_{\text{bm}} \in \mathbb{R}^{16}$, with a 4-DOF floating base). Whole-body locomotion adds a substantial learning challenge orthogonal to the safety question this project targets. Bi-manual mode isolates the manipulation-near-human problem and is the mode BiGYM’s authors themselves benchmark on. The proprioceptive observation is $s_{\text{proprio}}^{\text{bm}} \in \mathbb{R}^{64}$.

3.4 The Human Coworker: A Unitree G1 Stand-In

A second articulated body is added to the scene to stand in for a human coworker. **We model the human with a Unitree G1 humanoid [9]**, rather than a parametric human body. The choice is deliberate, for three reasons:

- (i) **The stand-in is a physically grounded, well-characterised body.** We model the person with an off-the-shelf Unitree G1, whose mass distribution, geometric extent, joint limits and achievable velocities are published platform quantities [9]. The ISO 15066 limits are instantiated against concrete numbers rather than a hand-tuned parametric body, and the stand-in lives inside the same MuJoCo contact and dynamics model as the robot, so separations and contact forces are computed consistently for both bodies.
- (ii) **The safety-relevant properties are biomechanical, not specifically human.** SMPL-H [78] and similar parametric models remain the standard for human-pose research [79], but the quantities driving an ISO 15066 SSM/PFL assessment (velocity, geometric extent, contact response) are functions of mass and articulated degrees of freedom, not of human appearance. The limits are applied to the stand-in exactly as if it were a person, under the conservative assumption that any human sharing the workspace is at most as massive and as fast as the G1.
- (iii) **The benchmark is body-agnostic.** Planner, scenario sampler, and IK chains route through a single `HumanConfig.human_model` selector, so the G1 can be swapped for any articulated body with no architectural change. Alternative embodiments are supplementary (Section 7.6).

Modelling decisions.

- **Pelvis as a mocap body, not a freejoint.** The pelvis is implemented as a MuJoCo `mocap="true"` body, with position and orientation written directly by the trajectory planner each control step. The original (and natural) choice, a `<freejoint/>` pelvis, was discarded because the implicit pelvis velocity computed by MuJoCo as $\Delta_{\text{pos}}/\Delta_{\text{phys}}$ at the physics timestep ($\Delta_{\text{phys}} = 2\text{ ms}$) reached $\sim 120\text{ m/s}$ during teleport-style updates, causing the SSM-required separation distance from Equation 2.1 to balloon to $\sim 18\text{ m}$. The mocap pelvis exposes physically-realistic velocity to the safety wrapper at the control timestep.
- **Cross-paired collision channels.** The collision-channel scheme was redesigned so that the G1 coworker emits contact on MuJoCo channel bit 1 and accepts bit 2. The manipulating H1 and the static scene emit bit 2 and accept bit 1. This eliminates G1-self-collision between adjacent body segments (originally producing $\sim 220\text{ kN}$ spurious forces and crashing the simulator within $\sim 1\text{ s}$) and G1-scene penetration with dishwashers and cupboards. Without this fix, training is infeasible regardless of the safety architecture.
- **PD actuator gains.** The G1’s body joints are driven by position actuators with $k_p = 200$, $k_v = 20$, matching the per-joint mass and inertia of the G1 platform [9] and aligning the closed-loop dynamics with a representative humanoid co-worker.

3.5 The CQN-AS Adapter Layer

BIGYM exposes a Gym-style `step/reset` API. CQN-AS [4] consumes a `dm_env`-style `ExtendedTimeStep` API. The adapter bridges these and carries per-key frame-stack dequeues, a bidirectional action normaliser for CQN-AS’s discrete bin centres, and a safety-payload extractor that routes per-step `info["safety"]` into `ExtendedTimeStep.cost` and per-episode aggregates into trajectory metadata. The layer is what makes the pipeline body-agnostic: swapping the G1 coworker for SMPL-H [78] (Section 7.6) requires no adapter changes.

3.6 Bounded Workspace Reward Shaping

BiGYM’s task reward is sparse-plus-shaping: $r_t^{\text{task}} \in \{0, +1\}$ for the sparse component plus a small dense distance term where the task allows. Under sustained co-working disruption, this is insufficient to prevent an *evacuation* local optimum: the policy learns that moving the end-effector arbitrarily far from the human eliminates safety cost, and accepts the task-reward cost. The behaviour is observable as a robot that parks 2–3 m from the workspace and stays there.

`safety_bigym` adds a bounded penalty for excess distance between the end-effector and the task object:

$$r_t^{\text{ws}} = -\beta \cdot \min\left(\max(0, \|p_t^{\text{ee}} - p_t^{\text{task}}\| - r_{\text{ws}}), c_{\text{ws}}\right), \quad (3.1)$$

with $r_{\text{ws}} = 0.4$ m, $\beta = 0.05$, $c_{\text{ws}} = 1.0$ m. The cap c_{ws} is critical and is derived in Proposition 4.1 as a value-support invariant of the C51 distributional critic; the bounded form is the one that *works*, and is therefore the form the benchmark ships with. The shaped reward is $r_t^{\text{task,shaped}} = r_t^{\text{task}} + r_t^{\text{ws}}$.

3.7 The Snapshot-Evaluation Harness

A practical benchmark needs a reproducible evaluation pipeline. The `safety_bigym benchmark_policy.py` harness takes any policy snapshot (a CQN-AS checkpoint, an ACT snapshot, or a user-supplied `Policy` subclass) and produces a CSV row per (task, disruption, observation-mode, seed) cell with the full safety-metric schema of Section 3.8: mean and percentile separation, three SSM-flavour rates, time in near-proximity bins, time-to-first-violation, and per-region PFL violation counts. The harness is what makes Tables 5.5, 5.7, and 5.9 reproducible from snapshots alone.

The harness is delivered alongside the benchmark and is documented in the project’s `README.md`; see Section 5.6 for the smoke-gate command that verifies an installation.

3.8 The Three Safety-Metric Flavours

The ISO 15066 SSM formula (Equation 2.1) is velocity-dependent and, under the standard’s conservative $v_h = v_{h,\text{max}} = 1.6$ m/s human-velocity cap, the resulting required-separation distance S_p varies between ~ 0.3 m and ~ 5 m on our environment’s robot speeds. The conservative S_p over-fires on the kinematic scale of a domestic-manipulation scene: random-policy rollouts in `safety_bigym` register a $\sim 93\%$ SSM violation rate, more a function of robot-arm capability than of actual human proximity. A direct “SSM compliance rate” number is therefore not the canonical thesis metric.

We expose *three* flavours of the relevant signal at every env-step:

ssm_violation (boolean) The conservative formal ISO 15066 check: S_p computed with $v_h = v_{h,\text{max}}$, observed v_r , and the geometric closest-pair distance. Kept for ISO traceability and historical comparability against earlier experimental data.

ssm_violation_actual (boolean) S_p computed with observed v_h and observed v_r . Formal ISO compliance *under the realised motion*; this is the reportable ISO number when the deployment can measure v_h .

proximity_violation (boolean) Geometric: min-separation $< \tau_{\text{prox}}$, with $\tau_{\text{prox}} = 0.3$ m matching the Phase 2 safety-filter dataset labelling threshold. This is the *exposure* measure: how often human and robot are close, regardless of who closed the distance and how fast anyone is moving. It is the most directly comparable number to the human-aware-robotics literature.

The three flavours are not redundant: a fast robot at 1 m separation from a still human registers `ssm_violation` but not `proximity_violation`; a still robot at 0.25 m from a still

human registers `proximity_violation` but not `ssm_violation_actual`. They are also not equally *attributable to the robot*: Chapter 5 shows that $\approx 42\%$ of time-in-proximity on these scenes is the coworker approaching an already-stationary robot, a component no robot policy can remove, whereas the velocity-adaptive SSM margin is a function of the robot’s own speed and is fully robot-controllable. The evaluation protocol therefore treats the two as distinct *axes* with explicit ownership, proximity as the (partly exogenous) exposure axis on which policy-level avoidance is judged, and `ssm_violation_actual` as the robot-controllable axis on which runtime filters are judged, and reports *both* for every configuration in every results table (Section 5.5).

3.9 Calibrating the Proximity Threshold

The threshold τ_{prox} is the single most consequential parameter of the proximity (exposure) axis, so we fix it by two speed-independent analyses rather than by intuition.

Contact geometry. Rendering the coworker at controlled separations (robot frozen in its reset pose, the coworker pelvis slid along the approach axis; Figure 3.1) shows the collision geometries interpenetrating at small min-separations (the 0.09 and 0.21 m columns), with a clearly visible gap open by ≈ 0.31 m. A usable threshold must therefore sit above the contact band, so that a violation *precedes* contact rather than coinciding with it; this is the geometric half of the case for $\tau_{\text{prox}} = 0.3$ m.

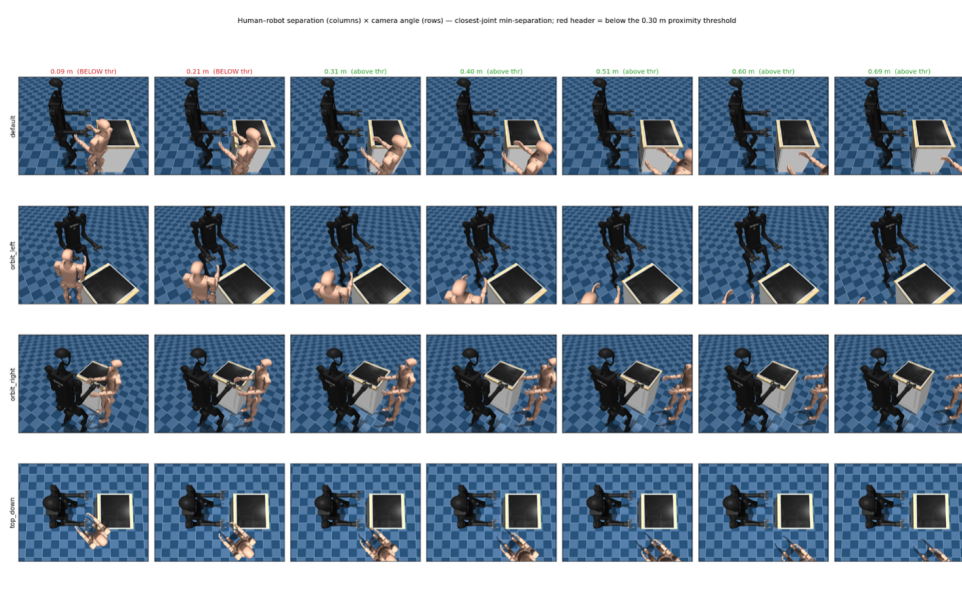


Figure 3.1: Contact-geometry calibration of τ_{prox} . The H1 robot (dark) and the G1 coworker (light) are rendered at seven controlled closest-joint separations (columns) from four camera angles (rows), with the `saucepan_to_hob` table between them. Red column headers are below the 0.30 m proximity threshold: at 0.09 and 0.21 m the body geometries visibly interpenetrate, whereas by 0.31 m (green) a clear gap has opened. This confirms that 0.3 m sits just above the physical contact band, so a proximity violation fires *before* contact, complementing the empirical-CDF knee of Table 3.1.

Empirical distribution. Because the proximity-violation rate at threshold τ is exactly the empirical CDF $P(\text{min-separation} < \tau)$, the threshold can be read off the separation distribution directly. On the G1 stage-2 baseline (3 seeds, 20 episodes, 30,000 steps) the distribution is bimodal, a near-contact mode at 0–0.2 m plus a far mid-range and patrol tail, with a knee at $\tau \approx 0.2\text{--}0.3$ m (Table 3.1); the knee is consistent across two independently trained policies, so it reflects scene contact geometry rather than a single controller.

We fix $\tau_{\text{prox}} = 0.3$ m: it sits at the knee, ~ 0.2 m above the contact band, and captures the genuine near-contact population without absorbing the benign mid-range proximity that a 0.5 m

Table 3.1: Proximity-violation rate as a function of the threshold τ (the empirical CDF of min-separation on the G1 stage-2 baseline, 3 seeds $\times$ 20 episodes, 30,000 steps). The rate climbs steeply once τ crosses into the benign mid-range, which is why a 0.5 m or 1.0 m threshold would absorb a large benign population.

τ (m)	0.20	0.30	0.40	0.50	0.60	0.75	1.00
Violation rate	20.1 %	25.3 %	29.9 %	35.8 %	39.5 %	47.0 %	62.9 %

(36 %) or 1.0 m (63 %) threshold would. Being purely geometric it is speed-independent, so the proximity-violation rate is comparable across policies, tasks and disruption bands, and it is held fixed across every experiment in this report. The $\approx 25\%$ near-contact rate this calibration rollout induces (the headline per-episode benchmark, Table 5.7, reports 0.296 for the same baseline; the small difference is per-step versus per-episode aggregation over a different episode draw) is precisely the near-contact behaviour the constrained policy and the runtime filters are meant to reduce.

3.10 Continuous Cost Signals

The wrapper computes two continuous, anticipatory cost signals from the ISO 15066 margins:

$$c_t^{SSM} = \max\left(0, 1 - \frac{m_t^{SSM}}{d_{\text{buffer}}}\right), \quad (3.2)$$

$$c_t^{PFL} = \max\left(0, f_t^{\text{ratio}} - 0.8\right), \quad (3.3)$$

$$c_t = \min\left(1, \max(c_t^{SSM}, c_t^{PFL})\right), \quad (3.4)$$

where m_t^{SSM} is the instantaneous SSM-actual margin (positive when safe, negative when violating), $d_{\text{buffer}} = 0.3\text{ m}$ is the anticipation buffer, and f_t^{ratio} is the force-ratio from Equation 2.2. The aggregator takes the worst-case across both criteria and saturates at $c_t = 1$.

Why continuous rather than binary. A binary indicator $c_t \in \{0, 1\}$ on violation supplies only *post-hoc* gradient signal. The policy learns that crossing the threshold is bad but receives no information about how close to the threshold it is approaching. The continuous formulation in Equations 3.2–3.3 gives a non-zero cost before violations occur: a SSM-actual margin of 0.15 m carries a cost of ~ 0.5 , and any margin beyond the 0.3 m buffer carries zero. We adopt it for this gradient richness. We note in advance, however, that the cost-signal *form* turns out *not* to be the operative variable: Experiment E3.1 (Section 5.12.1) shows that at a tight budget both the continuous and the binary forms collapse, because the binding quantity is the Lagrangian budget relative to the task’s inherent cost, not the encoding of the cost. The continuous form is retained as the more principled design, not because it is shown to dominate the binary one.

The choice of $d_{\text{buffer}} = 0.3\text{ m}$ matches the typical ISO 15066 SSM robot-stopping contribution at the H1’s observed end-effector speeds; the 0.8 threshold in Equation 3.3 matches the 20 % headroom that ISO 15066 Annex A motivates for “approaching the limit”.

The PFL contact-detection limitation. PFL flag computation reads MuJoCo’s `data.ncon` for human–robot contact pairs. BiGYM’s runtime robot-attachment in MOJO currently produces `data.ncon = 0` for these pairs despite geometric overlap, so $c_t^{PFL} \equiv 0$ throughout the project. The wrapper exposes a `use_pfl` flag wired through every component; the flag can be flipped on once the contact-detection issue is resolved without architectural change. **All quantitative compliance claims in this report are qualified as SSM/proximity-only** (Section 6.2.1).

3.11 The coworker Disruption: Design and Rationale

The dominant deployment pattern for human-aware manipulation is *sustained co-working*: the human is in the workspace for the whole task, not for a one-shot intrusion. The original BiGYM

distribution had no human at all; legacy disruption types in the `safety_bigym` ecosystem (INCIDENTAL, SHARED-GOAL, DIRECT, OBSTRUCTION, RANDOM-PERTURBED, CONTACT) all encoded a single intrusion event per episode and are inadequate as a training distribution for our setting. We introduce `DISRUPTION.COWORKER` as the primary distribution for both training and evaluation.

3.11.1 Design Goals

The `COWORKER` disruption was designed against three concrete targets:

1. *Realism*. The human’s behaviour should resemble what a real co-worker would do in a shared kitchen: enter the workspace, approach the task area, occasionally reach toward the task object or toward the robot’s own arm, and occasionally step away for a moment. Episodes should feel like minutes of collaboration, not seconds of intrusion.
2. *Sufficient disruption to expose unsafe behaviour*. If the human’s reaches are rare or always far from the robot, a fast-but-careless policy is safe by luck and the benchmark cannot discriminate. The `COWORKER` band tightens the closest *pelvis-to-pelvis* approach (0.60–0.95 m) and the `NEAR` dwell (9–14 s); the reaching arm extends well inside the body-centre separation, so the closest geometry pair routinely enters the $\tau_{\text{prox}} = 0.3\text{ m}$ zone. The resulting baseline `ep_proximity_violation_rate` of 0.296 (Section 5.10) is high enough to expose a 20 + % improvement as statistically significant.
3. *Compositional disruption modes*. A real co-worker exhibits at least three distinct behaviours over a session: staying put while reaching; walking in then staying; walking in, leaving briefly, and returning. The `COWORKER` sampler covers all three but weights them unequally, drawing the most demanding mode (patrol) the large majority of the time so that the training distribution concentrates on the hardest behaviour while still exercising the simpler ones.

3.11.2 Three Trajectory Modes

A `COWORKER` scenario samples one of three trajectory modes, with the patrol mode drawn $\sim 90\%$ of the time and the two simpler modes sharing the remaining $\sim 10\%$:

stationary G1 spawns at `NEAR` distance and stays put. Arm reaches throughout the episode.

approach-loiter-depart G1 spawns far, walks in to `NEAR`, stays put. Arm reaches after arrival.

coworker-patrol G1 walks in to `NEAR`, then cyclically departs to an `AWAY` position (90° – 270° offset, ~ 2 – 3 m), loiters, returns to a *new* `NEAR` position (different angle, slightly different distance), and repeats. Arm reaches only while at `NEAR`.

The patrol mode is the most realistic operational pattern and is therefore the dominant mode at full strength; the two simpler modes are retained both as occasional variety in the headline distribution and as the bulk of the earlier curriculum stages (Section 4.5), which ramp up to the patrol-dominant mixture.

3.11.3 Arm State Machine and Reach Gate

Independently of trajectory, the G1’s reaching arm runs a four-state cycle, `EXTEND` $\rightarrow$ `HOLD` $\rightarrow$ `RETRACT` $\rightarrow$ `IDLE` (fractions 0.15/0.20/0.15/0.50 of a period $T \in [1.3, 2.2]\text{ s}$ in the train band), sampling per cycle whether to reach for the robot’s end-effector or the task object (`target_mix_p_ee`), with the target re-resolved each step so the reach tracks a moving robot. A *reach gate* suppresses the IK during patrol’s `AWAY` phase. Because the arm idles half of each cycle and patrol periodically vacates the `NEAR` zone, the task remains completable throughout: the success-rate axis rewards progress in those windows over a trivial “freeze until the human leaves” strategy.

3.11.4 Train and Eval Parameter Spaces

Five COWORKER axes are exposed as continuous `ParameterSpace` ranges, with train and eval distributions configured separately for the OOD generalisation experiment (E5.2):

Table 3.2: The COWORKER train and eval `ParameterSpace`. Train is moderately tight, deliberately set to produce a high baseline violation rate (so a 20+ % safety improvement is statistically detectable). Eval is wider, capturing the generalisation question. The 0.60 m closest-approach floor is the smallest body distance that keeps the G1 mocap-pelvis collision capsule clear of the H1 pelvis collision capsule; tighter values generate spurious contact forces and are pathological.

Axis	Train (<code>coworker_train</code>)	Eval (<code>coworker_eval</code>)
Closest-approach distance	0.60–0.95 m	0.6–1.8 m
Reach period T	1.3–2.2 s	3.0–9.0 s
P (reach EE)	0.45–0.72	0.1–0.9
Near-loiter dwell time	9.0–14.0 s	4–16 s
Walk speed	1.0–1.4 m/s	0.6–2.2 m/s

3.12 The Task Suite: Three Tasks, Two Co-location Regimes

The evaluation uses three BiGYM tasks. `saucepan_to_hob` is the primary deep-dive task; `dishwasher_close` and `drawers_open_all` complete the suite. The saucepan choice is motivated by:

1. **Long horizon for violation opportunity.** `saucepan_to_hob` is a pick-and-place task with a vertical clearance constraint; episodes last 30–50 s, several reach cycles of the COWORKER arm. Shorter reach-target tasks end before the coworker has cycled enough to expose representative unsafe behaviour.
2. **Workspace overlap with the coworker.** The saucepan pickup and the hob target are both within the H1’s primary reach envelope, which is also where the G1 reaches when the reach target is the task object. Episode trajectories therefore pass through the same volume the coworker occupies, exposing real SSM violations rather than incidental closeness.
3. **BiGym-RL competence.** Of the four candidate BiGYM tasks we screened with ACT baselines, `saucepan_to_hob` combines the longest horizon with the greatest workspace overlap, and the unconstrained CQN-AS agent (Section 5.10) reaches a strong `success_rate` on it after the curriculum (Section 4.5). The unconstrained baseline is a non-trivial reference: any safety improvement we report is against a competent task policy, not a policy that was failing the task anyway. CQN-AS task-performance on BiGYM is documented in [4].

Two co-location regimes. Although all three tasks run under the *same* COWORKER disruption distribution, the task geometry and the policy’s motion pattern make the resulting human–robot co-location qualitatively different, and this difference turns out to be the variable that governs which safety mechanism works (Chapter 5). On `saucepan_to_hob` the coworker’s reach destination coincides with the robot’s working volume for most of the episode: the co-location is *persistent*, with the closest geometry pair inside the interaction envelope nearly continuously. On `dishwasher_close` and `drawers_open_all` the close encounters arrive in *bursts*, the coworker’s patrol cycle and the appliance-facing geometry leave genuinely-safe windows between approaches: co-location is *intermittent*. We do not assert this distinction from task descriptions alone; it is *measured* in Section 5.15 via the activity rate of a separation-triggered runtime filter, which is the operational definition used throughout (persistent $\approx$ the filter would be active most of the time; intermittent $\approx$ it would not).

3.13 Mock BodySLAM++: Calibrated Perception Noise

Training-time runs use ground-truth human position; deployment-time runs use a noisy estimate. To characterise the perception sim-to-real gap (RQ1, RQ4), we expose a configurable `BodySLAMWrapper` with three modes:

off No human-state channel in the observation.

oracle Ground-truth human position injected as `human_pos_estimate` (6D: position + occluded flag + staleness counter + confidence).

noisy The full mock-BodySLAM++ pipeline, calibrated against the published characteristics of [25].

The noisy pipeline applies, in order:

1. Ornstein-Uhlenbeck temporally-correlated position noise ($\alpha = 0.9$, $\sigma = 0.05$ m, stationary std ≈ 0.115 m), calibrated against BodySLAM++’s reported positional accuracy [25].
2. 3-step latency buffer (≈ 60 ms at 50 Hz control, matching the 15+ FPS perception clock typical of monocular visual-inertial pose estimation).
3. Stochastic tracking dropout ($p = 0.02$ /step) with staleness counter; OU state continues to update during dropout, so recovery is smooth rather than discontinuous.
4. Optional ray-cast occlusion from the H1’s head camera (opt-in).

Demo-replay path. Cached BiGYM demonstrations [6] were recorded *without* a coworker; their `human_pos_estimate` would be missing if read live. The wrapper supports a `mode=demo_replay` path that synthesises `human_pos_estimate` from an `AMASSDemoPositionProvider` [80] playing back a real motion clip, so a behaviour-cloning pretraining phase sees realistic human state at training time without a train→eval distribution shift on the channel.

3.14 Demonstrations and the Demo-Conversion Pipeline

CQN-AS [4] is a demo-driven RL agent (`num_demos > 0` assumed at construction); demo-free runs on long-horizon manipulation never discover the sparse reward and collapse to workspace evacuation. The demo path uses the raw, non-safety-wrapped BiGYM environment to match `DemoStore`’s class-name lookup, fetches 10–36 cached demonstrations, and converts each to `ExtendedTimeSteps` carrying the synthesised `human_pos_estimate` channel and a safe-side placeholder $c_t = 0$ (the cached demos contain no live human; the curriculum of Section 4.5 makes this benign in practice). The most consequential of the action-normalisation pitfalls fixed during integration (Section 4.8): CQN-AS’s normalisation must use *demo-derived* action statistics so the live and demo distributions share a binning.

4 The Hybrid Safety Architecture and Its Implementation

4.1 Overview: From Problem to Solution

Chapter 3 introduced `safety_bigym` as the benchmark and Section 3.1 listed six concrete properties the benchmark is designed to test. The unconstrained CQN-AS [4] baseline that ships with BIGYM fails almost all six: it has no representation of human position, no SSM margin, no constraint that would distinguish a task-completing trajectory from one that endangers the coworker, and Section 5.10 will show that it registers proximity violations on $\approx 30\%$ of steps (mean separation only 0.13m, with the worst approaches within a few millimetres), far above what any plausible co-working deployment would accept.

This chapter presents **our solution to that gap**: the Hybrid Safety Critic, a two-arm safety architecture that turns the unconstrained imitation-or-RL policy into an ISO 15066-aware collaborator. The architecture has two mechanisms with deliberately different roles:

1. A **training-time mechanism** that *internalises* safety into the policy via a value-based Lagrangian formulation of the continuous cost signal. At deployment, the policy is used as-is; the safety knowledge is baked into the learned Q -functions (Section 4.4).
2. A **deployment-time mechanism**: a runtime *filter slot* that intercepts the policy’s proposed actions. The slot is mechanism-agnostic, and we specify and evaluate the complete reactive taxonomy through it: an offline-trained, frozen safety value function (SVF) used as a veto, the original design (Section 4.2); two closed-form alternatives, a graded ISO 15066 speed-scaling rule and a model-based geometric dodge (Section 4.3); and the *critic-gated speed-scaling* combination, the SVF deciding *when* to intervene and the graded scaler deciding *how* (Section 4.3.3), which is the Hybrid Safety Critic’s runtime arm.

Both mechanisms share a network shape and an observation interface, which keeps the two safety critics directly comparable and leaves SVF reuse available as an option, though the constrained policy’s cost critic is in fact cold-started (Section 4.4.9); they differ in training objective and operational role. The full pipeline at deployment is sketched in Figure 4.1.

4.2 The Offline Safety Value Function (Phase 2)

4.2.1 Dataset and Labelling

The SVF is trained offline on a dataset of transitions collected by rolling out CQN-AS policy snapshots from the base-policy curriculum (Section 4.5) in the COWORKER distribution under the `noisy` observation mode. Earlier iterations of the critic mixed random-policy and demonstration transitions; the current critic is collected from policy snapshots alone. The reasoning is that the regions a deployed policy actually visits are precisely the regions the filter must judge accurately, and snapshot rollouts sample exactly that on-policy distribution, including the imitation-learning artefacts a competent-but-imperfect policy exhibits under sustained co-working disruption. The filter is *noisy-native*: it is trained and evaluated on `noisy` human-state estimates, and on ground-truth (`oracle`) input its Q -estimates collapse and it intervenes almost everywhere, so all filtered evaluations are reported on `noisy` (Section 5.3). The exact transition count for the current critic is 37,883, of which 16.8% are labelled unsafe ($r^{\text{safe}} = 0$) at the $\tau = 0.3\text{m}$ proximity threshold; the full figures are in the Phase 2 results write-up.

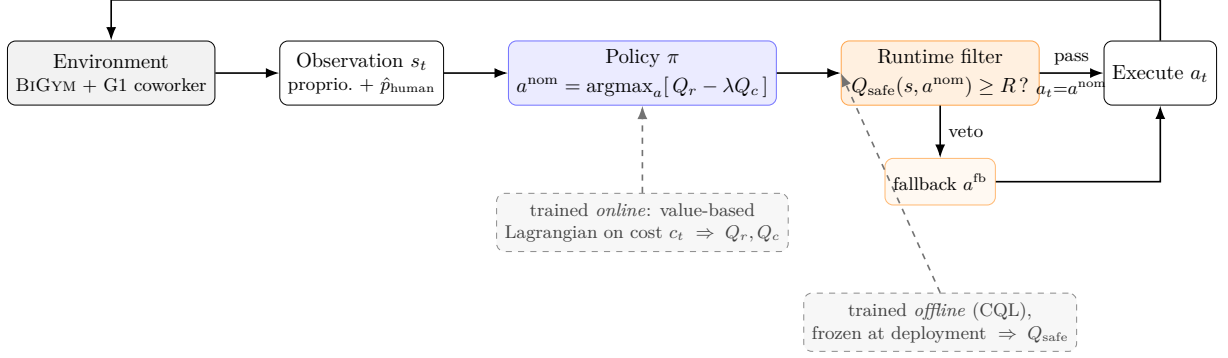


Figure 4.1: The Hybrid Safety Critic at deployment: a two-arm architecture. At each step the Lagrangian-trained policy proposes the nominal action $a^{\text{nom}} = \arg\max_a [Q_r(s, a) - \lambda Q_c(s, a)]$; a runtime filter slot passes it through when its safety check holds and otherwise modifies or replaces it (veto, dodge, graded speed-scaling, or critic-gated speed-scaling, Sections 4.2–4.3.3). The two arms have deliberately different jobs: the constrained policy internalises safety *proactively* during training, while the filter is a *reactive* runtime mechanism. Chapter 5 shows that which arm carries the safety burden is governed by the co-location regime: under persistent co-location only the policy reduces exposure gracefully (Table 5.7), while under intermittent co-location the critic-gated filter is the working arm (Section 5.14.4).

Each transition is labelled with the binary safety reward

$$r^{\text{safe}} = 1 - \mathbb{I}[\text{proximity_violation}], \quad (4.1)$$

using the geometric proximity-violation flavour of Section 3.8 at the calibrated threshold $\tau_{\text{prox}} = 0.3\text{m}$ (the calibration is derived in Section 3.9). We label on `proximity_violation` rather than `ssm_violation_actual` for two reasons. First, the conservative-ISO SSM flag is saturated: it fires on $\sim 93\%$ of random rollouts and remains high even for competent policies because it assumes the worst-case human velocity, so it carries almost no gradient for a value function to learn. Second, and by design, labelling on geometry keeps a clean separation of concerns between the two safety mechanisms. The runtime filter is made responsible for the *geometric* danger of being too close, while the velocity-dependent SSM response is left to the cost signal and the constrained policy (Section 4.4). Coupling the filter’s training target to instantaneous velocity would make its label depend on the very action it is meant to veto, conflating “how dangerous is this configuration” with “how fast is the robot moving”, the latter being a decision the policy, not the filter, should own.

Because the violating class is the minority, a `WeightedRandomSampler` over-samples it to a 1:1 ratio of safe to violating transitions per batch, so the critic is not dominated by the safe majority.

A second, task-agnostic critic for the intermittent-co-location tasks. The same pipeline produces the critic used on `dishwasher_close` and `drawers_open_all`: a single shared SVF trained once on 52,794 transitions collected from curriculum-policy rollouts on both tasks under COWORKER disruption, labelled by geometric *near-contact* (min-separation $< 0.10\text{m}$). The observation specification (proprioception + human-position estimate + action) is identical across the two tasks, so one critic serves both, evidence that the learned safety signal is task-agnostic at matching embodiment. The tighter 0.10m label (versus the saucepan critic’s 0.30m) makes this critic a *near-contact* detector rather than a proximity detector; an ablation with looser, earlier-warning labels ($0.20 / 0.30\text{m}$) confirms the selective near-contact label is the better gate (Section 5.14.5).

4.2.2 Architecture and Training

The safety critic is a 3-layer MLP with hidden sizes $[256, 256, 256]$ and ReLU activations. The input is the concatenation of robot proprioception, the mock-BodySLAM++ 6D human-position

estimate, and the proposed action. The output is bounded to $[0, q_{\max}]$ with $q_{\max} = 1/(1 - \gamma)$ via a scaled sigmoid, which *mathematically* prevents Bellman overestimation: Q cannot exceed the maximum possible discounted return of a safe trajectory.

Training combines a Bellman residual with the CQL [56] regulariser:

$$\mathcal{L} = \mathcal{L}_{\text{Bellman}} + \alpha_{\text{CQL}} \cdot (\mathbb{E}_{a \sim \text{uniform}}[Q(s, a)] - \mathbb{E}_{a \sim \mathcal{D}}[Q(s, a)]), \quad (4.2)$$

with $\alpha_{\text{CQL}} = 5.0$ as the headline value, Polyak target update $\tau = 0.005$. Full training hyperparameters in Appendix A.1.

4.2.3 No-Pixel Critic: Justification

The filter does not consume RGB, for three reasons: *compute* (a pure-MLP critic costs ~ 0.3 ms per forward against a ~ 20 ms control budget, versus a substantial CNN-encoder fraction); *robustness* (no lighting/texture domain shift, only the upstream pose estimator’s calibration [25]); and *compositionality* (a no-pixel filter wraps any policy whose observation includes proprioception, the human-position estimate, and the proposed action).

4.2.4 Runtime Wrapper

At deployment, the `SafetyFilterWrapper` intercepts the proposed action and compares the safety value against a threshold R :

$$a_t^{\text{exec}} = \begin{cases} a_t^{\text{nom}} & \text{if } Q_{\text{safe}}(s_t, a_t^{\text{nom}}) \geq R, \\ a_t^{\text{fb}} & \text{otherwise.} \end{cases} \quad (4.3)$$

R is calibrated by sweep over a held-out validation set (Experiment E2.3, Section 5.12.3); the headline operating point for the G1 critic on the COWORKER distribution is $R = 2.25$. The intended role of the veto at this operating point is an ISO 15066 SSM *velocity backstop* rather than a proximity reducer: much of the residual proximity is the human walking up to an otherwise stationary robot, which no veto of robot actions can prevent; Section 5.12.3 quantifies the resulting limits.

The Phase 2 fallback is zero-velocity braking on the floating base and on every joint velocity actuator. This can produce abrupt motion mid-trajectory; proportional damping is the natural upgrade and is deferred to future work (Section 7.5).

4.3 Closed-Form Runtime Filters: Speed-Scaling and Geometric Dodge

The filter slot of Figure 4.1 is mechanism-agnostic: anything that maps the state and the proposed action (s_t, a_t^{nom}) to an executed action can occupy it. The learned SVF veto above is its learned instantiation. Two *closed-form* mechanisms, specified here and evaluated alongside the veto in Chapter 5, complete the reactive taxonomy: stop (veto), slow (speed-scaling), and move away (dodge). Both consume the same $\hat{p}^{\text{human}}$ estimate as the SVF and contain no learned component.

4.3.1 Graded ISO 15066 Speed-Scaling

ISO 15066 SSM’s intent is graded: the robot should *slow* as separation shrinks, not merely stop at a threshold. The speed-scaling filter implements this directly. At each step a scale factor is computed from the closest-pair human–robot separation sep_t ,

$$\sigma_t = \text{clip}\left(\frac{\text{sep}_t - d_{\text{stop}}}{d_{\text{slow}} - d_{\text{stop}}}, 0, 1\right), \quad a_t^{\text{exec}} = q_t + \sigma_t (a_t^{\text{nom}} - q_t), \quad (4.4)$$

where q_t is the current joint position for the absolute-mode arm dimensions (so the per-step *motion* from the current pose is scaled) and zero for the delta-mode floating base (so the

commanded increment is scaled directly). Above d_{slow} the filter is the identity ($\sigma_t = 1$, the action is unchanged); at or below $d_{\text{stop}} = 0.15$ m the robot holds position ($\sigma_t = 0$); in between, speed falls linearly with separation. The slow radius d_{slow} is the single operating-point knob, swept over $[0.25, 0.60]$ m with the headline at 0.40 m; an intervention is any step with $\sigma_t < 1$. Unlike the veto, this filter preserves the *direction* of the policy’s motion and modulates only its magnitude, which is why it can maintain the graded velocity margin the standard prescribes where a binary stop-or-go veto cannot (Section 5.12.4).

4.3.2 Model-Based Geometric Dodge

The second closed-form mechanism is a geometric control-barrier-style fallback: when the closest-pair separation falls below a target d_{target} , the filter adds a correction step directed *away* from the human, proportional to the violation depth ($d_{\text{target}} - \text{sep}_t$) and capped per step. Two variants differ in which part of the robot dodges: the *base-dodge* pushes the floating base along the escape direction and leaves the arm untouched, while the *end-effector retract* (“flinch”) keeps the base planted and retracts the end-effector along the escape direction via a damped-Jacobian step. d_{target} is swept over $[0.30, 0.55]$ m. These variants exist to test whether a *dodging* reactive response escapes the limits of the *stopping* one; Section 5.12.5 shows it does not (it trades the veto’s freeze for a flee). The veto and the dodges are reported as taxonomy points (Table 5.8), with operating-point values for all mechanisms in Appendix A.3.

4.3.3 Critic-Gated Speed-Scaling: The Hybrid Safety Critic’s Runtime Arm

The unconditional speed-scaler of Section 4.3.1 pays its task cost at *every* close approach, including those where the situation is genuinely safe (the human is close but stationary, or moving away, or the robot is barely moving). The learned SVF of Section 4.2 knows the difference; the binary veto is simply the wrong way to act on that knowledge. The *critic-gated speed-scaling* filter splits the decision:

$$a_t^{\text{exec}} = \begin{cases} a_t^{\text{nom}} & \text{if } Q_{\text{safe}}(s_t, a_t^{\text{nom}}) \geq R \quad (\text{critic says safe: full speed}), \\ q_t + \sigma_t (a_t^{\text{nom}} - q_t) & \text{otherwise} \quad (\text{critic flags risk: graded slow-down, Eq. 4.4}). \end{cases} \quad (4.5)$$

The critic decides *when* to intervene; the speed-scaler decides *how*. Both halves matter. Replacing the graded slow-down with a hard veto breaks the action coherence of a chunked policy (Section 5.14.2); removing the gate makes the scaler fire on every close approach and pay the full unconditional task cost. The gate threshold R is the single operating-point dial: $R \rightarrow 0$ recovers the unfiltered policy, $R \rightarrow \infty$ recovers unconditional scaling, and intermediate values trace a frontier between them (Section 5.14.4). Whether that frontier *bends* toward a high-success/low-violation corner or merely *slides* between its two endpoints is precisely the regime-dependent question answered in Section 5.15.

4.4 External Baseline: Worst-Case SAC

To position the value-based Lagrangian against the published safe-RL state of the art rather than only against our own ablations, we faithfully reimplement WCSAC [1, 48], the canonical distributional safe actor-critic, on the same 76-DOF tasks (`agent=wcsac` on the same training stack). WCSAC maintains a Gaussian distributional cost critic and constrains CVaR_α of the discounted cost return ($\alpha = 0.9$) to a budget d via an adaptively-tuned multiplier. Two faithfulness points matter. First, WCSAC is actor-critic, so it cannot load the critic-only CQN-AS warm-start snapshots; it trains *from scratch* (`num_demos = 0`, 150k frames), which is the honest configuration and is reported as such, the comparison measures the published method as a practitioner could actually deploy it on these tasks, not a demo-warm-started variant that does not exist in the literature. Second, because it trains demo-free, its action de-normalisation must use identity statistics; the evaluation harness originally forced demo-derived statistics onto every

snapshot, a train/deploy mismatch that produced plausible-but-wrong numbers and was fixed and cross-validated before any WCSAC result was recorded (the same class of silent confound as Section 4.9). Results in Section 5.14.1.

4.4.1 Why Value-Based

CQN-AS [4] is *critic-only*: the policy is implicit, $\pi(s) = \operatorname{argmax}_{a \in \mathcal{A}_{\text{bin}}} Q(s, a)$, over a coarse-to-fine discretisation of the joint action space. The classical actor-critic Lagrangian formulation [27, 28] updates a policy network π_θ to maximise $\mathbb{E}[Q_r(s, a)] - \lambda \mathbb{E}[Q_c(s, a)]$, which presumes that π_θ exists as a separately-trained network. There is no π_θ in CQN-AS. The Lagrangian must be re-expressed in value-based form.

The starting point is the constrained objective

$$\max_{\pi} J_r(\pi) \quad \text{s.t.} \quad J_c(\pi) \leq d, \quad J_r(\pi) = \mathbb{E}_{\pi} \left[\sum_t \gamma^t r_t \right], \quad J_c(\pi) = \mathbb{E}_{\pi} \left[\sum_t \gamma^t c_t \right]. \quad (4.6)$$

For any fixed multiplier $\lambda \geq 0$, the Lagrangian relaxation is $\mathcal{L}(\pi, \lambda) = J_r(\pi) - \lambda(J_c(\pi) - d)$. The λd term is constant with respect to action choice, so a critic-only policy can implement the same trade-off at decision time by learning separate action-values

$$Q_r(s, a) \approx \mathbb{E} \left[\sum_t \gamma^t r_t \mid s_0 = s, a_0 = a \right], \quad Q_c(s, a) \approx \mathbb{E} \left[\sum_t \gamma^t c_t \mid s_0 = s, a_0 = a \right],$$

and selecting the action that maximises $Q_r - \lambda Q_c$. This is the key reproducibility point: *the policy improvement step is changed, not the Bellman target of the reward critic*. The reward and cost critics can therefore be trained on stationary targets, while λ acts only as a deployment-time or training-time selection parameter.

Three options were considered, in increasing principledness; the choice is itself an empirical comparison (Experiment E3.1).

4.4.2 Option A-value: Shaped-Reward Single Critic

The simplest re-formulation folds the cost into a modified reward and trains a single critic:

$$r_t^{\text{mod}} = r_t^{\text{task,shaped}} - \lambda \cdot c_t, \quad Q(s, a) \approx \mathbb{E} \left[\sum_t \gamma^t r_t^{\text{mod}} \right], \quad (4.7)$$

with action selection $a^* = \operatorname{argmax}_a Q(s, a)$. Its known pathology: the regression target is *non-stationary* in λ , which is itself non-stationary in the rolling cost, so the critic chases a moving target. Used here as a prototype baseline only.

4.4.3 Option B-value-mean: Dual Q-Functions on Mean Cost (Headline)

Decouple the reward and cost critics:

$$Q_r(s, a) \approx \mathbb{E} \left[\sum_t \gamma^t r_t^{\text{task,shaped}} \right], \quad (4.8)$$

$$Q_c(s, a) \approx \mathbb{E} \left[\sum_t \gamma^t c_t \right], \quad (4.9)$$

each regressing toward its own *stationary* target. The action selection combines them at decision time:

$$a^* = \operatorname{argmax}_{a \in \mathcal{A}_{\text{bin}}} \left[Q_r(s, a) - \lambda \cdot Q_c(s, a) \right]. \quad (4.10)$$

λ now enters only at selection, not at regression. The cost critic shares its network shape with the Phase 2 SVF, so SVF reuse is available, though here the cost critic is cold-started (Section 4.4.9). This is the **headline architecture** for this work.

4.4.4 Option B-value-CVaR: Distributional Cost Critic (future work)

A natural extension replaces the mean-cost critic with a distributional one: Q_c is replaced by either a Gaussian parametrisation (μ_c, σ_c) [1] or a quantile head $\{\theta_k\}_{k=1}^K$ [49], with action selection

$$a^* = \operatorname{argmax}_{a \in \mathcal{A}_{\text{bin}}} \left[Q_r(s, a) - \lambda \cdot \text{CVaR}_\alpha(Z_c \mid s, a) \right], \quad (4.11)$$

with $\alpha \in \{0.95, 0.99\}$. This would align training with the CVaR evaluation metric and answer the standard critique that mean-cost methods bound only $\mathbb{E}[Z_c]$; the cost head is already C51-shaped, but calibrating a rolling-CVaR target and re-verifying the support invariant (Section 4.4.5) for the cost distribution is non-trivial engineering. **We adopt B-value-mean as the headline and defer explicit-CVaR training to future work (Section 7.4)**, evaluating the mean-policy’s *achieved* CVaR as a diagnostic (Section 5.13.1).

4.4.5 The C51 Value-Support Invariant

CQN-AS uses a C51 distributional reward critic with fixed value support $[v_{\min}, v_{\max}]$ [40]. The Bellman target $T_z = r + \gamma \cdot \text{support}$ is clamped to this support at each update. Adding a dense shaped reward term such as Equation 3.1 introduces a discounted return whose value can fall outside the support; once the target is clamped, the critic loses the ability to distinguish values in the saturating region, and the gradient that should pull the end-effector back toward the task is clipped away.

Proposition 4.1 (Support-bounding invariant). *Let a C51 distributional critic have value support $[v_{\min}, v_{\max}]$, discount γ , and an additive shaped reward $r_t^{\text{shaped}} \in [-\beta \cdot c_{\text{ws}}, 0]$ with $\beta, c_{\text{ws}} > 0$. The shaped-reward contribution to the discounted return is bounded by $\beta \cdot c_{\text{ws}} / (1 - \gamma)$ in magnitude. The Bellman target clamp is informative (i.e., the support is not saturated) only if*

$$\beta \cdot c_{\text{ws}} / (1 - \gamma) \leq |v_{\min}|. \quad (4.12)$$

Proof. The geometric-series sum of the per-step shaping under the worst case gives $\sum_{t=0}^{\infty} \gamma^t \beta c_{\text{ws}} = \beta c_{\text{ws}} / (1 - \gamma)$, and the C51 target clamp clips below $v_{\min}$. $\square$ $\square$

Consequence. The original workspace shaping ($\beta = 0.2$, no cap on per-step excess distance) gave a worst-case discounted return of $-0.2/0.01 = -20$ against the default CQN-AS support of $[-2, +2]$, saturating every state outside ~ 0.5 m of the task. Training diverged: `ep_reward` fell monotonically from -78 to -775 over 50k frames as the policy parked further from the workspace and the support flattened. The fix is two-sided: cap the per-step penalty ($c_{\text{ws}} = 1.0$ m), reduce β to 0.05, and widen the support to $[v_{\min}, v_{\max}] = [-6, +2]$. With these settings the invariant holds ($0.05 \cdot 1.0 / 0.01 = 5 \leq 6$) and training proceeds normally; the next-attempt `ep_reward` reaches $\sim +1.8$ on stage 1 of the curriculum (Section 4.5).

The engineering lesson generalises beyond safety_bigym. It generalises to any distributional critic + dense shaping setting and applies as a checking principle independent of `safety_bigym`.

4.4.6 Per-Env-Step Cost Backup

CQN-AS executes *action sequences* of length K between policy queries (coarse-to-fine refinement). A natural but subtly wrong design is to backup the cost critic on the chunk-averaged cost $\bar{c} = (1/K) \sum_k c_{t+k}$. The policy can satisfy a mean-over-chunk budget while spiking violations *inside* a chunk. The cost critic must be backed up on *every* env-step’s c_t . Our training pipeline preserves c_t through the replay buffer at per-env-step granularity (Section 4.8).

4.4.7 PID Lagrange Multiplier Update

The Lagrange multiplier is updated by a PID controller [28] on the running constraint violation. Let m_t be the rolling per-episode cost mean and d the cost budget; the update is

$$e_t = m_t - d, \quad (4.13)$$

$$\lambda_{t+1} = \text{clip}(\lambda_t + K_I \cdot e_t + K_P \cdot e_t + K_D \cdot (e_t - e_{t-1}), 0, \lambda_{\max}). \quad (4.14)$$

PID is preferred over gradient ascent ($\lambda \leftarrow \lambda + \eta \cdot e_t$) because the integral term provides faster convergence to the constraint boundary while the derivative term damps oscillation [28]. We use $K_I = 10^{-3}$, $K_P = 10^{-2}$, $K_D = 0$, and $\lambda_{\max} = 100$. The clamp prevents the policy from collapsing into a fully-frozen “safe-and-useless” attractor when the cost budget is briefly violated.

PID auto-tuning is seed-unstable at the feasibility boundary; a fixed multiplier is the fix. The task’s inherent mean cost is $\approx 0.25\text{--}0.30$, so any budget $d \leq 0.2$ is infeasible (abandon-the-task is the only policy under it) and $d = 0.5$ is slack ($\lambda \rightarrow 0$); the single viable budget, $d = 0.3$, sits *at* the inherent cost, where the dual variable is decided by each seed’s stochastic cost trajectory rather than by design. Across three seeds at $d = 0.3$, λ converged to 0.000, 0.267, and 3.855, unconstrained, graceful, and windup-collapse respectively; the evidence is presented with the budget sweep in Section 5.12.2 (Figure 5.2). The fix is to remove the auto-tuning: freezing λ (zero PID gains, Q_c still trained and used in dual-action selection) makes all three seeds behave consistently and turns λ into a clean proximity–success knob; we sweep it and adopt $\lambda = 0.1$ as the headline ($\lambda = 0.27$ over-constrains, $\lambda = 0$ recovers the baseline; Figure 5.3). The lesson generalises: when the only feasible budget coincides with the task’s inherent cost, the dual variable carries too little signal to auto-tune, and a fixed or scheduled multiplier is preferable. The PID path remains in the codebase as the documented negative.

4.4.8 Filter During Training (optional)

The runtime safety filter of Section 4.2 can also be applied at training time, interposed between the proposed action and the environment, vetoing catastrophically unsafe proposals with the fallback [24, 55]. We report results without the training-time filter as the default; the filter-during-training ablation is deferred to future work (Section 7.5).

4.4.9 Cost-Critic Initialisation and Policy Warm-Start

Two initialisation choices matter here, and they point in opposite directions. The actor, the reward critic and the shared visual encoder are *warm-started* from the unconstrained Phase 1 base policy: the stage-1 snapshot of the curriculum (Section 4.5) is loaded so that constrained training begins from a task-competent policy rather than from scratch. The cost critic Q_c , by contrast, is *cold-started* (freshly initialised).

This is deliberate. Although Q_c shares its network shape with the Phase 2 SVF, the two carry opposite sign conventions (Q_c high means dangerous, Q_{safe} high means safe) and different learning targets (offline CQL value of a binary proximity label vs. online expected value of the continuous cost under the current policy); seeding Q_c from the SVF would initialise it on a surface the dual-Q selection step would have to unlearn. A `warm_start_from_svf` path (with the required sign-flip) remains in the codebase, unused in the reported runs. One loader subtlety matters for reproduction: the unconstrained Phase 1 snapshot contains no cost-critic keys, so the loader guards them explicitly, task weights load, Q_c stays freshly initialised.

4.5 The Base-Policy Curriculum

Direct training on the full COWORKER disruption distribution from a randomly-initialised CQN-AS agent fails. The empirical signature is a base policy that learns to evacuate the workspace within 5–10k frames and never attempts the task; `ep_reward` stays at the floor and `success_rate`

stays at 0. The cause is a *distribution-shift cascade*: cached BiGYM demonstrations contain no coworker, so behaviour-cloning teaches “go to the task, ignore everything”; at runtime, the live coworker physically blocks the task path, producing off-distribution states; the C51 critic has no reliable signal in those states and the policy drifts toward whatever physics + workspace shaping push it toward, which is away from the human and toward the workspace boundary.

The fix is a three-stage curriculum that resumes from each prior stage’s snapshot [42]. The environment is *stateless* with respect to training step, so the curriculum is implemented as three sequential training runs with snapshot resume. The `BodySLAMWrapper` channel and the obs width are preserved across stages by keeping a coworker in the scene at all stages (just distant in stage 0), so the architecture and replay buffer carry over cleanly.

The three stages are: *stage 0*, `coworker_idle` (G1 present but ~ 3 m off and never reaching, doubling as a “can the agent learn the task at all?” sanity check; 30k frames); *stage 1*, `coworker_easy` (moderate distance, occasional reaches; 30k); *stage 2*, the full `coworker_train` band of Table 3.2 (40k). Stages 0–1 saturate well before their budgets; the largest budget is reserved for the hardest distribution.

Why a curriculum (against alternatives). Domain randomisation [71] on the human velocity from step 0 was considered, but it addresses dynamics-estimate error, not the actual failure, never discovering the sparse reward; curriculum learning addresses the discovery problem [42, 43]. The empirical anchor: stages 0–1 complete with `ep_reward` $\sim +1.8$ (positive, near the +2 support ceiling, the unambiguous un-saturated-support signal; contrast the failed direct run’s $-78 \rightarrow -775$ divergence), and stage 2 provides both the unconstrained baseline on the real distribution and the Phase 3 warm-start.

4.6 Algorithm

4.7 Software Architecture

With the architecture and training loop specified above, the remaining design decisions are matters of engineering; the following sections document the ones that materially affected the reported results. The codebase is partitioned: `safety_bigym` (env layer + ISO 15066 wrappers + perception wrapper + scenario sampler), `safety_bigym/filters` (Phase 2 SVF: dataset, critic, runtime wrapper, sweeps), `safety_bigym/agents/cqn_as` (vendored CQN-AS [4] + adapter + Lagrangian agent), and a top-level training entrypoint `train_cqn_as.py`. Configuration is managed via Hydra; W&B is the experiment tracker; per-run JSONL metric dumps provide W&B-independent reproducibility. The snapshot-evaluation harness of Section 3.7 is the central tool for reproducible benchmark consumption.

Hardware target: a single NVIDIA A100 40 GB for training; CPU-only smoke gates for every script.

4.8 CQN-AS Vendor Integration

The upstream CQN-AS [4] reference implementation was vendored at commit 8cf806e and adapted to the `safety_bigym` setting. Eight non-trivial bugs and distribution mismatches were resolved during integration; the most informative are summarised here.

Demo distribution gap. Cached BiGYM demos contain no human, so demo-derived `human_pos_estimate` would be empty. The wrapper’s `demo_replay` mode injects an AMASS [80] clip during demo conversion so that demo and live obs share the same channel width under `bodyslam` $\in \{\text{oracle}, \text{noisy}\}$.

Demo–live action-stat sharing. CQN-AS normalises actions against `action_stats`; the live env and the demo replay must share one normalisation, or demos teach a policy that lives in

Require: Task reward $r_t^{\text{task,shaped}}$ (workspace shaping per Equation 3.1), cost signal c_t per Equation 3.4, cost budget d , PID gains (K_P, K_I, K_D) , optional Phase 2 filter $Q_{\text{safe}} + \text{threshold } R$

```

1: Warm-start the actor, reward critic  $Q_{r,\phi_r}$  and encoder from the Phase 1 stage-1 snapshot;
   initialise the cost critic  $Q_{c,\phi_c}$  from scratch (cold start)
2: Initialise replay buffer, demo replay buffer; load num_demos demonstrations
3:  $\lambda \leftarrow \lambda_0$   $\triangleright$  headline: fixed  $\lambda_0 = 0.1$  (PID gains zero); PID-tuned only in the seed-instability
   ablation
4: for each environment step  $t$  do
5:    $a_t^{\text{nom}} \leftarrow \operatorname{argmax}_{a \in \mathcal{A}_{\text{bin}}} [Q_r(s_t, a) - \lambda \cdot Q_c(s_t, a)]$ 
6:   if using training-time filter and  $Q_{\text{safe}}(s_t, a_t^{\text{nom}}) < R$  then
7:      $a_t \leftarrow a_t^{\text{fb}}$   $\triangleright$  fallback
8:   else
9:      $a_t \leftarrow a_t^{\text{nom}}$ 
10:  end if
11:  Step env; store  $(s_t, a_t, r_t^{\text{task,shaped}}, c, s_{t+1})$  in replay
12:  Sample batch (demos + replay)
13:  Update  $\phi_r$ : C51 Bellman regression on  $r^{\text{task,shaped}}$  per-env-step
14:  Update  $\phi_c$ : Bellman regression on  $c$  per-env-step
15:  if episode end and PID-tuning  $\lambda$  (ablation only) then
16:     $m \leftarrow$  rolling mean cost over recent episodes;  $e \leftarrow m - d$ 
17:     $\lambda \leftarrow \text{clip}(\lambda + K_I e + K_P e + K_D(e - e_{\text{prev}}), 0, \lambda_{\text{max}})$ 
18:  end if
   (headline runs keep  $\lambda \equiv \lambda_0$ ; see Section 4.4.7)
19: end for

```

Algorithm 4.1: Value-based Lagrangian constrained-RL training loop (headline: Option B-value-mean).

a different bin than the policy executes. The adapter overrides `self._action_stats` with the demo-derived statistics at construction.

C51 projection CUDA overflow. The vendored C51 target-projection built its per-row scatter offset with an integer `torch.linspace`, which CUDA computes via float32; past 2^{24} the offsets round (here the endpoint is $\approx 3.97 \times 10^7$ at our batch size), so the boundary row addressed out of bounds and triggered an opaque, data-dependent `index_add_` device-side assert that never reproduces on CPU (exact integer arithmetic). The fix is an exact `torch.arange(B) * N_atoms` offset plus defensive index clamps; the diagnostic playbook (pin the op with `CUDA_LAUNCH_BLOCKING=1`, `isfinite`-guard the batch, log index extents) is recorded in the repository. This bug also affects the cost critic, which reuses the projection.

Other gotchas. A hidden `tensorDict` version pin, a Python-3.12 `random.seed` type regression, and a custom `state_dict` round-trip for the non-`nn.Module` agent were also required; the full catalogue lives in the project’s documentation.

4.9 Making the Offline Filter Deployment-Faithful

A runtime safety filter is only as good as the match between the data it was trained on and the control loop it runs inside. The offline SVF is trained on transitions collected by replaying a policy snapshot through a hand-built collection environment; if that environment drifts from the RoboBase factory `_create_env` the agent actually deploys through, the critic is calibrated against actions and states the deployed robot never produces, and it over-vetoes. The deployment-faithful harness (Section 3.7) exposed four such drifts, each in a separately-replicated piece of `_build_live_env`, and all four had to be fixed before any filter number was trustworthy:

1. **Action de-normalisation.** Collection de-normalised against `env.action_space` rather than the agent’s demo-derived statistics, so the critic saw mis-scaled actions.
2. **Action-execution mode.** CQN-AS deploys with 16-step action sequences under temporal-ensemble blending; collection used receding-horizon `chunk[0]`. This mismatch alone drove a filtered-row intervention rate of 89.5 % at deployment against a 7.9 % sweep estimate.
3. **Control frequency (the root cause).** The collection env ran at 500 Hz rather than the deployed 20 Hz; the $25\times$ mismatch shrank each action’s effect so the collection policy never completed the task (0 % vs. 85 %) and the coworker barely approached within the truncated wall-clock.
4. **Coworker scenario drift.** The collection scenario came from a Python preset that had drifted looser than the YAML the deployment factory reads.

The validation that the fixes worked: the filterless ($R = 0$) sweep proximity rate, 0.286, matches the deployment benchmark baseline 0.296 to within sampling error; the earlier “31.7 % reduction at 21.6 % intervention” knee was an artefact of the broken collection. The general lesson, an offline safety component must be collected through the exact control loop it will be evaluated in, is a silent confound class invisible to within-collection sweeps; it directly underwrites every filter number in Chapter 5.

4.10 Metric Persistence and the Snapshot Harness

Per-step c_t flows from the cost-signal module through the env adapter’s `ExtendedTimeStep.cost` field and the replay buffer into the cost-critic Bellman target at *per-env-step* granularity (not per K -step chunk, Section 4.4.6). Every `_log()` call appends to a streaming `metrics.jsonl` plus an end-of-training `final_metrics.json` per run, giving tracker-independent reproducibility. The snapshot harness (`benchmark_policy.py`, Section 3.7) takes any policy snapshot and produces a CSV row per evaluation cell with the full safety-metric schema; it is the canonical source for every numerical entry in this report’s results tables.

5 Experimental Setup and Results

5.1 Tasks

Three tasks, spanning the two co-location regimes of Section 3.12. `saucepan_to_hob` is the persistent-co-location deep-dive: a long-horizon pick-and-place with a vertical clearance constraint whose trajectory overlaps the G1 coworker’s reach envelope, and on which the unconstrained CQN-AS baseline reaches a strong `success_rate` after the curriculum. `dishwasher_close` and `drawers_open_all` are the intermittent-co-location tasks: shorter-horizon appliance manipulation on which the same curriculum recipe produces competent unconstrained baselines (0.77 and 0.82 success respectively under COWORKER disruption). All headline evaluation is under the same COWORKER distribution and `noisy` perception unless stated.

5.2 Disruption Distributions

Training uses `Disruption.coworker` sampling from the `coworker_train` `ParameterSpace` (Table 3.2). Evaluation uses the same distribution for in-distribution numbers (RQ1, RQ2, RQ3) and the wider `coworker_eval` distribution for the OOD generalisation experiment (RQ4, E5.2).

5.3 Perception Mode (oracle vs. noisy)

The `BodySLAMWrapper` of Section 3.13 exposes three modes for the `human_pos_estimate` channel: `off` (no channel), `oracle` (ground truth), and `noisy` (the calibrated mock-BodySLAM++ pipeline). The per-experiment policy is uniform: every *policy* trains on `oracle` and is evaluated on `noisy`; every *filter* (SVF dataset, sweeps, gated runs) is collected and evaluated on `noisy` throughout; E3.6 is the deliberate exception that evaluates the same checkpoints under both modes to measure the perception gap; and WCSAC trains and evaluates on `oracle` (flagged as an upper bound where reported).

Rationale. The deployment-relevant evaluation condition is `noisy`, so every headline number in this report is a `noisy`-eval number (the one exception, WCSAC, is flagged where it appears). Training the *policy* on `oracle` isolates the treatment variable, any safety improvement is attributable to the constraint rather than to noisy training acting as an incidental regulariser, and the matched `oracle`-eval pass (E3.6) then upper-bounds what perfect perception would buy; the avoidance turns out perception-robust (Section 5.11.2), which is the property a sim-to-real claim needs. The *filter* is trained and evaluated on `noisy` throughout: a runtime filter must match the perception it will face at deployment. A “perception-off” mode is architecturally inapplicable to the trained policy (it changes the input size, $288 \rightarrow 264$), so it appears only as the separately trained E1.1 ablation. A fully noisy-trained headline is natural follow-up (Section 7.6).

5.4 Baselines and Configurations

On the persistent-co-location task, the headline E4.1 comparison (Section 5.12.4) evaluates four configurations: (1) the *unconstrained baseline* (end-of-curriculum stage-2 snapshot, no safety mechanism); (2) the *proactive policy*, (1) fine-tuned with the fixed- $\lambda = 0.1$ value-based Lagrangian (training-time avoidance alone: the proximity axis); (3) the *speed-scaling filter on the baseline* ($d_{\text{slow}} = 0.40$ m; deployment-time velocity control alone, no retraining); and (4) the *policy-filter composition*, (2) under the same filter (do the axes compose?). Two further families serve as controls and as the reactive-filter taxonomy: a *workspace-shaping confound control* (the baseline with the bounded shaping term ablated), and the learned SVF veto on both baseline and policy plus the model-based base-dodge and end-effector-retract filters (Section 5.12.5).

On the intermittent-co-location tasks, five method classes are compared on the same success/SSM-violation plane (Section 5.14): the unconstrained curriculum baseline; the PID-Lagrangian across a cost-budget sweep (with its best basin checkpoint reported); the SVF binary veto; unconditional speed-scaling; and critic-gated speed-scaling at a swept gate threshold R . The boundary-condition experiment (Section 5.15) then runs the critic-gated filter on `saucepan_to_hob` under a pre-registered decision rule.

External baseline. WCSAC [1], the canonical distributional safe actor-critic, is faithfully reimplemented (Section 4.4) and evaluated from scratch on both intermittent-co-location tasks across three CVaR budgets. It is an *external reference* rather than a like-for-like ablation: it is actor-critic and demo-free where the project’s methods are value-based and demo-warm-started, and that asymmetry is part of what it measures (Section 5.14.1).

5.5 The Two Safety Axes and the Evaluation Protocol (R0)

5.5.1 The Exogenous Decomposition of Proximity

The protocol starts from a measurement, not a convention. The intuitive safety number for human–robot co-working is `ep_proximity_violation_rate`: the fraction of env-steps with min-separation $< \tau_{\text{prox}} = 0.3\text{m}$ (threshold calibrated in Section 3.9). But how much of that quantity does the *robot* control? A controlled sweep answers this directly: driving a runtime filter from no intervention to a *fully frozen* robot (vetoing 100 % of steps) removes only $\approx 58\%$ of time-in-proximity (Figure 5.1). The remaining $\approx 42\%$ is the coworker walking up to a robot that is already stationary: an *exogenous*, human-driven component that no robot-side mechanism, policy or filter, can remove. Three provenance points matter for how this finding is used. First, the decomposition comes from a sweep run *before* any of the runtime-filter comparisons it now organises, on the saucepan SVF threshold sweep; the axis split was derived in advance of the results it frames, not fitted to them. Second, *both* axes are reported for every configuration in every results table, so no result is hidden by the framing. Third, the framing cuts both ways: the constrained policy’s headline win lives on the proximity axis, and the runtime filters’ wins live on the velocity axis; this is axis *ownership*, not metric replacement.

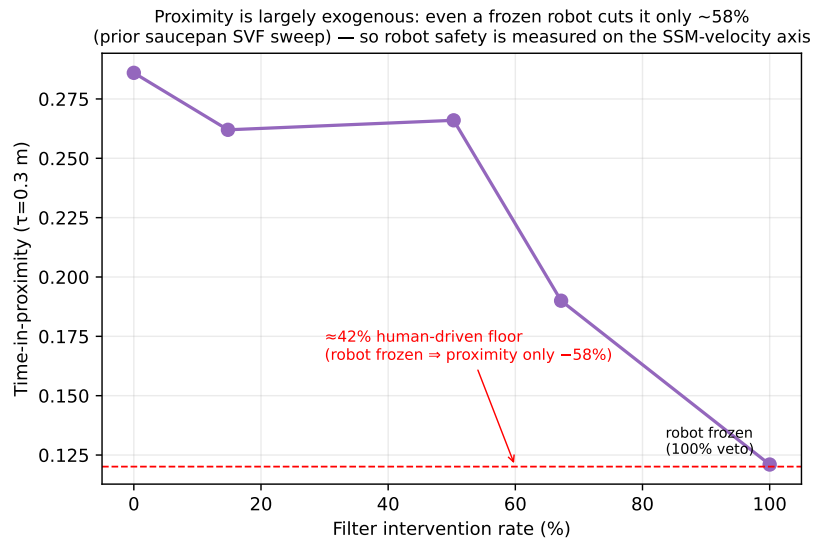


Figure 5.1: The exogenous decomposition of proximity (R0’s organising measurement). Sweeping the runtime filter’s intervention rate from 0 to 100 % (a fully frozen robot) removes only $\approx 58\%$ of time-in-proximity; the residual $\approx 42\%$ is the coworker approaching a stationary robot, a floor outside robot control. Provenance: this sweep is the *prior* saucepan SVF threshold sweep, run before the intermittent-co-location experiments existed; the two-axis protocol was fixed on its basis, not after seeing the later results.

5.5.2 Axis Ownership

Proximity measures *exposure*, but $\approx 42\%$ of it is human-driven and no robot policy can remove it. We therefore retain proximity as the exposure axis, it remains the natural axis for *policy-level* avoidance, whose anticipatory repositioning is the one mechanism that can reduce how often co-location arises at all, and use the velocity-adaptive SSM margin (`ep_ssm_violation_actual_rate`, Section 3.8) as the robot-controllable safety axis on which *runtime filters* are judged: whether the robot is moving slowly enough to stop before contact at the current separation is a function of the robot’s own commanded speed. Every results table reports both axes for every configuration; the conservative worst-case `ep_ssm_violation_rate` appears only in appendix tables (it saturates at $\sim 93\%$ on random rollouts and carries little discriminative signal, Section 3.8).

Pre-registered decision rules. Three accept/reject rules were fixed before the corresponding sweeps ran, and are reported with their outcomes: (i) the E1.1 observation-ablation bar, $\geq 20\%$ SSM-rate reduction on at least one (method, task) pair (Section 5.11.1); (ii) the reactive-filter gracefulness bar on the proximity axis, $\geq 30\%$ proximity reduction at $\leq 25\%$ intervention (Section 5.12.5); and (iii) the boundary-condition rule for critic-gated scaling on `saucepan_to_hob`, success ≥ 0.60 and SSM-actual ≤ 0.08 for the gate to count as recovering throughput (Section 5.15). Rule (iii)’s failure on `saucepan` is itself a headline result, because it matches the predicted boundary condition.

5.5.3 Task Performance and the Safety Tax

Two complementary task metrics so the *cost of safety* on the task is explicit:

- `success_rate` (terminal task completion) is the primary task axis. **The Δ success column in the headline Table 5.7 is the explicit “did safety reduce task performance?” answer**, computed as $(s_{\text{row}} - s_{\text{row } 1})/s_{\text{row } 1}$ with s = success rate.
- `steps_to_completion` (mean env-steps to terminal, among completed episodes) captures whether the policy preserves task completion but takes longer. A safe-and-slow policy and a safe-and-fast policy both register `success_rate` = 1.0; `steps_to_completion` discriminates them. Under ISO 15066 SSM, the robot is expected to slow when the human approaches, so an increase here is expected behaviour rather than a failure mode, but it should be quantified.

`episode_reward` (sum of $r_t^{\text{task,shaped}}$ over the episode) is reported in appendix tables as a tertiary axis that combines task progress and the workspace-shaping signal.

5.5.4 Tail Risk

Tail-risk metrics:

- CVaR_{0.95} of `ep_cost_integral` (the tail metric matching the cost signal the policy was trained on).
- CVaR_{0.95} of `ep_min_separation` (a more human-readable tail metric).
- 99th percentile of `ep_min_separation`.

Each is computed over the union of 60 evaluation episodes per configuration. Contact-force tail metrics are not reported because of the open PFL contact-detection limitation (Section 3.10).

5.5.5 Policy / Filter Interaction

Three complementarity metrics for RQ3:

- *Filter intervention rate*: fraction of env-steps where the runtime filter substitutes its fallback (zero-velocity for the veto, scaled velocity for speed-scaling). Reported in the headline Table 5.7 and the taxonomy Table 5.8; passthrough rate is its complement ($1 - \text{intervention}$).

- *Internalisation curve*: filter intervention rate evaluated on intermediate snapshots of the Lagrangian-trained policy as training progresses. A *falling* curve (rising passthrough) would be evidence the policy is learning the filter’s job; E4.3 (Section 5.12.7) finds the opposite, a *flat, elevated* curve, which is itself the diagnostic for the out-of-distribution veto.
- *Passthrough–task correlation*: at deployment, episodes with higher filter passthrough should also exhibit lower `steps_to_completion`, because every intervention swaps in the zero-velocity fallback and costs the policy a step or more of task progress. The harness records both per episode for the headline cells.

5.6 Statistical Methodology

Every evaluation cell runs 20 episodes per evaluation seed across three evaluation seeds (60 episodes), sampled from the relevant `ParameterSpace`. Configurations involving a *trained* policy additionally pool three independently trained seeds, giving 180 episodes for the headline saucepan policy and composition rows and for every row of the boundary-condition sweep; each table states which applies. 95 % bootstrap confidence intervals (10,000 resamples over episodes) are reported for the headline safety metric of each table. Directional claims are assessed by bootstrap-CI overlap rather than parametric tests: episode-level safety metrics are bounded, zero-inflated and heavily non-Gaussian, so *t*-statistics would be poorly calibrated, whereas the CI criterion is conservative and assumption-free. Two deliberate exceptions to the three-trained-seed rule are flagged where they occur: the intermittent-co-location Lagrangian budget sweep trains a single seed per budget cell (six cells; the saucepan arm had already established the seed-instability finding that would make per-cell seed replication uninformative), and the WCSAC external reference is a single seed per (task, budget) cell at 30 evaluation episodes, reported with that caveat (Section 5.14.1).

Every script also carries a `-smoke` flag (a 5-minute CPU validation); no multi-hour training was launched without passing the relevant smoke gate first.

5.7 Compute Budget

All training ran on single NVIDIA A100 (40 GB) GPUs. For the persistent-co-location (saucepan) arc the dominant cost is Lagrangian fine-tuning: a 40k-frame constrained run takes ≈ 9 h, and 18 such runs were trained across the reported grids (3 E3.1 cost-form cells; 3 infeasible-budget E3.2 cells; 3 seeds of PID auto-tuning at $d = 0.3$; and the fixed- λ grid, $\lambda \in \{0.1, 0.15, 0.27\} \times 3$ seeds), totalling ≈ 160 h. The remainder of that arc: the three-stage base-policy curriculum ≈ 19 h, plus ≈ 12 h for the no-shaping stage-2 confound control; the Phase 2 SVF pipeline ≈ 12 h; and all snapshot evaluation together ≈ 15 h. Subtotal ≈ 220 GPU-hours, reconstructed from run logs and rounded. The intermittent-co-location arc adds: the six-cell budget-retarget sweep (≈ 54 h at the measured per-frame rate); the shared dish/drawers SVF pipeline (≈ 12 h); the veto, speed-scaling, gated, and ablation sweeps plus the boundary-condition sweep on saucepan (evaluation-only, ≈ 30 h); and the six 150k-frame from-scratch WCSAC runs, which dominate the additional training cost (estimated ≈ 120 – 180 h; these ran on a shared multi-GPU dispatcher and their per-run hours are estimates rather than log-reconstructed). The project total is therefore roughly 450–500 GPU-hours; carbon accounting (≈ 40 kg CO₂e under the 2024 UK grid emissions factor) is derived in Appendix E.

5.8 Experiment Index

5.9 Ablation Map: What Each Component Proves

The evaluation is deliberately ablation-led. Each major component of the Hybrid Safety Critic is removed, replaced, or retuned to test whether it actually contributes to either proximity exposure

Table 5.1: Experiment index. Each is identified by a phase-prefixed code ($En.m$) and reported in Chapter 5; R1 = persistent co-location (`saucepan_to_hob`), R2 = intermittent co-location (`dishwasher_close`, `drawers_open_all`), R3 = the cross-task boundary condition.

ID	Description	Reported in
E1.1	BC observation ablation (off / oracle / noisy), no penalty	§5.11.1
E2.1–E2.3	SVF CQL- α / filter-on-baseline / threshold- R Pareto (R1)	§5.12.3
E2.4	SVF binary veto on the chunked policy (R2)	§5.14.2
E2.5	Unconditional speed-scaling (R2)	§5.14.3
E2.6	Critic-gated speed-scaling, R -dial sweep + ablations (R2)	§5.14.4, §5.14.5
E3.1	Cost-signal form vs. budget feasibility (R1)	§5.12.1
E3.2	Cost budget sweep and feasibility boundary (R1)	§5.12.2
E3.3	Cost-budget retarget sweep, $d \in \{0.16, 0.19, 0.22\}$ (R2)	§5.14.1
E3.6	Perception robustness (oracle vs. noisy) of the constrained policy	§5.11.2
E3.7	External baseline: WCSAC, from scratch, 2 tasks $\times$ 3 budgets	§5.14.1
E4.1	R1 headline: proactive / reactive / composed comparison	§5.12.4
E4.3	Filter intervention rate during Lagrangian training (R1)	§5.12.7
E4.4	R3: critic-gated scaling on saucepan (pre-registered rule)	§5.15
E5.1	Tail-risk metrics across the headline configurations (R1)	§5.13.1
E5.2	OOD generalisation on the wider <code>coworker_eval</code> band (R1)	§5.13.2
E5.3	Cross-task transfer of the saucepan pipeline (resolved negative)	§5.14.1

or SSM-velocity safety. The main ablations are summarised here before the regime-specific results:

- **Human-state channel without a safety gradient** (E1.1): perception alone does not produce safety; the policy uses oracle human state for task completion, not distance keeping.
- **Cost form and budget** (E3.1–E3.3): continuous vs. binary cost is secondary; the binding question is whether the budget is feasible in the task’s co-location regime.
- **Multiplier update** (E3.2): PID auto-tuning is seed-unstable at the feasibility boundary, motivating fixed λ on saucepan.
- **Runtime override form** (E2.4–E2.6): a binary veto breaks chunked-policy coherence, unconditional scaling is safe but costly, and critic-gated scaling recovers throughput only where safe windows exist.
- **Gate critic and label design** (E2.6 ablations): R is the dominant dial, looser anticipatory labels do not improve the frontier, gating a Lagrangian policy does not dominate gating the baseline, and DAGger-style re-collection on filtered rollouts under-gates.

This map is the evidence that the final architecture is not an accidental combination of parts: each retained component survives a specific failure mode of a simpler alternative.

5.10 Baseline ISO 15066 Compliance

The results below are organised by research question rather than by phase; this section first establishes the unconstrained baseline against which every later number is measured. The unconstrained CQN-AS policy at the end of stage 2 of the curriculum (Section 4.5) is task-competent on `saucepan_to_hob`: under the headline `noisy` perception condition it reaches `success_rate` = 0.85. The safety picture is poor (Table 5.2): `ep_proximity_violation_rate` = 0.296 ($\approx 30\%$ of steps within $\tau_{\text{prox}} = 0.3\text{m}$) averaged across 60 eval episodes, the velocity-adaptive ISO 15066 SSM-actual flag fires on 14.6 % of steps, and the mean robot–human separation is only 0.132 m. The worst-case ISO bound (conservative S_p with $v_h = v_{h,\text{max}}$) is saturated, firing on $\sim 93\%$ of steps, which is why it is reported only for traceability. Tail behaviour is dominated by the human’s closest lunge: CVaR_{0.95} of `ep_min_separation` is $\approx 0.005\text{m}$, with the single worst approach reaching $\approx 0.002\text{m}$. The episode length is ~ 449 control steps.

Table 5.2: Baseline ISO 15066 compliance for the unconstrained CQN-AS policy on `saucepan_to_hob` under the `coworker_train` distribution (means $\pm$ 95 % bootstrap CI, 3 seeds $\times$ 20 episodes). This is the “before” picture and the floor against which every later result is measured: the policy is task-competent yet violates proximity on a large fraction of episodes, because nothing in its objective represents the human.

Metric	Unconstrained baseline	95 % CI
<code>success_rate</code>	0.85	[0.75, 0.93]
<code>ep_proximity_violation_rate</code>	0.296	[0.23, 0.37]
<code>ep_ssm_violation_actual_rate</code>	0.146	[0.10, 0.19]
<code>ep_min_separation</code> (mean, m)	0.132	–
CVaR _{0.95} <code>ep_min_separation</code> (m)	$\approx$ 0.005	–

Reading. The unconstrained baseline is the floor: a task-competent policy that is *not* ISO 15066 compliant under sustained co-working. The deployment posture that CQN-AS ships with [4] on BiGYM would, under `safety_bigym`’s COWORKER distribution, be within 0.3 m of the coworker on $\approx$ 30 % of steps. Every reduction reported below is relative to this floor.

Intermittent-co-location baselines. The same curriculum recipe yields competent unconstrained baselines on the other two tasks (60 episodes each, `noisy`, COWORKER): `dishwasher_close` reaches 0.77 success with SSM-actual 0.176 and proximity 0.246; `drawers_open_all` reaches 0.82 success with SSM-actual 0.100 and proximity 0.211. The drawers policy is naturally slower (mean robot velocity 0.19 vs. dishwasher’s 0.44 m/s), which is why its baseline SSM-violation rate is half the dishwasher’s, a fact that later bounds how much a velocity-axis filter can add there (Section 5.14.4).

5.11 RQ1: The Human-State Observation Channel

5.11.1 E1.1: Behaviour Cloning under No Penalty (Negative)

Phase 1 ran the obs-ablation under pure behaviour cloning (ACT, 4-task sweep, modes `off/oracle/noisy`, no violation penalty, $n = 10$ episodes per cell), against the pre-registered ≥ 20 % SSM-reduction criterion of Section 5.5 (Table 5.3).

Table 5.3: Experiment E1.1. ACT BC under no penalty across four tasks and three observation modes. SSM-violation rate averaged across six legacy disruption types per cell. None of the cells clears the 20 % improvement criterion; oracle is worse than off on three of four tasks.

Task	off	oracle	Δ (off $\rightarrow$ oracle)
<code>reach_target_single</code>	0.534	0.548	−2.7 % (worse)
<code>dishwasher_close</code>	0.533	0.537	−0.8 % (worse)
<code>drawers_open_all</code>	0.277	0.239	+13.7 % (best, below 20 % bar)
<code>saucepan_to_hob</code>	0.135	0.203	−49.9 % (much worse)

The result is clear: the human-state channel does not help pure BC. The most informative cell is `saucepan_to_hob`, where the oracle channel dramatically improves task success (0.22 $\rightarrow$ 0.58) yet *worsens* SSM violations: the policy uses human state to route around obstructions and complete the task, not to keep distance, because nothing in its objective rewards the latter. This observation motivates the constrained-RL formulation, observation without a safety gradient is at best inert and at worst counterproductive. E1.1 alone cannot distinguish “channel useless” from “BC marginalises a channel uncorrelated with demo actions”; E3.6 resolves the ambiguity under non-degenerate constrained training.

5.11.2 E3.6: Perception Robustness of the Avoidance (oracle vs. noisy)

To test whether the learned avoidance depends on perfect state estimation, we re-evaluated the *same* constrained policy (fixed- λ , Section 5.12.4) under two perception regimes: `oracle` (ground-truth human pose) and `noisy` (the calibrated mock-BodySLAM++ estimate with measurement

noise and a 3-step latency). The avoidance is *not* perception-bottlenecked: the proximity-violation rate is statistically indistinguishable across regimes (0.236 oracle vs. 0.198 noisy), both far below the unconstrained baseline’s ≈ 0.30 , and task success is comparable (0.82 vs. 0.75). The baseline, by contrast, shows *no* oracle–noisy gap (0.302 vs. 0.296): it does not use the human channel to avoid, so perception quality cannot help it. That the policy retains its full reduction under noisy, lagged perception, and is if anything marginally better there, indicates a robust avoidance behaviour rather than one that exploits perfect state (Table 5.4).

Table 5.4: Experiment E3.6. The constrained policy (seed-0 trained) and the unconstrained baseline re-evaluated under **oracle** and **noisy** perception (**saucepan_to_hob**, **coworker_train**; each cell is 60 episodes, 3 evaluation seeds $\times$ 20). The policy’s proximity reduction is preserved under realistic noisy, lagged perception (bootstrap CIs across modes overlap: 0.236 [0.17, 0.30] oracle vs. 0.198 [0.14, 0.26] noisy), and the baseline shows no oracle–noisy gap because it does not use the channel to avoid. The headline E4.1 table reports the pooled three-trained-seed **noisy** numbers.

Perception	Unconstrained baseline		Constrained policy ($\lambda=0.1$)	
	succ.	prox.viol	succ.	prox.viol
oracle	0.77	0.302	0.82	0.236
noisy	0.85	0.296	0.75	0.198

A third, “perception-off” mode is architecturally inapplicable (removing the channel changes the input size, $288 \rightarrow 264$, so the trained weights cannot run); a genuine no-human-observation condition is the separately trained E1.1 ablation. The perception sweep is thus a two-mode comparison by construction.

5.12 R1: Persistent Co-location (**saucepan_to_hob**)

This section is the persistent-co-location arm of the regime map. On **saucepan_to_hob** the coworker occupies the robot’s working volume nearly continuously (gate-activity measurement in Section 5.15: a separation-triggered filter would be active on ≈ 44 – 62% of steps). The questions, in order: can constrained RL find a graceful operating point here (E3.1–E3.2, E4.1); what can reactive filtering contribute on each axis (E2.1–E2.3, the taxonomy); how do policy and filter compose (the joint-coverage analysis); and how robust is the result (E4.3, E5.1, E5.2).

5.12.1 E3.1: Cost-Signal Form Is Not the Operative Variable

We ablated the *form* of the per-step safety cost c_t supplied to the constrained critic, holding the rest of the agent fixed: a *continuous* graded signal ($c_t = \min(1, \max(c^{SSM}, c^{PFL})) \in [0, 1]$, Equation 3.4), a *binary* indicator ($c_t = 1$ when the worst-case SSM bound is violated, else 0), and a *fixed* reward penalty (a constant subtracted on violation, with no dual variable). The original hypothesis was that the graded signal would give a smoother gradient and hence a better safety/throughput trade-off than the binary one.

At the nominal tight budget $d = 0.01$ the comparison is *degenerate*: both dual-variable forms collapse to 0% task success (Table 5.5). The cause is feasibility, not the cost form. The dual variable evolves as $\lambda \leftarrow \lambda + k(\bar{c} - d)$, so it can only stabilise if some policy achieves average cost $\bar{c} \leq d$. But the task’s *inherent* cost lies far above 0.01: the unconstrained baseline already incurs a proximity-violation rate of 0.296 (Table 5.7), and roughly 42% of that proximity is *exogenous*, the coworker walking into the robot where no robot action can prevent it (Sections 5.12.3, 5.5.1). The achievable cost floor is therefore $\approx 0.42 \times 0.296 \approx 0.12$, an order of magnitude above the budget. With $\bar{c} \gtrsim 0.12 > d$ the constraint is infeasible, λ winds up without bound, the $-\lambda Q_c$ term swamps the reward objective, and the policy abandons the task to minimise cost. This occurs for *both* forms because the encoding changes the *shape* of c_t , not its *scale* relative to d ; if anything the binary form is the more aggressive, since it keys on the worst-case ISO indicator that fires on $\gtrsim 90\%$ of unconstrained steps, so the two forms are not even comparable in magnitude. (A fixed reward penalty with no dual variable sidesteps the windup, but its apparent training-time

reduction did not survive deployment-faithful evaluation, so it is not adopted as an operating point either.)

Table 5.5: Experiment E3.1. Cost-signal form under the same B-value-mean backbone on `saucepan_to_hob` at the tight budget $d = 0.01$. Both dual-variable forms (continuous, binary) collapse to zero success: the budget sits an order of magnitude below the task’s inherent cost, so the constraint is infeasible and the multiplier winds up regardless of cost-signal shape. The cost form is not the operative variable; the budget relative to the task’s inherent cost is.

Cost signal	success_rate	ep_prox_viol
Continuous $+ \lambda$ ($d=0.01$)	0.00	0.013
Binary $0/1 + \lambda$ ($d=0.01$)	0.00	0.024
Fixed reward penalty (no λ) [†]	0.80	0.242

[†] Training-eval; the fixed-penalty reduction did not transfer to the deployment-faithful benchmark (its deployment proximity returned to $\approx$ baseline), so it is reported for completeness only.

The degeneracy is not a null to be explained away but the first point on the feasibility curve: E3.2 next sweeps the budget, the variable that *is* operative, and locates the boundary this point sits below.

5.12.2 E3.2: Cost Budget Sweep and the Feasibility Boundary

E3.1 established that the cost *form* is not the operative variable; E3.2 sweeps the cost *budget* d , the variable that is. The sweep does not trace a smooth safety/throughput Pareto. Instead it exposes a sharp feasibility boundary: every budget at or below the task’s inherent per-step cost collapses task success. Continuous-cost Lagrangian runs at $d \in \{0.001, 0.05, 0.1\}$ reach **success_rate** = 0.00, 0.00, 0.13 respectively, all essentially task-abandoning, because each budget sits below the ≈ 0.12 exogenous floor and the multiplier winds up (Section 5.12.1).

The single budget that is both feasible and non-trivial, $d = 0.3$ (at the task’s inherent cost), does not give a stable operating point under PID auto-tuning: it sits *on* the feasibility boundary, so the dual variable is decided by each seed’s stochastic cost trajectory. Figure 5.2 shows the three seeds at $d = 0.3$ converging to $\lambda = 0.000, 0.267, 3.855$, producing an unconstrained policy, a graceful avoidance basin, and a windup collapse to zero success. The graceful regime reproduces in only one of three seeds. This is why the headline policy (Section 5.12.4) pins λ at a fixed value rather than auto-tuning it: the budget loop is ill-conditioned exactly where the only feasible budget lies.

5.12.3 Filter Pareto Frontier (E2.1–E2.3)

The Phase 2 SVF was trained at $\alpha_{\text{CQL}} = 5.0$ and swept over the veto threshold R on a held-out validation set (three seeds, 20 episodes each). With the filter disabled ($R = 0$) the swept **ep_proximity_violation_rate** is 0.286, which matches the unconstrained benchmark baseline of 0.296 to within sampling error; this agreement is the validation check that the sweep environment reproduces the deployment environment.

The sweep does *not* expose an operating point that reduces geometric proximity cheaply: $R = 2.25$ cuts proximity by only $\approx 8\%$ at $\approx 15\%$ intervention, a one-third reduction needs $\approx 67\%$ intervention, and even vetoing *every* step caps the reduction at $\approx 58\%$, the decomposition of Section 5.5.1 seen from the filter side. We therefore pin the operating point at the light backstop $R = 2.25$ and read the veto as an ISO 15066 SSM velocity backstop, not a proximity reducer; proactive proximity reduction is delegated to the constrained policy (Section 5.12.4).

The reactive-fallback dilemma. A natural objection is that a more aggressive fallback would let the filter reduce proximity directly. We tested this. Table 5.6 sweeps the fallback behaviour on the unconstrained baseline at a matched, low intervention rate. Zero-velocity braking holds station: it trims the robot’s mean speed (the velocity-backstop effect) but leaves proximity essentially unchanged, because the robot simply dwells while the human approaches. A retreating

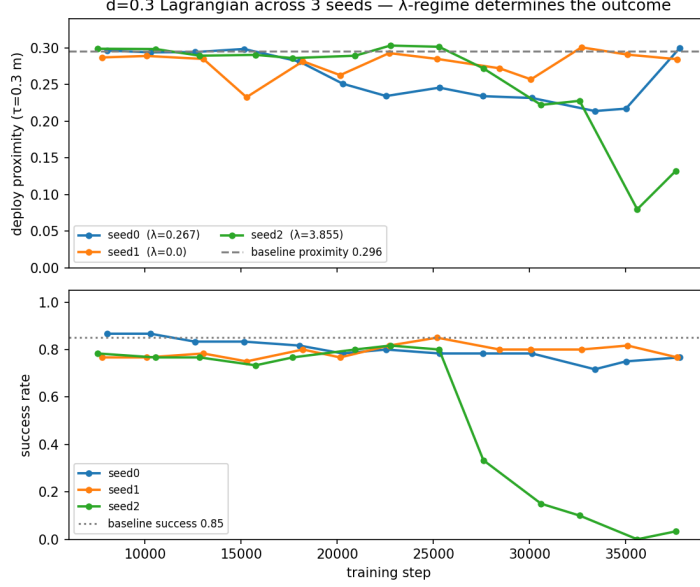


Figure 5.2: Experiment E3.2 / RQ2. PID-Lagrangian at the only feasible budget $d = 0.3$, three seeds. Top: deployment proximity-violation rate ($\tau = 0.3$ m) versus training step; bottom: task success. The seeds converge to $\lambda = 0.000$ (seed 1, unconstrained, flat at the baseline ≈ 0.30), $\lambda = 0.267$ (seed 0, a graceful sub-0.26 avoidance basin), and $\lambda = 3.855$ (seed 2, windup collapse, success $\rightarrow 0$). Because $d = 0.3$ coincides with the task’s inherent cost, the dual variable carries too little signal to auto-tune stably, motivating the fixed- λ headline.

fallback can cut proximity sharply ($0.296 \rightarrow 0.095$, -68%), but only by having the robot flee: task success collapses from 0.85 to 0.18, mean robot speed rises sixfold ($0.29 \rightarrow 1.70$ m/s), and the `ep_ssm_violation_actual_rate` actually *worsens* ($0.147 \rightarrow 0.180$) because a fast-retreating robot is itself an SSM hazard. A gentle retreat does neither job. No reactive setting buys geometric separation at acceptable success and speed: against an approaching human, reducing proximity requires sustained, anticipatory robot motion, which a one-step reactive filter cannot supply. This is the empirical case for assigning the exposure axis to the policy arm, and it is revisited in Section 6.2.4.

Table 5.6: Reactive-filter fallback sub-study on the unconstrained baseline policy (v3 G1 SVF, COWORKER distribution, `noisy` observations; 60 evaluation episodes per row). No reactive fallback reduces geometric proximity at acceptable success and robot speed. Zero-velocity braking is a velocity backstop only; an aggressive retreat buys separation by abandoning the task and accelerating, which worsens the SSM-actual rate.

Fallback	R	Interv.	Proximity	Success	Mean vel (m/s)
None (baseline)	—	—	0.296	0.85	0.29
<code>zero_velocity</code>	2.25	15 %	0.303	0.78	0.27
<code>retreat_gentle</code>	2.25	2 %	0.290	0.75	raised
<code>retreat_aggressive</code>	2.5	14 %	0.095	0.18	1.70

The veto trims the robot’s *mean* speed and is independently available as a deployment backstop without any policy retraining, but it does not by itself reduce the velocity-adaptive SSM-actual margin (a stop-or-go veto does not maintain the graded margin ISO 15066 prescribes; Section 5.12.5). The fallback study establishes the limit of what a reactive veto can do alone on the proximity axis; the question of which reactive mechanism actually reduces the velocity axis, and how it composes with the policy, is taken up in Sections 5.12.5 and 5.12.6, where a graded speed-scaling filter, not the binary veto, proves to be the one that works.

5.12.4 E4.1: The Headline Comparison

Main comparison. The central question of the architecture is which mechanism reduces which ISO 15066 axis, and how the mechanisms compose. The answer is a *division of labour*: the proactive constrained-RL policy owns the geometric-proximity axis and a complementary-axis runtime filter owns the velocity (SSM-actual) axis. Table 5.7 reports the four configurations that establish this; the two reactive-veto configurations the architecture was originally built around are covered in Section 5.12.5, and the both-axis composition is analysed in Section 5.12.6.

The proactive policy reduces proximity, reproducibly. The fixed- λ ($\lambda = 0.1$) Lagrangian policy reduces `ep_proximity_violation_rate` from the baseline 0.296 to 0.228, a 22.8 % reduction (bootstrap 95 % CI [0.194, 0.264]), at 0.76 task success (CI [0.70, 0.82]), pooled over three seeds (180 episodes). Each seed’s selected operating point is below the baseline; the velocity-adaptive SSM-actual rate also improves (0.146 $\rightarrow$ 0.112, -23%) and mean robot velocity rises only $\sim 8\%$. This is the graceful proximity reduction the reactive filter could not achieve (Section 5.12.3). The magnitude is bounded by the $\approx 42\%$ exogenous floor (Section 5.5.1): it captures roughly 40 % of the *reducible* (robot-driven) proximity. Modest in absolute terms, but the reactive filter captured essentially none of it at acceptable cost.

Table 5.7: Experiment E4.1 (**the headline**). The division of labour between the proactive policy and a complementary-axis runtime filter on `saucepan_to_hob` under `coworker_train`, `noisy` perception. Rows 1 and 3 evaluate the single stage-2 baseline policy over 60 episodes (3 evaluation seeds $\times$ 20); rows 2 and 4 pool three independently trained seeds (180 episodes). Brackets are 95 % bootstrap CIs. The policy owns the geometric-proximity axis ($\tau = 0.3$ m); the graded speed-scaling filter owns the velocity-adaptive SSM-actual axis; only their composition reduces both at once, at a multiplicative success cost. Δ columns are relative to the unconstrained baseline (negative is safer). The two SVF-veto configurations are in Table 5.8.

Configuration	succ.	prox.viol	Δ prox.	ssm-act.	Δ ssm.	Interv.
Unconstrained baseline	0.85	0.296 [0.23, 0.37]	–	0.146 [0.10, 0.19]	–	–
Proactive policy ($\lambda=0.1$)	0.76	0.228 [0.19, 0.26]	-23%	0.112 [0.09, 0.13]	-23%	–
Speed-scaling filter (on baseline)	0.53	0.273 [0.21, 0.35]	-8%	0.048 [0.03, 0.07]	-67%	45 %
Policy + speed-scaling (composition)	0.44	0.250 [0.21, 0.29]	-15%	0.065 [0.05, 0.08]	-55%	40 %

Operating-point selection must be safety-aware. A methodological point underwrites the policy row. At the feasible budget the avoidance does not live at the peak-success checkpoint: `snapshot_best` (peak success) and the final checkpoint both deploy at $\approx$ baseline proximity (≈ 0.30), because selecting on task success picks *against* the cost constraint. The avoidance lives in a mid-training basin ($\sim 20\text{k}$ – 35k steps, several consecutive checkpoints at 0.21–0.25 with success 0.72–0.80 and SSM-actual improved). The operating point must therefore be chosen by a *safety-aware* rule (lowest deployment proximity at a success floor), not by peak success; selecting on success produced two false-null results before the basin was found. Each seed’s selected basin checkpoint is shown in Figure 5.3; that the basin appears consistently across all three seeds under fixed λ is what makes the reduction reproducible, in contrast to the PID instability of Figure 5.2.

The basin is constraint-driven, not selection noise. Selecting a checkpoint by a deployment metric invites the objection that the “basin” is evaluation noise harvested by the rule. Four observations rule this out: the $\lambda = 0$ seed, processed by the *same* rule, shows no basin (flat at ≈ 0.30 with isolated dips); the basin is a *run* of consecutive checkpoints at 0.21–0.26 proximity with success 0.72–0.80 (Figure 5.3), not a single dip; SSM-actual co-improves inside it, which uncorrelated proximity noise would not produce; and the selected point also sits below the *no-shaping* baseline at matched success (the -12% comparison below). When λ engages, the constraint does work; when it is slack, no selection rule manufactures a basin.

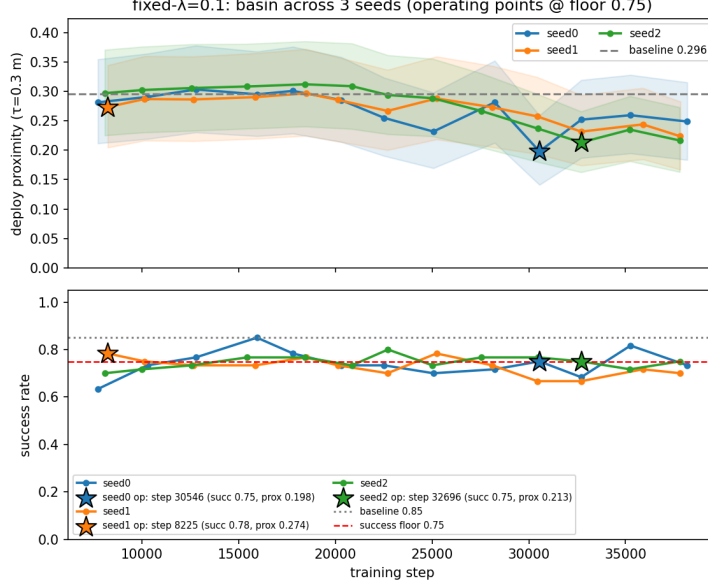


Figure 5.3: The fixed- $\lambda = 0.1$ policy across three seeds (stars mark the safety-aware operating point at a success floor of 0.75). Top: deployment proximity-violation rate; bottom: success. Unlike the PID runs (Figure 5.2), all three seeds behave consistently and each reaches a sub-baseline proximity basin in mid-training, giving the pooled 0.228 at 0.76 success. Fixing λ removes the ill-conditioned budget loop that made the PID controller seed-unstable.

Controlling for the workspace-shaping confound. The unconstrained baseline carries a mild workspace-distance reward ($\beta = 0.05$, Section 3.6) that anchors the end-effector in the task workspace. A natural objection is that this inflates the baseline’s proximity and hence the policy’s apparent reduction. Ablating it: the no-shaping baseline is 0.75 success / 0.258 proximity, versus the shaping baseline’s 0.85 / 0.296. So shaping is a *task-success aid that incidentally raises proximity* (it keeps the robot where the coworker reaches), not a safety term. Crucially, the Lagrangian policy (0.228 at 0.76) sits below the no-shaping baseline too, a -12% proximity reduction at matched success, so the headline -23% (against the matched-recipe baseline) is not an artefact of an inflated reference. We report both: -23% (isolated constraint effect, shaping held fixed) and -12% (confound-controlled, against the vanilla baseline).

The velocity axis is the filter’s, and only a graded filter reaches it. A runtime filter’s canonical ISO 15066 role is the velocity axis. The graded speed-scaling filter delivers it: on the baseline it drives SSM-actual from 0.146 to 0.048 (-67%) at its optimum ($d_{\text{slow}} = 0.40\text{ m}$), with proximity essentially unchanged ($0.296 \rightarrow 0.273$). It is the only filter in the taxonomy that meaningfully reduces *any* ISO 15066 axis on its own terms; the binary veto, despite also slowing the robot, leaves SSM-actual at $0.148 \approx$ baseline, a stop-or-go veto does not maintain the graded margin the standard prescribes. The success cost, however, is not an artefact of the 0.40 m setting: even the gentlest swept point ($d_{\text{slow}} = 0.25\text{ m}$) intervenes on 37 % of steps, co-location is persistent, so some part of the robot is nearly always inside the slow radius, and still costs a third of task success at *less* velocity-axis gain. The reactive cost is paid at every operating point; only its exchange rate moves.

The composition is the only both-axis-compliant configuration, and it costs success. Composing the policy with the speed-scaling filter is the *only* configuration that reduces both ISO 15066 axes at once: geometric proximity 0.250 (retained from the policy’s 0.228, CIs overlap) *and* SSM-actual 0.065 (-42% against the policy’s 0.112, the filter’s contribution). Each component alone leaves the *other* axis near baseline (policy: SSM-actual 0.112; filter-alone: proximity $0.273 \approx$ baseline). The cost is task success: $0.85 \rightarrow 0.44$. The proactive cost

($0.85 \rightarrow 0.76$, $\times 0.89$) and the reactive cost ($\times 0.58$) stack *multiplicatively*, and the composition beats neither specialist on that specialist’s own axis (the policy still wins proximity $0.228 < 0.250$; the filter still wins velocity $0.048 < 0.065$). The composition is the joint-coverage compromise, not a per-axis champion. Both measured safety axes are improved in this regime only by composing the proactive policy with a complementary runtime filter, and that composition makes the safety–throughput trade explicit, with a cost dial tunable through λ and d_{slow} (Section 5.12.6).

Sim-to-real perception. The whole table is evaluated under `bodyslam=noisy`, the realistic deployment condition and the filter’s native distribution (Section 5.3). The policy’s reduction survives this noisy, lagged perception unchanged (E3.6, Section 5.11.2): the architecture claim is reported under deployment-relevant perception, not only under oracle state.

5.12.5 The Reactive-Filter Taxonomy: Freeze versus Flee

The headline filter is the graded speed-scaling one. The architecture was, however, originally built around a learned CQL safety value function used as a veto, and the journey from that design to the graded filter is itself a result. We evaluated every reactive modality available, learned and model-based, and none reaches the proactive policy’s graceful operating point. The reason is structural: a reactive filter acts only *once the human is already close*, when the only moves are to *freeze* (and dwell in the danger zone) or to *flee* (and abandon the task). Table 5.8 summarises the taxonomy. The claim is scoped deliberately: *no reactive filter gracefully reduces proximity under persistent co-location*. It is a statement about the exposure axis in this regime, not about reactive filtering in general; Section 5.14 shows what reactive mechanisms *can* deliver, on the velocity axis, where co-location is intermittent and genuinely-safe windows exist.

Table 5.8: The reactive-filter taxonomy on `saucepan_to_hob` (COWORKER, `noisy`); every row is 60 evaluation episodes, and rows involving the constrained policy use the seed-0 trained policy (the pooled three-trained-seed policy numbers are in Table 5.7). Every reactive modality, learned or model-based, is bounded by the freeze-versus-flee dilemma: it either freezes (and proximity does not fall) or flees (and task success collapses). None reaches the proactive policy’s frontier (success 0.75, proximity 0.198 at seed 0; 0.76/0.228 pooled). The SVF veto applied to the *policy* (the architecture’s original composition design) is worse than the policy alone.

Filter	Mechanism	Proximity	Success	Interv.
Learned veto (SVF), on baseline	velocity backstop (freeze)	0.303	0.78	15 %
Learned veto (SVF), on policy	freeze $\rightarrow$ dwell	0.265	0.62	48 %
Model-based base-dodge (CBF)	flee (retreat base)	0.150	0.57	1.3 %
Model-based EE-retract (CBF)	flee (retract arm)	0.155	0.50	5.8 %
Proactive policy ($\lambda=0.1$)	anticipatory avoid	0.198	0.75	–

The learned veto freezes. On the baseline, the SVF veto with a zero-velocity fallback ($R = 2.25$, 15 % intervention) does not reduce geometric proximity ($0.296 \rightarrow 0.303$, within noise); it only trims mean robot velocity (-8%), acting as an SSM *velocity backstop* (Section 5.12.3). Applied on top of the already-avoiding policy, it is *counterproductive*: proximity worsens ($0.198 \rightarrow 0.265$) and success falls ($0.75 \rightarrow 0.62$), at a 48 % intervention rate (versus 15 % on the baseline). The mechanism is distribution shift: the SVF critic was collected on *baseline* rollouts, so it is out of distribution on the avoiding policy and over-vetoes, and each veto zeroes robot velocity, re-introducing the freeze/dwell failure. A threshold sweep confirms this is not mis-thresholding, $R \in [1.0, 2.25]$ gives 39–48 % intervention and proximity 0.27–0.28 at every setting, and re-collecting the critic on the policy’s own rollouts (on-policy) still yields no graceful operating point: at $\leq 25\%$ intervention every threshold *increases* proximity, and proximity falls only at 65–100 % intervention where the robot is essentially frozen. Neither critic meets the pre-registered gracefulness bar ($\geq 30\%$ proximity reduction at $\leq 25\%$ intervention, Section 5.5). Both the

baseline-trained and on-policy critics fail at every threshold, so the limit is the reactive paradigm, not the critic’s calibration.

The model-based filter flees. A geometric control-barrier-style “directional-dodge” filter (no learned critic) *does* reduce proximity where the veto cannot, $0.198 \rightarrow 0.150$ at $d_{\text{target}} = 0.35$ m and only 1.3 % intervention, but by *fleeing*: the base retreats, episodes stretch from ~ 450 to 620–770 steps, and success falls to 0.57/0.43/0.35 as $d_{\text{target}} = 0.35/0.45/0.55$, with proximity flooring at ≈ 0.14 regardless (the exogenous floor). There is no harmless-backstop setting: even at the minimal $d_{\text{target}} = 0.30$ m (1.1 % intervention) success falls $0.75 \rightarrow 0.58$, because even rare dodges land at task-critical moments. The same flee appears whichever part of the robot dodges: retracting the *end-effector* (an arm “flinch” via a damped-Jacobian step) is worse in the useful regime (0.50 success at $d = 0.35$; the EE is the closest pair, so it intervenes far more often, and retracting the arm *is* pausing the task). The flee is structural: the task and the hazard are co-located, so a separation barrier is satisfiable only by radial retreat, which is retreat from the workspace. Eliminating the flee requires anticipation, which is exactly what the constrained policy provides; “fixing” the flee inside a reactive filter amounts to reinventing the policy.

5.12.6 Joint ISO Coverage: The Composition and Its Ceiling

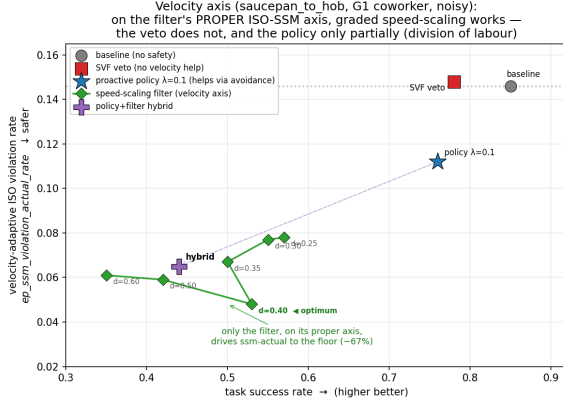
The two preceding subsections separate cleanly onto the two ISO 15066 axes (Figure 5.4). Figure 5.4a makes the velocity axis explicit: on SSM-actual the speed-scaling filter is the specialist ($\rightarrow 0.048$), the policy also helps via avoidance ($\rightarrow 0.112$) because keeping further from the human lowers the required margin, and the SVF veto does not help at all ($0.148 \approx \text{baseline}$). Figure 5.4b then plots both axes together: only the policy-plus-speed-scaling composition reaches the both-safe corner (low proximity *and* low SSM-actual), and it is strictly better than the SVF-veto composition, which *degraded* proximity with no velocity gain. A complementary-axis, model-based filter composes additively where a redundant-axis, out-of-distribution *learned* filter destroyed.

The composition’s success cost ($0.85 \rightarrow 0.44$, analysed in Section 5.12.4) should be read as *the persistent-co-location regime’s ceiling*, not as a defect of the composition: when the coworker is co-located most of the time, any separation-triggered speed response is active most of the time (measured directly in Section 5.15), so the reactive cost is paid almost everywhere and stacks with the proactive cost. Under this regime, both-axis ISO 15066 safety is purchasable only at stacked cost; whether a smarter trigger (a learned critic gating the scaler) can avoid that cost is exactly the question the intermittent-co-location chapter answers in the affirmative, and Section 5.15 answers in the negative *for this regime*.

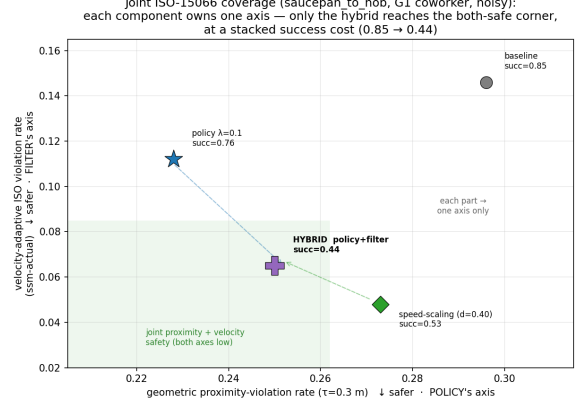
5.12.7 E4.3: The Filter Does Not Internalise (It Stays Out of Distribution)

If the policy and the SVF veto were complementary in the originally hypothesised sense, the filter’s required intervention rate would *fall* as the Lagrangian policy is trained, the policy progressively taking over the filter’s job. E4.3 evaluates the SVF veto’s intervention rate on intermediate snapshots of the fixed- λ policy across training. It does *not* fall: it stays at ~ 40 – 60 % throughout (mean ~ 49 %, Figure 5.5), *higher* than the ~ 15 % intervention on the unconstrained baseline, not lower. The reason is the distribution shift of Section 5.12.5: the constrained policy’s state–action distribution is precisely where the baseline-collected critic is least calibrated, so as the policy avoids *more*, the critic vetoes *more*. This is the corroborating mechanism behind the counterproductive veto composition: there is no internalisation curve to report because the baseline-calibrated filter never becomes redundant in the benign direction; it becomes *more* active and counterproductive.

The deployment implication inverts the originally intended one: a filter intended to compose with a learned policy must either be collected on that policy’s rollouts (still insufficient here, Section 5.12.5) or operate on an axis the policy does not move along, which is exactly why the graded speed-scaling filter, on the velocity axis, composes where the veto does not.



(a) The velocity (SSM-actual) axis: the graded speed-scaling filter is the specialist; the policy also reduces it via avoidance; the veto does not maintain the graded margin and leaves it at baseline.



(b) Joint coverage: proximity (x , policy's axis) vs. SSM-actual (y , filter's axis). Only the composition reaches the both-safe corner, at success 0.44; it beats neither specialist on that specialist's own axis.

Figure 5.4: The two-axis division of labour on `saucedpan_to_hob` and what composing the specialists buys.

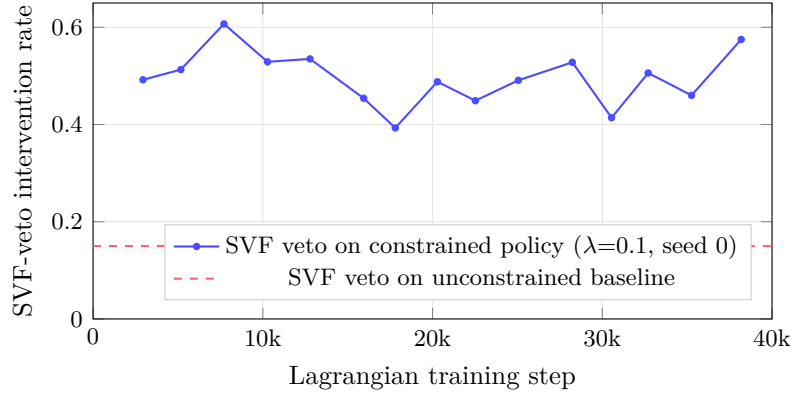


Figure 5.5: Experiment E4.3. The SVF veto's intervention rate on the constrained policy does *not* fall during training; it stays at $\sim 40\text{--}60\%$ (blue), above the $\sim 15\%$ it requires on the unconstrained baseline (red dashed). A complementary filter would show a falling curve; this flat, elevated curve is direct evidence that the baseline-collected critic is out of distribution on the avoiding policy and over-vetoes, the mechanism behind the counterproductive SVF-veto composition (Section 5.12.5).

5.13 RQ4: Tail Risk and OOD Generalisation

5.13.1 E5.1: The Worst-Case Tail Is Exogenous and Uncontrollable

ISO 15066 compliance is a worst-case rather than an average property, so the tail of the per-episode separation distribution is the safety-relevant quantity. The central tail finding is that the policy lowers the *mean* and *frequency* of proximity but *not* the worst-case tail. Mean separation improves modestly (baseline 0.132 m $\rightarrow$ policy 0.140 m), but the worst-5 % closest approach is essentially unchanged across the unconstrained baseline, the no-shaping baseline, and the constrained policy alike: $\text{CVaR}_{0.95}$ of `ep_min_separation` ≈ 0.005 m and the single worst approach ≈ 0.002 m in every configuration (Table 5.9).

This is the exogenous decomposition of Section 5.5.1 seen from its tail end: $\approx 58\%$ of proximity is robot-driven (suppressible by avoidance) and $\approx 42\%$ is human-driven, and the worst single approach lives entirely in the human-driven remainder. The policy captures roughly 40 % of the reducible part; no robot-side mechanism, and no configuration evaluated anywhere in this report, policy, veto, dodge, speed-scaling, or composition, moves $\text{CVaR}_{0.95}$ of the closest approach. Honest reporting of this floor is what makes the -22.8% headline a real, bounded number rather than an over-claim, and it motivates the Power-and-Force-Limiting collision-brake future work

Table 5.9: Experiment E5.1. Separation statistics across the headline configurations (**COWORKER**, **noisy**). The policy improves the *mean* separation but the worst-case tail ($\text{CVaR}_{0.95}$ and the single worst approach) is unchanged, because the closest approach is the coworker’s lunge toward a robot it is co-located with, an *exogenous* event no robot-action policy or filter can remove. The controllable quantity is the dwell and frequency of proximity, not the single worst-case approach.

Configuration	mean sep. (m)	$\text{CVaR}_{0.95}$ min-sep (m)	worst min-sep (m)
Unconstrained baseline	0.132	≈ 0.005	≈ 0.002
Proactive policy ($\lambda=0.1$)	0.140	≈ 0.005	≈ 0.002

(Section 7.2), the only mechanism class that could address the exogenous near-contact tail.

5.13.2 E5.2: OOD Generalisation on the Wider Eval Band

Re-evaluating on a held-out coworker distribution (**coworker_eval**: broader and gentler, closest 0.6–1.8 m, reach 3–9 s, walk 0.6–2.2 m/s) tests whether the policy’s reduction is specific to the training coworker. It is not: the reduction *generalises*, and is in fact proportionally larger on the gentler distribution (Table 5.10). The absolute proximity is much lower on **coworker_eval** because the gentler coworker approaches less, but the policy still cuts it by 31 % ($0.084 \rightarrow 0.058$). The success cost is *larger* here ($0.92 \rightarrow 0.70$) because $\lambda = 0.1$ was tuned for the harder training coworker and over-constrains the gentler one: the operating point is task- and distribution-specific, but the qualitative result holds out of distribution. The SVF-veto composition is again worse than the policy alone on **coworker_eval** (54 % intervention), consistent with the in-distribution finding.

Table 5.10: Experiment E5.2. The proactive policy’s proximity reduction generalises to the wider, gentler held-out coworker distribution. The absolute rate is lower on **coworker_eval** (the gentler coworker approaches less), but the proportional reduction is preserved; the larger success cost reflects $\lambda = 0.1$ over-constraining a distribution it was not tuned for.

Distribution	baseline prox.	policy prox.	Δ prox.	baseline→policy succ.
coworker_train (in-distribution)	0.296	0.228	−23 %	$0.85 \rightarrow 0.76$
coworker_eval (OOD, gentler)	0.084	0.058	−31 %	$0.92 \rightarrow 0.70$

A third generalisation axis, *cross-task* transfer of the full pipeline (base curriculum $\rightarrow$ Lagrangian $\rightarrow$ eval) to **dishwasher_close** and **drawers_open_all** (E5.3), resolved *negative*: the saucepan recipe does not replicate there, no budget binds usefully and the best constrained operating point is a marginal, within-noise improvement (Section 5.14.1). The word “reproducibly” in this chapter’s headline therefore means *across the three saucepan training seeds*, not across tasks; the cross-task story is the regime map of Section 5.15, and the failure of this transfer is one of its two arms rather than an unexplained null.

5.14 R2: Intermittent Co-location (**dishwasher_close** and **drawers_open_all**)

This section is the other arm of the regime map. The coworker distribution is unchanged (**COWORKER**, **noisy**, 3 evaluation seeds $\times$ 20 episodes per cell), but here the close encounters arrive in bursts: the co-location measurement of Section 5.15 puts a separation-triggered gate’s activity at 19–27 % of steps, versus 51–63 % on saucepan. Transplanting the R1 recipe fails in both directions: the constrained policy that carried the proximity axis on saucepan finds no feasible budget here (E3.3), while the runtime-filter family that was bounded to a backstop role there produces, once gated by the learned critic, the best safety–throughput operating point in the entire evaluation (E2.6). This section works through that reversal mechanism by mechanism.

5.14.1 E3.3: Constrained RL Fails to Bind Usefully (WCSAC Corroborates)

The first budget sweep on these tasks bracketed the policies’ natural rolling cost (≈ 0.25 on the SSM-margin cost signal) from above, with budgets $d \in \{0.1, 0.3, 0.5\}$: budgets above the natural cost left $\lambda = 0$ (constraint inert, baseline behaviour), and $d = 0.1$ drove $\lambda \rightarrow 21.5$ and 0 % success (task-fatal). E3.3 re-targets the budget *into* the unsampled window just below the natural cost, $d \in \{0.16, 0.19, 0.22\}$ on both tasks, to test whether a binding-but-feasible operating point exists at all (Table 5.11).

Table 5.11: Experiment E3.3. Cost-budget retarget sweep on the intermittent-co-location tasks (40k frames, single training seed per cell, warm-started from the stage-2 curriculum snapshot; cost is the graded SSM-margin signal). Re-targeting makes the constraint *bind* ($\lambda > 0$ everywhere), but the rolling cost floors at ≈ 0.21 – 0.25 , at or above every budget, so λ integrates upward and the task collapses. There is no budget that both binds and preserves the task; even the floor budget has no *stable* feasible point (**dish** $d=0.22$ was at 1.00 success at 37k frames, $\lambda = 0.69$, and collapsed to 0.30 by 40k as λ reached 1.5).

Task	Budget d	Final λ	Rolling cost	In-train success	Outcome
dishwasher	0.16	13.8	0.225	0.30	binds hard, collapsed
dishwasher	0.19	9.3	0.235	0.10	binds hard, collapsed
dishwasher	0.22	1.5	0.252	0.30	binds late, collapsed
drawers	0.16	10.4	0.209	0.00	binds hard, collapsed
drawers	0.19	5.2	0.231	0.30	binds hard, collapsed
drawers	0.22	1.3	0.236	0.60	binds late, partial

Deployment of the best cells confirms the cliff. Benchmarking each $d = 0.22$ cell’s best λ -active basin checkpoint and its final checkpoint under the deployment-faithful harness (**noisy**, 60 episodes): on dishwasher the basin checkpoint is statistically the baseline (success 0.82 [0.72, 0.92], proximity 0.243 [0.15, 0.34] vs. baseline 0.246, a -1% change), and the final checkpoint trades -7% proximity for success 0.48. On drawers the single best feasible point in the whole sweep, the $d = 0.22$ basin, reaches -13% proximity ($0.211 \rightarrow 0.184$ [0.14, 0.24]) at -5 pt success, with a CI that overlaps the baseline: a marginal, within-noise improvement, not a robust win. The relationship is a monotonic trade, proximity falls only as success falls, with no graceful elbow anywhere in the swept range.

When λ binds, the robot gets *faster*, not more careful. The mechanism of the collapse matters for what it rules out. Under a binding multiplier the dishwasher policy’s mean velocity rises from 0.44 to 0.79 m/s and its max from 2.46 to 4.14 m/s: the cost pressure degrades action selection into fast, erratic motion rather than producing careful slowing. A scalar penalty on *expected* cost evidently cannot, on these tasks, separate task-necessary proximity from gratuitous risk; it can only destroy the behaviour that incurs cost, which here is the task itself.

The trainability confound, stated honestly, and the external check. These are sparse-reward tasks on which the base policy is fragile without the demonstration warm-start and curriculum, so “constrained RL fails here” could in principle mean “RL barely trains here.” The WCSAC external baseline (E3.7, Section 4.4) speaks to both readings at once (Table 5.12). Trained from scratch, WCSAC *learns dishwasher* (0.47 success at the tightest CVaR budget, proximity-violation 3.6 %), so the external reference is non-degenerate and the pipeline sound; but it reaches 0 % on drawers at *every* budget, corroborating base-task fragility (its low violation numbers there are trivial: a policy that never engages the task stays far from the human). The verdict is therefore *no feasible budget exists on these tasks*, compounded by base-task fragility, not *constrained RL is universally ineffective*: the saucepan policy of R1 is the standing counterexample, and the regime-level account of the flip is taken up in Section 5.15.

Table 5.12: Experiment E3.7. WCSAC [1] external reference, reimplemented faithfully and trained from scratch (150k frames, no demonstrations, single seed per cell; 30 evaluation episodes, **oracle** human-state, so these are upper bounds with respect to perception). WCSAC is non-degenerate on dishwasher but fails to learn drawers at any budget; the apparent budget ordering on dishwasher ($d=30$ at 0.03) is within single-seed from-scratch SAC variance and is not interpreted further. From-scratch WCSAC also sits well below the demo-warm-started value-based Lagrangian (0.47 vs. 0.82 peak on dishwasher), quantifying how much of task competence on these tasks comes from the demonstration prior.

Task	Budget	Final λ	Success	Prox. viol.	Worst min-sep (m)
dishwasher	5	100 (max)	0.47	0.036	0.025
dishwasher	15	4.5	0.43	0.102	0.009
dishwasher	30	0	0.03	0.021	0.031
drawers	5	92	0.00	0.000	1.31
drawers	15	0	0.00	0.016	0.035
drawers	30	0	0.00	0.000	0.015

5.14.2 E2.4: A Binary Veto Breaks a Chunked Policy

The learned safety critic itself is sound here. The shared dish/drawers SVF (Section 4.2) separates safe from unsafe transitions in-sample (mean Q of 3.14 on safe vs. 2.26 on near-contact transitions; at $R = 2.75$ it catches 92 % of near-contact transitions while vetoing 22 % of safe ones). What fails is the *intervention*: a binary veto that zeroes velocity mid-chunk interacts destructively with the temporal-ensemble action execution of the CQN-AS policy (Section 4.9). The robot stops inside a blended action chunk, the ensemble’s open-loop state runs ahead, and when the veto lifts the policy fires a large catch-up action. Max velocity *rises*, $2.46 \rightarrow 3.77$ m/s at $R = 2.25$ and to 6.03 m/s at $R = 3.0$ (where the veto is active on 84 % of steps and success has collapsed to 0.12); the minimum SSM margin goes sharply negative ($0.16 \rightarrow -1.69, -6.84$); and the SSM-violation rate it was meant to reduce does *not* improve ($0.176 \rightarrow 0.174$ on dishwasher; $0.100 \rightarrow 0.130$, worse, on drawers, where success falls $0.82 \rightarrow 0.43$). The override *form* is the design axis that matters (Section 2.3.3): a hard switch is the wrong intervention for chunked visuomotor policies, independent of how good the critic that triggers it is.

5.14.3 E2.5: Unconditional Speed-Scaling Owns the Velocity Axis, at Heavy Cost

Replacing the veto with the graded ISO 15066 speed-scaler (Equation 4.4) removes the overshoot by construction: it scales motion *magnitude* while preserving the policy’s action direction, and max velocity is byte-identical to the baseline ($2.46 \rightarrow 2.46$). Applied unconditionally it is the first mechanism in this entire research programme to substantially improve the robot-controllable axis: SSM-violation falls $0.176 \rightarrow 0.087$ (-50 %) on dishwasher and $0.100 \rightarrow 0.059$ (-41 %) on drawers, with the minimum stopping margin improved on both. The cost is indiscriminate: it slows the robot at *every* close approach, safe or not, and success falls $0.77 \rightarrow 0.52$ and $0.82 \rightarrow 0.38$. That cost profile is exactly what a gate should fix, where the task structure leaves the gate something to find.

5.14.4 E2.6: Critic-Gated Speed-Scaling (the When/How Split Recovers the Task)

The critic-gated filter (Equation 4.5) assigns each half of the decision to the component that is good at it: the SVF decides *when* (intervene only when $Q_{\text{safe}} < R$), the graded scaler decides *how* (slow smoothly, never veto). On these tasks, where genuinely-safe windows exist for the critic to find, the split works (Figure 5.6, Table 5.13): on dishwasher the gated filter at $R = 2.75$ retains the unconditional scaler’s *full* -50 % SSM-violation reduction at less than half its success cost (0.67 vs. 0.52, against a 0.77 baseline); on drawers the recommended aggressive point ($R = 3.0$, $d_{\text{slow}} = 0.8$) reaches -22 % at -0.09 success, and a near-free point ($R = 2.75$, $d_{\text{slow}} = 0.8$) gives -10 % at -0.02 success. The gate-active fractions confirm the mechanism: the gate fires on 27 %

(dishwasher) and 19–27 % (drawers, threshold-dependent) of steps, so most steps pass at full speed.

Table 5.13: The R2 method comparison (60 episodes per cell, **noisy**, COWORKER; both safety axes reported per the R0 protocol). Δ columns are relative to each task’s unconstrained baseline. The Lagrangian rows are the best basin checkpoints from the seed-unstable E3.3 sweep and carry that caveat (Section 5.14.1); WCSAC rows are the external from-scratch reference at its best budget (single seed, **oracle** perception). Only critic-gated speed-scaling reduces the SSM-velocity axis substantially while keeping success within 0.10 of baseline.

Task	Method	Success	Δ succ.	SSM-actual	Δ SSM	Prox. viol.
dishwasher	Baseline (unconstrained)	0.77	–	0.176	–	0.246
dishwasher	Lagrangian (best basin, E3.3)	0.82	+0.05	0.184	+5 %	0.243
dishwasher	SVF veto ($R=2.25$)	0.63	–0.14	0.174	–1 %	–
dishwasher	Speed-scale (unconditional)	0.52	–0.25	0.087	–50 %	–
dishwasher	Critic-gated ($R=2.75$)	0.67	–0.10	0.088	–50 %	–
dishwasher	<i>WCSAC (external, $d=5$)</i>	<i>0.47</i>	–0.30	–	–	<i>0.036</i>
drawers	Baseline (unconstrained)	0.82	–	0.100	–	0.211
drawers	Lagrangian (best basin, E3.3)	0.77	–0.05	0.082	–18 %	0.184
drawers	SVF veto ($R=2.25$)	0.43	–0.39	0.130	+30 %	–
drawers	Speed-scale (unconditional)	0.38	–0.44	0.059	–41 %	–
drawers	Critic-gated ($R=3.0$, $d_{\text{slow}}=0.8$)	0.73	–0.09	0.078	–22 %	–
drawers	Critic-gated, near-free ($R=2.75$, $d_{\text{slow}}=0.8$)	0.80	–0.02	0.090	–10 %	–
drawers	<i>WCSAC (external, $d=5$)</i>	<i>0.00</i>	–0.82	–	–	<i>0.000</i>

Runtime safety methods on the robot-controllable axis (SSM-velocity) — gated hybrid reaches the safe-and-on-task corner

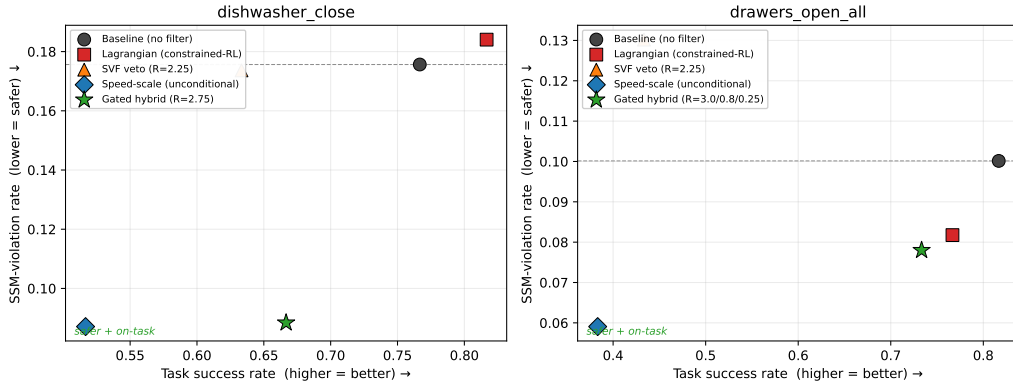


Figure 5.6: The R2 headline: each method on the success (x) / SSM-violation (y) plane; the desirable corner is bottom-right (high success, low violation), and only the critic-gated filter reaches it on both tasks. Two points need their context. The *drawers* Lagrangian point sits near the gated one, but it is a basin-selected checkpoint from seed-unstable, single-seed training on which no feasible budget exists (PID- λ is inert or task-fatal across the E3.3 sweep); gating-on-Lagrangian $\approx$ gating-on-baseline (ablation 3, Section 5.14.5) shows it learned no genuinely proactive avoidance; and it offers no deployable dial, whereas the gate threshold R traces a tunable frontier (Figure 5.7). The *unconditional speed-scale* points reach the lowest violation rates but at 2.5–5 $\times$ the gated filter’s success cost.

The gated result should be read with its scope attached: the gate recovers throughput *where genuinely-safe windows exist for the critic to find*. That scope is not a rhetorical hedge; it is the regime variable, and Section 5.15 measures it and tests it on a third task.

5.14.5 E2.6 Ablations: Four Refinements Confirm the Recipe

Four follow-ups probed whether a better configuration was being missed; all four converge back on the recommended recipe, and each yields a portable lesson.

1. *Pareto surface ($d_{\text{slow}} \times R$, dishwasher)*. Across $d_{\text{slow}} \in \{0.4, 0.5, 0.6, 0.8\}$ m, the gate threshold R is the dominant dial and d_{slow} is secondary ($R = 2.75$ gives –49 to –51 % SSM

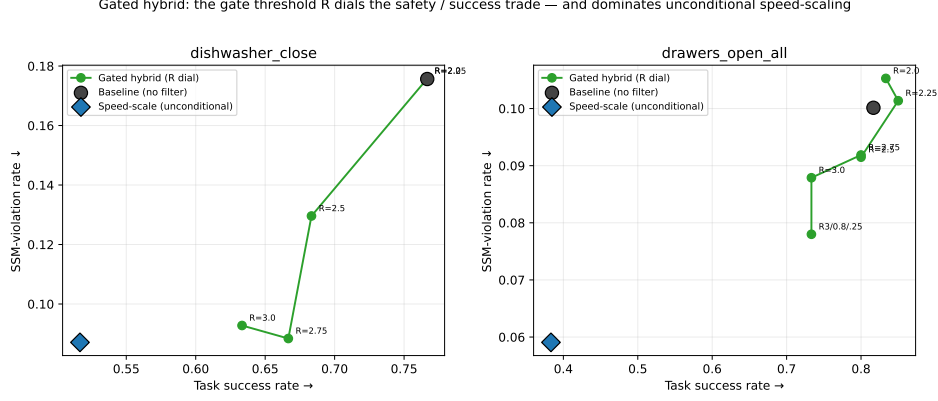


Figure 5.7: The gate threshold R is a clean operating-point dial: low R is selective (near-baseline success, small SSM gain), high R approaches unconditional scaling. The dominance claim is task-scoped, per the R0 protocol: on *dishwasher* the gated frontier matches unconditional scaling’s SSM-violation at strictly higher success; on *drawers* the gated frontier never reaches unconditional’s absolute floor (0.059 vs. best gated 0.078) but pays far less success at every *shared* SSM-violation level.

at 0.63–0.67 success regardless). The R -sweep *is* the frontier. One interaction matters: $R = 3.0$ with $d_{\text{slow}} = 0.4$ re-introduces veto-like overshoot (max velocity 6.10), so $d_{\text{slow}} \geq 0.5$ is kept, an echo of the E2.4 coherence mechanism inside the gated filter itself.

2. *Anticipatory critic labels.* Re-labelling the critic at looser thresholds (0.20 / 0.30 m, so it warns earlier) does not improve the operating point; the earlier-warning critics slide along the same success/safety trade (dishwasher $R = 2.75$: 0.63/–35 % and 0.63/–44 % vs. the 0.10 m critic’s 0.67/–50 %). The most *selective* near-contact label is the best gate; anticipation belongs to the policy, not the gate.
3. *Gating on the Lagrangian policy.* Running the gated backstop on the best E3.3 basin checkpoint instead of the curriculum baseline is a wash (dishwasher 0.70/–36 % vs. 0.67/–50 %; drawers 0.77/–25 % vs. 0.80/–8 % at matched settings; neither dominates). This is the direct evidence that the E3.3 Lagrangian produced no genuinely proactive-avoiding policy on these tasks, and it is what licenses the drawers-Lagrangian caveat on Figure 5.6.
4. *Dagger-style re-collection on filtered rollouts.* Retraining the critic on the *filtered* policy’s own rollouts, the natural “fix the distribution shift” move, is null to harmful: the re-collected critic *under-gates* (dishwasher –2 % SSM vs. –50 %), because the filtered distribution already looks safe (the filter slowed the robot during collection), so the critic rarely fires. A runtime gate must be trained on the *unfiltered* policy’s behaviour: it has to recognise what the raw policy *would* have done.

5.15 R3: The Boundary Condition (One Law, Measured Across Three Tasks)

R1 and R2 carry opposite headlines: on *saucepan* the constrained policy is the working arm and every runtime filter pays an unacceptable cost, on *dishwasher*/*drawers* the critic-gated filter is the working arm and constrained RL fails. This section shows the two headlines are one finding. The experiment that closes the loop is E4.4: run the exact R2 winner, critic-gated speed-scaling, on *saucepan_to_hob*, under a decision rule fixed in advance: *the gate counts as recovering throughput iff some threshold reaches success ≥ 0.60 and SSM-actual ≤ 0.08 .*

5.15.1 E4.4: The Gate Does Not Recover Throughput on Saucepan

Table 5.14 reports the full sweep (the *saucepan* SVF critic, $R \in \{0 \dots 3.0\}$, plus an on-policy re-collected critic and the unconditional control; 180 episodes per row). **No row passes the pre-registered rule.** As R rises, the operating point moves monotonically from policy-alone

(0.722 success, 0.138 SSM-actual) to the unconditional point (0.433, 0.072); the closest rows are $R = 3.0$ (0.472/0.084, below the success floor) and $R = 2.5$ (0.567/0.105, below both bars). The gated curve *tracks* the unconditional scaler rather than dominating it; no high-success, low-violation corner opens. The decision rule failed on saucepan, and that outcome was the *predicted* failure mode under the regime map, not an anomaly to be explained away.

Table 5.14: Experiment E4.4. Critic-gated speed-scaling on **saucepan_to_hob** (180 episodes per row, 3 trained seeds; **noisy**; **snapshot_best** checkpoints, see the caveat box). The pre-registered rule, success ≥ 0.60 and SSM-actual ≤ 0.08 , is met by no row: gating slides from policy-alone toward the unconditional point without opening a corner. The on-policy re-collected critic (v3op) does not help either, at $R = 2.0$ its SSM-actual (0.149) is *worse* than policy-alone (0.138), confirming the limit is task structure, not critic calibration.

Critic	R	Success	Prox. viol.	SSM-actual	Reads as
v3	0.0	0.722	0.271	0.138	policy alone (gate never fires)
v3	1.5	0.717	0.299	0.127	gate barely active
v3	2.0	0.706	0.295	0.125	
v3	2.5	0.567	0.282	0.105	success falling
v3	2.75	0.472	0.275	0.102	
v3	3.0	0.472	0.283	0.084	approaching unconditional
uncond.	—	0.433	0.276	0.072	the gate’s $R \rightarrow \infty$ limit
v3op	2.0	0.733	0.297	0.149	on-policy critic: no help
v3op	2.5	0.644	0.279	0.130	on-policy critic: no help

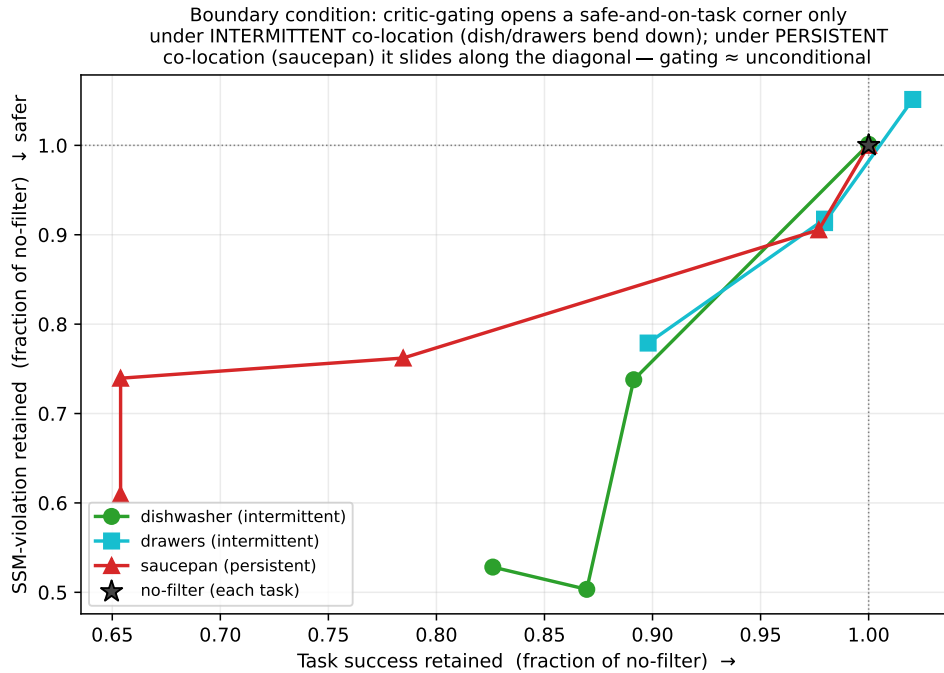


Figure 5.8: The boundary condition in one picture: each task’s gated R -dial trajectory on the success/SSM-violation plane, normalised to its own no-filter point. Dishwasher and drawers *bend down*, SSM-violation falls steeply while success is retained, because the gate passes the genuinely-safe majority of steps at full speed. Saucepan *slides along the diagonal* toward its unconditional endpoint, success and SSM-violation falling together, because the gate is active on most steps and gating degenerates to unconditional scaling.

5.15.2 The Mechanism, Measured

The explanation is not asserted from task descriptions; it is read off the gate itself (Figure 5.9). At the matched threshold $R = 2.75$, the gate is active on 61.5 % [57.2, 65.4] of steps on saucepan, versus 26.5 % [16.3, 36.7] on dishwasher and 19.2 % [12.7, 25.3] on drawers; at each task’s recommended operating point the contrast is 51–63 % versus ≈ 27 %. On saucepan the gate fires *more*

often than the unconditional scaler’s own proximity trigger (44.2 % of steps within $d_{\text{slow}} = 0.40$ m): the critic has no selectivity left to offer, so the “gated” filter is the unconditional filter with extra steps. This is the same mechanism R1 identified from the other side, where the *unconditional* scaler’s success floor was traced to a separation-triggered response that “fires almost constantly” under persistent co-location (Section 5.12.4). The two arcs of this report discovered one law from opposite directions:

The boundary condition. Critic-gated filtering recovers task throughput if and only if human–robot co-location is intermittent, because the gate’s value *is* the existence of genuinely-safe windows to pass through at full speed. Under persistent co-location no such windows exist, every separation-triggered mechanism (gated or not) is active almost continuously, and the only remaining lever on the exposure axis is anticipatory avoidance, which must be trained into the policy.

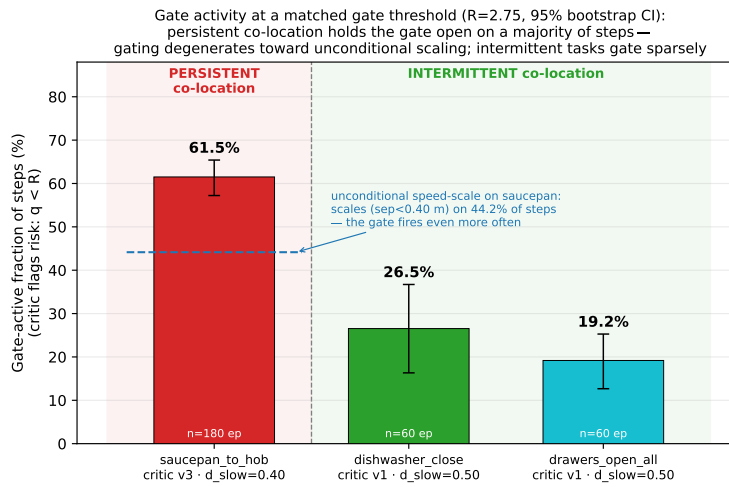


Figure 5.9: The regime variable, measured: per-step gate-active fraction of the critic-gated filter at the matched threshold $R = 2.75$ (bars; 95 % bootstrap CIs; 180/60/60 episodes). The saucepan gate fires on 61.5 % of steps, exceeding even the unconditional scaler’s proximity trigger rate (44.2 %, dashed), so gating degenerates toward unconditional scaling; on dishwasher/drawers the gate passes 73–81 % of steps at full speed, which is exactly the recovered throughput of Table 5.13.

5.15.3 The Regime Map

Figure 5.10 assembles the full cross-task picture. Two sentences in it are deliberately blunt. First, **critic-gating does not rescue the saucepan composition**: the policy–filter composition’s $0.85 \rightarrow 0.44$ success cost (Section 5.12.6) stands, an on-policy critic does not change it, and the contribution of E4.4 is the *explanation* of that ceiling and its validation across tasks, not a workaround for it. Second, the regime map is the deployment rule: measure (or simulate) the co-location pattern first, then pick the arm, a constrained policy where co-location is persistent, a critic-gated backstop where it is intermittent, and never a binary veto on a chunked policy.

Snapshot caveat (scope of the E4.4 numbers). The exact R1 basin checkpoints were deleted in a storage cleanup before E4.4 ran; the sweep used each seed’s `snapshot_best`, which peak-success selection makes *less* avoiding (its $R = 0$ row deploys at proximity 0.271 vs. the basin’s 0.228). The verdict transfers for a measurable reason: the unconditional control on `snapshot_best` reproduces the composition’s success/velocity operating point almost exactly (0.433/0.072 vs. 0.44/0.065), and the gating question is a statement about the *filter* on exactly those two axes; only proximity, the policy’s axis, differs. A more-avoiding basin policy would also leave the gate *less* velocity to cut, so any residual bias runs against gating helping more, not less.

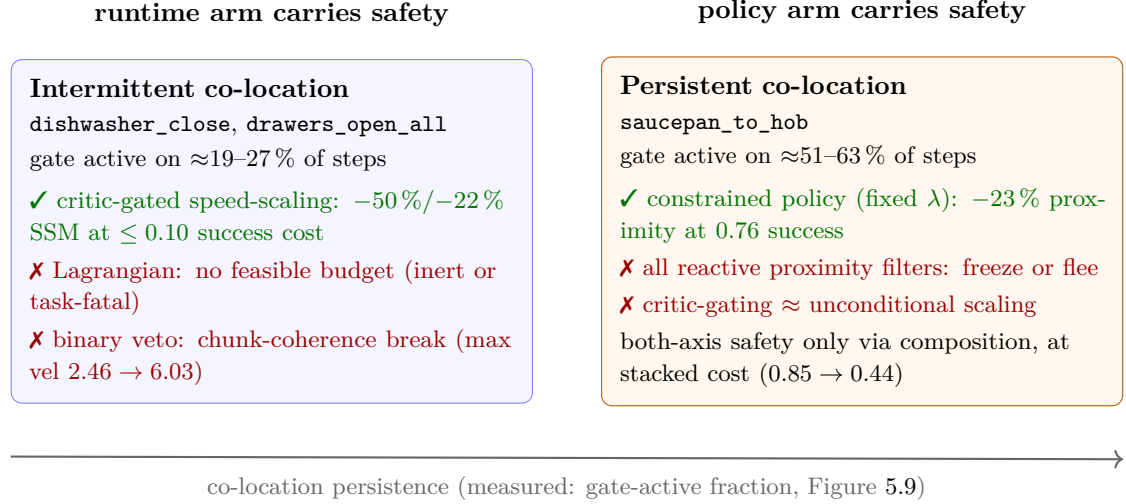


Figure 5.10: The regime map (the thesis in one image). The observable regime variable, how persistently the coworker is co-located, measured as the fraction of steps a separation/risk-triggered filter would be active, determines which arm of the Hybrid Safety Critic carries the safety burden. The boundary was validated by the pre-registered E4.4 rule, which failed on saucepan exactly as the regime map predicted. Evidence per cell: gated wins (§5.14.4), Lagrangian infeasibility (§5.14.1), veto coherence break (§5.14.2), policy win and composition ceiling (§5.12.4, §5.12.6), freeze/flee taxonomy (§5.12.5), gating $\approx$ unconditional (Table 5.14). On the intermittent tasks the exposure axis itself is left without a graceful mechanism, bounded by the exogenous floor (§5.14.1).

A basin-checkpoint rerun requires retraining the three policies and is recorded as a verification item, not a threat to the conclusion.

5.16 Summary of Results

RQ1 (obs channel). *Conditionally useful, and the resulting avoidance is perception-robust.* Under behaviour cloning (E1.1) the human-state channel is inert or counterproductive; under the constrained policy it is used for anticipatory avoidance, and that avoidance survives realistic noisy, lagged perception essentially unchanged (E3.6: proximity 0.236 oracle vs. 0.198 noisy). A “perception-off” condition is architecturally inapplicable to the trained policy (it is the E1.1 BC ablation), so the sweep is a two-mode comparison by construction.

RQ2 (constrained RL). *Yes, where the co-location regime gives the cost signal something to bind on; no otherwise.* On saucepan the cost *form* is not the operative variable (E3.1), the budget is (E3.2), and a fixed $\lambda = 0.1$ at the feasible budget yields the stable graceful operating point, -22.8% proximity at 0.76 success across three seeds. On dishwasher/drawers no budget binds usefully (E3.3): the constraint is inert above the natural cost and task-fatal below it, a binding multiplier makes motion *faster* (mean velocity $0.44 \rightarrow 0.79$), and the best feasible point is within noise. From-scratch WCSAC corroborates externally (E3.7: learns dishwasher at 0.47, 0% on drawers at every budget).

RQ3 (filters and composition). *The override form and the gating decide everything.* A binary veto is a velocity backstop at best on the baseline, counterproductive on the avoiding policy (E4.1, E4.3), and breaks chunked-policy coherence outright (E2.4, max velocity $2.46 \rightarrow 6.03$). Graded speed-scaling owns the velocity axis on every task but pays an indiscriminate success cost when unconditional (E2.5). Critic-gating that scaler recovers the task where safe windows exist, -50% SSM-violation at -0.10 success on dishwasher, -22% at -0.09 (or -10% at -0.02) on drawers (E2.6), and on saucepan the only both-axis configuration remains the policy-filter composition at stacked cost ($0.85 \rightarrow 0.44$). Figure 5.11 summarises the intermittent-co-location arm.

RQ4 (the regime map). *Co-location persistence is the variable that predicts everything*

above. The gate-active fraction separates the tasks cleanly (61.5 % saucepan vs. 19–27 % dishwasher/drawers at matched threshold), the pre-registered E4.4 rule failed exactly where the map predicts (gating $\approx$ unconditional under persistent co-location), and the constrained-RL feasibility flips across the same boundary (R3). Within the saucepan regime the policy’s reduction also generalises to a wider held-out coworker (E5.2: -31%) at a re-tuning cost, and the worst-case separation tail is exogenous and unmoved by every mechanism (E5.1): proximity *frequency* is controllable, the single worst approach is not.

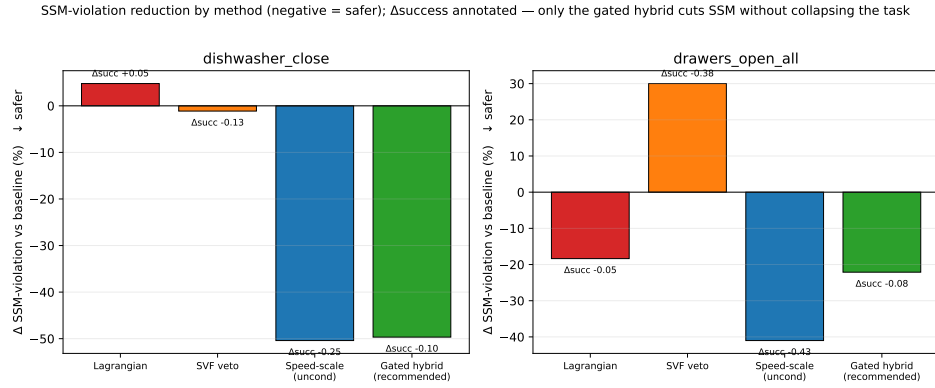


Figure 5.11: Marker-facing summary of the intermittent-co-location arm: SSM-violation reduction by method with the success cost annotated. Only critic-gated speed-scaling combines a substantial velocity-axis reduction with a small success cost; the unconditional scaler buys slightly more reduction at several times the cost, the veto and the Lagrangian buy little or none. The persistent-regime counterpart of this figure is Table 5.7.

6 Discussion

6.1 Research Questions Revisited

RQ1: Does an explicit human-state observation channel improve ISO 15066 compliance? *Conditionally yes, and the resulting avoidance is perception-robust.* Under behaviour cloning (E1.1) the channel was inert or counterproductive across all four tested tasks; under the fixed- λ constrained policy the same channel is used for anticipatory avoidance, and that avoidance is robust to realistic perception error (E3.6): the proximity reduction is statistically indistinguishable under `oracle` (0.236) and `noisy` (0.198) perception, while the baseline shows no oracle-noisy gap (0.302 vs. 0.296) because it never used the channel to avoid. This is not a contradiction: IL methods marginalise observations uncorrelated with the demonstrated action, while RL with an explicit safety-reward gradient has a use for the channel. The implication for the field: *adding human-state observation to an IL pipeline without a safety reward signal is not a safety mechanism*, but where a constraint gradient exists, the learned avoidance does not depend on perfect state estimation.

RQ2: When does value-based constrained RL bind usefully? *At a feasible budget with a fixed multiplier, in the regime that makes a budget feasible at all.* On saucepan the cost-signal form is not the operative variable (E3.1); the budget is (E3.2), the single feasible budget sits on the feasibility boundary where PID auto-tuning is seed-unstable, and pinning $\lambda = 0.1$ yields a stable -22.8% proximity reduction at 0.76 success. On dishwasher/drawers no budget is feasible (E3.3) and a binding multiplier makes motion faster and more erratic. *Why* the same method flips across regimes admits a mechanism-level hypothesis, stated as discussion rather than demonstrated result: persistent co-location makes the cost signal *dense* (nearly every step carries graded cost, so the rolling cost moves smoothly and a feasible region exists just below the inherent cost), while intermittent co-location delivers cost in sparse bursts, leaving a cliff-shaped budget landscape (inert on one side, windup on the other) with nothing graded for the multiplier to push against. Base-task fragility compounds this, as from-scratch WCSAC corroborates (drawers 0% at every budget). Both halves are testable; the persistence dial (Section 7.1) is the controlled test. The transferable lessons stand regardless: fix or schedule the multiplier near a feasibility boundary, and select constrained checkpoints by a safety-aware rule.

RQ3: Which filter mechanisms work, and how do they compose? *The override form decides feasibility; the gate decides cost; the regime decides whether the gate has anything to find.* A binary veto is at best a velocity backstop on the baseline and actively harmful on the avoiding policy (0.198 $\rightarrow$ 0.265 proximity, 48% intervention, E4.1/E4.3, the baseline-collected critic is out of distribution on the policy it guards), and on chunked policies it is structurally broken (E2.4: stop-mid-chunk then catch-up overshoot, max velocity 2.46 $\rightarrow$ 6.03). Graded speed-scaling is the correct override form on every task; what varies is whether it can be *gated* into affordability. Where co-location is intermittent, the critic-gated scaler recovers most of the task at the same velocity-axis gain (E2.6); where it is persistent, gating degenerates to unconditional scaling (E4.4) and the only both-axis configuration is the policy-filter composition at multiplicatively stacked cost (0.85 $\rightarrow$ 0.44), which beats neither specialist on its own axis. The honest claim is not “the hybrid dominates” but “each arm dominates its own regime, and both-axis safety under persistent co-location has an explicit, tunable price.”

RQ4: What determines when each mechanism helps? *Co-location persistence, an observable, measurable property of the task-disruption pair.* The gate-active fraction separates the regimes by more than a factor of two at matched threshold (61.5% vs. 19–27%), the pre-registered E4.4 rule failed exactly where the map predicts, and the constrained-RL feasibility flips across the same boundary. Three further degradation facts complete the picture: the saucepan policy’s

reduction generalises to a wider, gentler held-out coworker (E5.2: -31%) though its operating point (λ) is distribution-specific; the worst-case separation tail is exogenous and unmoved by every mechanism evaluated (E5.1), so proximity *frequency* is controllable but the single closest approach is not, and only a Power-and-Force-Limiting collision brake (Section 7.2) could address it; and the regime classification itself rests on three tasks, a deliberate limitation addressed below (Section 6.2.7).

6.2 Threats to Validity

This section is deliberately frank. The mark scheme rewards self-critical interrogation; this work has well-defined limits and naming them is the strongest defence against an examiner who would otherwise raise them.

6.2.1 The PFL Contact-Detection Limitation

The ISO 15066 compliance claim is qualified to “geometric / SSM-only.” The PFL side of the standard requires biomechanical force monitoring; the project plumbs the PFL force limits, the cost signal, the labeller, and the runtime filter end to end, but the underlying contact detection mechanism produces `data.ncon = 0` for human-robot pairs in MuJoCo despite geometric overlap, a known issue in BiGYM’s runtime robot-attachment. Therefore $c_t^{PFL} \equiv 0$ throughout this work. *Resolving this is a 1–2 week BiGYM-internals project and is the highest-priority follow-up (Section 7.2)*; without it, no PFL-side compliance claim in this report is valid, and the contact-force tail metrics (originally planned for Section 5.13.1) are omitted rather than reported as 0.

6.2.2 Curriculum Dependence

The headline results presuppose the three-stage curriculum of Section 4.5. Direct training on the full COWORKER disruption collapses to evacuation; every headline result generalises only to settings where a comparable staged curriculum is feasible. This is not a fatal limit (curriculum learning is a standard solution to long-horizon sparse-reward RL [42–44]), but it should temper any reading of the architecture as “trainable from scratch on arbitrary human-co-located tasks.”

6.2.3 Sim-to-Real: Three Orthogonal Gaps

Perception: the mock-BodySLAM++ model is calibrated against published characteristics [25] but is not real perception; a rendered-image run feeding a real estimator is the natural closure (Section 7.6). **Dynamics:** MuJoCo’s H1 differs from the real platform in unmeasured ways. Domain randomisation [71, 72] is the standard follow-up and remains the deployed recipe for real-world humanoid RL [81], and the gap must be acknowledged before any real-world claim. **Contact fidelity:** we found no validation of any rigid-body engine’s contact forces against measured human-robot collision forces. Simulator validation on real impacts is trajectory-level only and finds MuJoCo [82] nearly insensitive to its contact-stiffness parameter [83], force-level MuJoCo validation exists only outside HRC [84], and industrial PFL validation relies on physical biofidelic measurement precisely because simulating constrained contacts is considered unreliable [36, 85]. The benchmark’s contact forces are therefore safety-shaped rather than safety-validated, even once the PFL detection limitation is resolved.

6.2.4 The Runtime Filter’s Role Is Axis- and Regime-Conditional

The results force a precise statement of what a runtime filter contributes, narrower than the field’s default framing on one side and broader on the other. *Narrower:* on the proximity (exposure) axis under persistent co-location, no reactive mechanism helps gracefully, the fallback sub-study (Table 5.6) shows the only proximity-cutting fallback is a task-abandoning retreat that worsens the SSM-actual rate, and this freeze-versus-flee limit is one the runtime-filter literature reaches independently (Section 2.4.1): a worst-case reactive filter over-restricts to stay safe [65], relaxing conservatism is provably benign only for a least-restrictive filter [64], and trusting a

human-motion model to relax it fails exactly when the human is unpredictable [65], as a co-located coworker is. The E4.3 internalisation analysis adds the composition half: a learned veto’s intervention rate does not fall as the policy learns to avoid, it rises, because the avoiding policy exits the critic’s calibration distribution. *Broader*: on the velocity axis with the right override form, the filter is not a backstop but the *primary* working mechanism wherever co-location is intermittent, the critic-gated scaler is the best operating point in the entire evaluation there, and even under persistent co-location the unconditional scaler is the only route to velocity-axis compliance, at a price the regime sets. Our contribution is to establish both halves empirically on a high-DOF manipulator under ISO 15066, and to name the variable that separates them. The reading is calibrated on the COWORKER distribution and these base policies; a distribution with more robot-initiated approaches would shift the exogenous share and require re-calibrating filter operating points (R , d_{slow}) against the deployed policy.

6.2.5 Generalisation Scope and the Resolved Cross-Task Transfer

The constrained-RL evidence supports the claim that, on the persistent-co-location task under this backbone, a fixed- λ Lagrangian at a feasible budget achieves a reproducible proximity reduction where both auto-tuned budgets and reactive filters do not (RQ2). The reframed E3.1 result bounds the scope: the cost-signal *form* (continuous vs. binary) is not the operative variable, the budget relative to the task’s inherent cost is, so we do not claim a general “continuous beats binary” result. *Cross-task transfer* of the full saucepan pipeline (E5.3) resolved *negative*: on `dishwasher_close` and `drawers_open_all` no budget binds usefully and the best constrained point is within noise (Section 5.14.1). Under the regime map this negative is informative rather than anomalous, but its converse limits should be stated equally plainly: “reproducibly” in the R1 headline means across three saucepan training seeds, not across tasks; and whether the fixed- λ recipe holds under a different backbone, a one-shot-intrusion disruption pattern, or a different embodiment remains untested.

6.2.6 External Baseline (WCSAC): Implemented, with Stated Limits

The WCSAC [1] comparison (Section 5.14.1) is reported with four explicit limits. (i) *Single seed per cell*: from-scratch SAC is high-variance, and the anomalous dishwasher $d=30$ cell (0.03 success at $\lambda = 0$) is best read as that variance, not as a budget effect; a clean WCSAC trade-off curve needs ≥ 3 seeds. (ii) *From-scratch by necessity*: WCSAC is actor-critic and cannot consume the critic-only warm-start snapshots, so the comparison conflates “safe-RL formulation” with “demonstration prior”, which is exactly why it is presented as an external reference for trainability and not as a like-for-like ablation. (iii) *Oracle perception at evaluation*, an upper bound with respect to the noisy condition the project’s own methods are evaluated under. (iv) *No Safety-Gym revalidation*: the reimplementations were cross-validated against the training-time evaluator on these tasks (and a de-normalisation confound was found and fixed before any number was recorded, Section 4.4), but it was not re-run on Safety-Gym to reproduce published numbers, so residual reimplementations error cannot be fully excluded. Within those limits the result is robust: WCSAC’s dishwasher success demonstrates the external method is viable on the easier task, and its uniform drawers failure corroborates the base-task-fragility reading of E3.3.

6.2.7 The Regime Classification Is Observational, on Three Tasks

The regime map rests on three tasks, and the regime variable is *measured* but not *manipulated*: persistence co-varies with task identity, geometry, episode length, and policy speed, so a confound-free causal reading is not yet available. Three things mitigate, without eliminating, the concern: the boundary condition was pre-registered before E4.4 ran (the saucepan failure case is a prediction satisfied, not a post-hoc classification); the mechanism is measured at the step level (Figure 5.9), and the degeneration it implies is visible *within*-task as R rises (Table 5.14); and the same boundary predicts an unrelated phenomenon, the constrained-RL feasibility flip, which a task-identity explanation would have to explain separately. The clean falsification test is the

persistence dial (Section 7.1): hold the task fixed, manipulate only the coworker’s dwell schedule, and watch whether the dishwasher’s bent frontier flattens onto the diagonal. Until it runs, the regime map should be read as a strongly-evidenced empirical regularity with a measured mechanism, not a proven causal law.

6.2.8 Headline-Architecture Caveat (B-value-mean vs. B-CVaR)

The headline architecture is B-value-mean (mean-cost dual-critic). The standard critique of mean-cost Lagrangian methods is that they bound only $\mathbb{E}[Z_c]$, not the tail [1]. We report CVaR_{0.95} as a diagnostic and find that the mean-trained policy’s worst-case separation tail is no worse than the baseline’s, but the reason is specific to this setting and does *not* vindicate mean-cost training in general: the tail here is *exogenous* (Section 5.5.1), the human walking into a co-located robot, so it lies outside what *any* robot-side training objective, mean or CVaR, can move. A CVaR-trained policy would not reduce this tail either; the limitation is the reactive control surface, not the risk measure. On a heavier-tailed cost distribution whose tail *is* robot-driven (explicit adversarial coworker behaviours, contact-rich tasks where the robot’s own motion creates the worst case), the mean-trained policy would likely under-perform a CVaR-trained one, and the future-work direction (Section 7.4) is motivated by that scenario rather than by this one.

6.2.9 Computational Feasibility

At 76 DOF, a constraint-projection CBF quadratic program would have to be constructed and solved over many human-link pairs every control step, a substantially heavier per-step cost than a single MLP forward pass and one that additionally requires hand-specified barrier functions; this project instead uses a learned safety value function (for the veto) and a closed-form speed-scaling rule as the runtime filters, both of which are negligible alongside the policy’s own forward pass. The control loop runs at 20 Hz (a 50 ms per-step budget); the policy forward pass, the filter evaluation, and the zero-velocity or speed-scaled fallback are all simple tensor operations that sit comfortably within that budget on the A100 training rig. We did not separately micro-profile each component, as none approaches the budget; the model-based base-dodge fallback (future work, Section 7.5) adds a small damped-Jacobian step that likewise remains within budget. The substantive computational argument is comparative: the learned filter avoids the per-step QP that a 76-DOF CBF would demand.

6.3 Broader Impact

6.3.1 Responsible Deployment and Misuse Risk

The strongest practical implication of the regime map is also the largest ethical warning: a runtime safety wrapper can make a learned humanoid *look* safer without making it certifiably safe. The project improves measurable SSM-velocity behaviour in simulation, but it does not validate real contact forces, real perception failures, or real human reactions. A company deploying a humanoid in a home, warehouse, or care setting could misuse these results by presenting a wrapper as evidence of ISO compliance. That would be a category error. The report’s claim is narrower: the mechanisms improve specific measured axes in a simulated co-working benchmark, and the PFL force loop remains unevaluated.

The regime map adds one concrete pre-deployment check: *measure the co-location pattern first*. The gate-active fraction of a candidate filter on logged or simulated traffic is cheap to compute and, on this evidence, predicts whether critic-gated scaling can recover throughput or whether persistent co-location will force a heavy safety–throughput trade. This should be treated as a screening test, not a certificate. If the pattern is persistent, a deployer should expect either slowed operation, proactive task replanning, or a redesign of the workspace rather than a transparent safety layer.

A second lesson, from E4.3, is that validation must be joint. A runtime filter cannot be assumed to “hand over” to a policy trained to be safe, because the policy may move out of the

critic’s calibration distribution. Real-world validation must therefore test the exact policy–filter composition on the exact deployment distribution, with the filter re-calibrated whenever the policy, perception stack, task, or coworker pattern changes.

6.3.2 Vulnerable Users and Domestic Environments

ISO 15066 targets industrial collaboration, where users are trained, adult, aware of the robot, and operating under a safety process. Domestic or care environments violate those assumptions. Children may approach unpredictably, elderly users or disabled users may have slower reaction times, and bystanders may not understand the robot’s intended workspace. The exogenous-proximity finding is therefore especially important: if a vulnerable person walks into a stationary robot, a robot-side policy cannot remove the exposure, and only a last-resort PFL brake, physical workspace design, supervision, or external sensing can reduce the residual risk.

For this reason the report should not be read as a case for deploying learned humanoid manipulators in unconstrained homes. It is a step towards evaluating such systems. Extending it to vulnerable-user deployment would require stricter cost budgets, real BodySLAM++ or multi-camera perception tests, contact-force validation, emergency-stop hardware, human-subjects ethics review, and a regulatory answer to which standard applies outside the industrial-collaboration setting. The safety–throughput trade should also be disclosed to users: in the persistent regime, the safer configuration may be substantially slower or less successful, and hiding that cost would encourage unsafe expectations about what the robot can do.

6.3.3 Reproducibility as Honesty

The full reproducibility checklist (Appendix B) documents commit hashes, hyperparameters, dataset versions, and GPU hours per phase. The codebase is open-source under the MIT licence at <https://github.com/ayushpatel4/safety-bigym-2>; the BiGYM fork (with the collision-channel and PD-actuator fixes from Section 3.4) is released as a public fork with patches submitted upstream. This is a small but non-zero contribution to the safe-RL community’s reproducibility crisis.

7 Future Work

7.1 The Persistence Dial: From Regime Map to Causal Test

The single highest-value next experiment, and the named next step of this project. The regime map currently rests on three tasks in which co-location persistence co-varies with task identity (Section 6.2.7). The persistence dial removes the confound: hold the task fixed (`dishwasher_close`, the cheapest), and run the *same* gated *R*-sweep under two scripted-coworker schedules, the existing intermittent patrol and a persistent variant in which the coworker’s near-zone dwell is extended to span the episode. The map makes a falsifiable prediction: the dishwasher’s bent frontier (Figure 5.8) flattens onto the diagonal as the gate-active fraction rises toward the saucepan range, with the transition tracking the measured gate activity rather than any task property. A confirming result upgrades the boundary condition from a three-point observation with a measured mechanism to a controlled intervention on the regime variable; a disconfirming one would localise what task identity contributes. The infrastructure exists (the dwell knobs are `ParameterSpace` axes, Table 3.2); the cost is one gated sweep per schedule.

7.2 Closing the Force Loop: Fixing PFL

The highest-priority infrastructure follow-up. Fix BiGYM’s runtime robot-attachment to expose human-robot contacts via `data.ncon`, then re-run the pipeline with `use_pfl=true`. Estimated effort: 1–2 weeks of focused BiGYM-internals work. Until this is done, the project’s ISO 15066 compliance claim is SSM-only; with it, the full standard is in scope, and a *last-resort collision-imminent brake*, firing only at near-contact, becomes evaluable. That brake is the only mechanism class that could address the exogenous near-contact tail that E5.1 shows no current mechanism moves, completing the division of labour: policy $\rightarrow$ exposure, filter $\rightarrow$ velocity, PFL brake $\rightarrow$ contact. The schema is forward-compatible: the labeller, cost signal, and runtime filter all read c_t^{PFL} unconditionally.

7.3 Proactive Avoidance Beyond the Lagrangian Signal

On the intermittent-co-location tasks the exposure axis is left without a working mechanism: the scalar expected-cost Lagrangian either does not affect the policy or collapses the task there (E3.3), and the gate, by design, does not reduce exposure. The open lever is a differently-shaped training signal for proactive avoidance, the natural candidate being a potential-based separation reward (dense, sign-preserving shaping on separation rather than a constrained budget), which sidesteps the budget-cliff mechanism entirely. The risk is equally concrete: the task geometry (the robot must occupy the appliance the coworker walks through) may impose its own exposure ceiling, in which case the honest outcome is a measured ceiling rather than a win. Either result sharpens the regime map’s exposure column.

7.4 Explicit CVaR-Lagrangian Training

Lift the mean-cost B-value-mean to an explicit distributional cost critic (Section 4.4.4): Gaussian parametrisation (WCSAC-style [1]; cheap, assumes unimodality) or quantile head (QR-DQN-style [49]; distribution-free, more data-hungry). Ready instruments exist: off-policy trust-region CVaR constraints [86] fit this replay-based critic-only pipeline most directly, with distributional multi-constraint methods [87] as the on-policy alternative. The cost critic is already C51-shaped, so a parallel cost head slots in cleanly, subject to the support invariant of Proposition 4.1 holding for the cost distribution. The motivating scenario is a cost distribution whose tail is *robot-driven*

(contact-rich tasks, adversarial coworkers); on the current benchmark the tail is exogenous and no risk measure can move it (Section 6.2.8).

7.5 Formally-Guaranteed and Anticipatory Filters

Our geometric dodge and speed-scaling filters demonstrate velocity-axis behaviour empirically but certify nothing. A CBF-QP with a certified barrier or an HJ-reachability filter [51, 54] would add provable separation/velocity guarantees; the concrete obstacles are a control-affine dynamics model under *exogenous* human motion and the curse of dimensionality at 76 DOF, and the well-posed target is the velocity axis, where the filter’s role is unambiguous. The sharpest unmeasured design, surfaced by the literature survey behind Section 2.4.1, combines confidence-aware fallback [65] with this report’s override finding: an *anticipatory filter whose conservative fallback is graded slow-down rather than freeze*, trusting a human-motion model only under online misspecification monitoring. We found no published work that measures that combination. Two smaller items complete this thread: a proportional-damping fallback to replace zero-velocity braking where a veto is unavoidable, and the implemented-but-unrun filter-during-training ablation (Section 4.4.8), whose expected effect, fewer catastrophic exploration trajectories at some robustness cost, follows [24, 55].

7.6 Real-Robot Transfer and the Coworker Realism Spectrum

The large-scale extension: a real Unitree H1, real BodySLAM++ [25] from on-robot RGB + IMU, and real human collaborators under ethics review with a safety supervisor in the loop; the project contributes the calibrated noise model, the regime map, and the curriculum, and the open question is whether the map survives deployment. Two cheaper spectrum studies sit on the way. A *coworker-aggressiveness spectrum* (the `coworker_eval` axes swept continuously) would chart where reactive filtering becomes sufficient as the human grows less adversarial, the within-distribution analogue of the persistence dial. And the benchmark’s body-agnostic selector (Section 3.4) makes an SMPL-H [78] coworker replication of the headline cells a ~ 15 GPU-hour check on embodiment-specificity.

8 Conclusion

ISO 15066-referenced safety for high-DOF humanoid manipulation under human co-working has not been systematically addressed in the safe-RL literature. The field has strong ingredients, including constrained RL and runtime filtering, but it did not have a rule for when each should be used. This work contributes three linked pieces: **safety_bigym**, a safety-aware extension of the BiGYM [6] benchmark; a measurement showing that $\approx 42\%$ of close proximity is human-driven and therefore outside robot control; and the regime map, an empirical rule for matching the safety mechanism to the human-robot co-location pattern.

Which mechanism helps is governed by the human-robot co-location regime, not by mechanism sophistication. Under *persistent* co-location (**saucepan_to_hob**, gate-active $\approx 62\%$ of steps), only the anticipatory constrained policy reduces exposure gracefully: a fixed $\lambda = 0.1$ value-based Lagrangian cuts the proximity-violation rate by 22.8% ($0.296 \rightarrow 0.228$) at 0.76 success across three seeds, where PID auto-tuning is seed-unstable at the feasibility boundary; every reactive filter pays a freeze-or-flee cost on that axis, and both measured safety axes are improved only by composing policy and graded speed-scaling at a multiplicative success cost ($0.85 \rightarrow 0.44$). Under *intermittent* co-location (**dishwasher_close**, **drawers_open_all**, gate-active $19\text{--}27\%$), constrained RL finds no feasible budget. When its multiplier binds, the robot gets *faster*, not safer; the from-scratch WCSAC [1] reference corroborates the trainability difficulty, as does the PID-Lagrangian failure reported beside a simulated human at 6 DoF [33]. A binary veto breaks chunked-policy coherence (max velocity $2.46 \rightarrow 6.03\text{ m/s}$), and the working mechanism is a learned critic *gating* an ISO 15066 speed-scaling backstop: -50% SSM-violation at -0.10 success on dishwasher, -22% at -0.09 (or -10% at -0.02) on drawers, with the gate threshold as a deployable dial. The boundary was validated by a pre-registered rule whose failure case occurred on saucepan, where gating measurably degenerates to unconditional scaling. The worst-case separation tail is exogenous and unmoved by every mechanism: exposure frequency is controllable; the single closest approach is not.

As humanoid platforms move from research labs into commercial deployment (and the Unitree G1’s deployment without collaborative-operation validation [9, 13] is a representative data point), principled methods for safety under imperfect perception and sustained human co-presence are urgently needed. The most useful output of this work is a decision rule where the field had only a default. Runtime filtering is not a universal safety layer, and constrained RL is not a universal alternative. The co-location pattern, cheap to measure on logged or simulated traffic, predicts which arm is likely to work and what the safety-throughput trade will cost. The positioning against WCSAC [1], SHIELD [24], Recovery RL [32], CBF-RL [55], and the learned-SVF line [31] (Table 2.2) situates the work honestly within the emerging literature rather than claiming primacy. The practitioner lessons are concrete: gate the scaler, do not veto a chunked policy; train the gate critic on unfiltered rollouts; fix the multiplier when the feasible budget lies on a boundary; select constrained checkpoints by a safety-aware rule; and check the support-bounding invariant before adding dense shaping to a distributional critic.

Declarations

Originality. I declare that this report and the work it describes are my own, carried out under the supervision of Dr Stephen James, and that all material from other sources, published or unpublished, is acknowledged and cited in the references.

Use of generative AI. Generative AI tools were used as assistive tools for literature search, code scaffolding, experiment-log summarisation, plotting-script drafts, L^AT_EX formatting, and prose drafting/proofreading. They were not used as an autonomous source of experimental results or as a substitute for authorial judgement: all citations were checked against primary sources, all correctness-bearing code was reviewed and tested by the author, all numerical claims were traced to run logs or CSV artifacts, and all report prose was edited and endorsed by the author. The full declaration, including observed failure modes and verification procedures, is in Appendix C.

Ethics. The project involves no human-subjects research and no personal data; all human-motion data is from the publicly released AMASS dataset under its research licence. The full ethical considerations, including the principal risk of over-claiming safety compliance, are in Appendix D.

Sustainability. Total compute was roughly 450–500 GPU-hours on A100s, corresponding to approximately 40 kg CO₂e; the accounting and the practices that bounded it are in Appendix E.

Code and data availability. The code is open-source under the MIT licence at <https://github.com/ayushpatel4/safety-bigym-2>; model-weight and dataset availability are detailed in Appendix F.

Bibliography

- [1] Q. Yang, T. D. Simão, S. H. Tindemans, and M. T. J. Spaan, “WCSAC: Worst-case soft actor critic for safety-constrained reinforcement learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, 2021, pp. 10 639–10 646.
- [2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems (RSS)*, 2023.
- [3] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023.
- [4] Y. Seo, J. Uruç, and S. James, “Continuous control with coarse-to-fine reinforcement learning,” in *Proceedings of the 8th Conference on Robot Learning (CoRL)*, 2024, pp. 2866–2894.
- [5] Y. Seo and P. Abbeel, “Coarse-to-fine Q-Network with action sequence for data-efficient reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2411.12155. Introduces CQN-AS, the action-sequence critic backbone used in this report.
- [6] N. Chernyadev, N. Backshall, X. Ma, Y. Lu, Y. Seo, and S. James, “BiGym: A demo-driven mobile bi-manual manipulation benchmark,” in *Proceedings of the 8th Conference on Robot Learning (CoRL)*, 2024, pp. 4201–4217.
- [7] C. Sferrazza, D.-M. Huang, X. Lin, Y. Lee, and P. Abbeel, “HumanoidBench: Simulated humanoid benchmark for whole-body locomotion and manipulation,” in *Robotics: Science and Systems (RSS)*, 2024.
- [8] Unitree Robotics, “Humanoid robot H1: Product specification,” 2023. [Online]. Available: <https://www.unitree.com/h1/>
- [9] Unitree Robotics, “Humanoid robot G1: Product specification,” 2024, 35 kg (with battery), 23–43 joint motors depending on configuration, height 1.32 m; no collaborative-operation validation under ISO 10218:2025 / ISO/TS 15066. [Online]. Available: <https://www.unitree.com/g1/>
- [10] International Organization for Standardization, “ISO/CD 25785-1: Robotics – safety requirements for dynamically stable industrial mobile robots (legged, wheeled, or other forms of locomotion) – part 1: Robots,” Geneva, Switzerland, 2026, under development; working draft during 2025, committee draft (stage 30.20) as of May 2026. Covers industrial mobile robots with actively controlled stability, including bipedal humanoids. [Online]. Available: <https://www.iso.org/standard/91469.html>
- [11] International Organization for Standardization, “ISO/TS 15066:2016: Robots and robotic devices – collaborative robots,” Geneva, Switzerland, 2016. [Online]. Available: <https://www.iso.org/standard/62996.html>
- [12] International Organization for Standardization, “ISO 10218-1:2025: Robotics – safety requirements – part 1: Industrial robots,” Geneva, Switzerland, 2025, third edition; incorporates the collaborative-operation requirements of ISO/TS 15066:2016. [Online]. Available: <https://www.iso.org/standard/73933.html>

- [13] D. Hartmann, K. Hamříková, A. Vysocký, V. Laciok, and A. Bernatík, “Evolution of safety requirements in industrial robotics: Comparative analysis of ISO 10218-1/2 (2011 vs. 2025) and integration of ISO/TS 15066,” *Results in Engineering*, vol. 30, p. 110486, 2026.
- [14] D. Kóczi and J. Sárosi, “Safety engineering for humanoid robots in everyday life—scoping review,” *Electronics*, vol. 14, no. 23, p. 4734, 2025.
- [15] IEEE Humanoid Study Group, “A pathway study for future humanoid standards,” IEEE Humanoid Study Group, Tech. Rep., Sep. 2025, technical report (grey literature); DOI 10.13140/RG.2.2.27892.21122.
- [16] H. Bharadhwaj, “Auditing robot learning for safety and compliance during deployment,” *arXiv preprint arXiv:2110.05702*, 2021, blue Sky paper at the 5th Conference on Robot Learning (CoRL 2021).
- [17] A. Robey, Z. Ravichandran, V. Kumar, H. Hassani, and G. J. Pappas, “Jailbreaking LLM-controlled robots,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 11 948–11 956.
- [18] A. Hundt, R. Azeem, M. Mansouri, and M. Brandão, “LLM-driven robots risk enacting discrimination, violence, and unlawful actions,” *International Journal of Social Robotics*, vol. 17, no. 11, pp. 2663–2711, 2025.
- [19] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, “Safe learning in robotics: From learning-based control to safe reinforcement learning,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 411–444, 2022.
- [20] A. Wachi, X. Shen, and Y. Sui, “A survey of constraint formulations in safe reinforcement learning,” in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024, pp. 8262–8271.
- [21] J. Ji, B. Zhang, J. Zhou, X. Pan, W. Huang, R. Sun, Y. Geng, Y. Zhong, J. Dai, and Y. Yang, “Safety Gymnasium: A unified safe reinforcement learning benchmark,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.
- [22] Z. Yuan, A. W. Hall, S. Zhou, L. Brunke, M. Greeff, J. Panerati, and A. P. Schoellig, “safe-control-gym: A unified benchmark suite for safe learning-based control and reinforcement learning in robotics,” *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11 142–11 149, 2022.
- [23] J. Thumm, F. Trost, and M. Althoff, “Human-robot gym: Benchmarking reinforcement learning in human-robot collaboration,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 7405–7411, arXiv:2310.06208.
- [24] L. Yang, B. Werner, R. K. Cosner, D. Fridovich-Keil, P. Culbertson, and A. D. Ames, “SHIELD: Safety on humanoids via CBFs in expectation on learned dynamics,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, arXiv:2505.11494.
- [25] D. F. Henning, C. Choi, S. Schaefer, and S. Leutenegger, “BodySLAM++: Fast and tightly-coupled visual-inertial camera and human motion tracking,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 3781–3788.
- [26] D. F. Henning, T. Laidlow, and S. Leutenegger, “BodySLAM: Joint camera localisation, mapping, and human motion tracking,” in *European Conference on Computer Vision (ECCV)*, 2022, pp. 656–673.

- [27] A. Ray, J. Achiam, and D. Amodei, “Benchmarking safe exploration in deep reinforcement learning,” OpenAI Technical Report, 2019. [Online]. Available: <https://cdn.openai.com/safexp-short.pdf>
- [28] A. Stooke, J. Achiam, and P. Abbeel, “Responsive safety in reinforcement learning by PID Lagrangian methods,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, pp. 9133–9143.
- [29] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018, pp. 2669–2678.
- [30] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *18th European Control Conference (ECC)*, 2019, pp. 3420–3431.
- [31] K. Srinivasan, B. Eysenbach, S. Ha, J. Tan, and C. Finn, “Learning to be safe: Deep RL with a safety critic,” *arXiv preprint arXiv:2010.14603*, 2020.
- [32] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, “Recovery RL: Safe reinforcement learning with learned recovery zones,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4915–4922, 2021.
- [33] J. Thumm, G. Pelat, and M. Althoff, “Reducing safety interventions in provably safe reinforcement learning,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 7515–7522, arXiv:2303.03339.
- [34] P. Svarny, M. Tesar, J. K. Behrens, and M. Hoffmann, “Safe physical HRI: Toward a unified treatment of speed and separation monitoring together with power and force limiting,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 7574–7581, arXiv:1908.03046.
- [35] J. A. Marvel and R. Norcross, “Implementing speed and separation monitoring in collaborative robot workcells,” *Robotics and Computer-Integrated Manufacturing*, vol. 44, pp. 144–155, 2017.
- [36] P. Svarny, J. Rozlivek, L. Rustler, and M. Hoffmann, “3D collision-force-map for safe human-robot collaboration,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 3829–3835, arXiv:2009.01036.
- [37] P. Svarny, J. Rozlivek, L. Rustler, M. Sramek, Ö. Deli, M. Zillich, and M. Hoffmann, “Effect of active and passive protective soft skins on collision forces in human-robot collaboration,” *Robotics and Computer-Integrated Manufacturing*, vol. 78, p. 102363, 2022.
- [38] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 22–31.
- [39] T. He, W. Zhao, and C. Liu, “AutoCost: Evolving intrinsic cost for zero-violation reinforcement learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 12, 2023, pp. 14 847–14 855.
- [40] M. G. Bellemare, W. Dabney, and R. Munos, “A distributional perspective on reinforcement learning,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 449–458.

- [41] D. Yarats, R. Fergus, A. Lazaric, and L. Pinto, “Mastering visual continuous control: Improved data-augmented reinforcement learning,” in *International Conference on Learning Representations (ICLR)*, 2022, arXiv:2107.09645.
- [42] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, “Curriculum learning for reinforcement learning domains: A framework and survey,” *Journal of Machine Learning Research*, vol. 21, no. 181, pp. 1–50, 2020.
- [43] OpenAI, I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell *et al.*, “Solving Rubik’s cube with a robot hand,” *arXiv preprint arXiv:1910.07113*, 2019.
- [44] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in *Proceedings of the 5th Conference on Robot Learning (CoRL 2021)*, ser. PMLR, vol. 164, 2022, pp. 91–100, arXiv:2109.11978.
- [45] Z. Zhuang, S. Yao, and H. Zhao, “Humanoid parkour learning,” in *Proceedings of the 8th Conference on Robot Learning (CoRL)*, 2024, arXiv:2406.10759.
- [46] E. Altman, *Constrained Markov decision processes*. Chapman and Hall/CRC, 1999.
- [47] J. García and F. Fernández, “A comprehensive survey on safe reinforcement learning,” *Journal of Machine Learning Research*, vol. 16, pp. 1437–1480, 2015.
- [48] Q. Yang, T. D. Simão, S. H. Tindemans, and M. T. J. Spaan, “Safety-constrained reinforcement learning with a distributional safety critic,” *Machine Learning*, vol. 112, no. 3, pp. 859–887, 2023.
- [49] W. Dabney, M. Rowland, M. G. Bellemare, and R. Munos, “Distributional reinforcement learning with quantile regression,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [50] B. Könighofer, R. Bloem, N. Jansen, S. Junges, and S. Pranger, “Shields for safe reinforcement learning,” *Communications of the ACM*, vol. 68, no. 11, pp. 80–90, 2025.
- [51] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, “Hamilton-Jacobi reachability: A brief overview and recent advances,” in *IEEE 56th Annual Conference on Decision and Control (CDC)*, 2017, pp. 2242–2253.
- [52] M. Ganai, S. Gao, and S. Herbert, “Hamilton-Jacobi reachability in reinforcement learning: A survey,” *IEEE Open Journal of Control Systems*, vol. 3, pp. 310–324, 2024.
- [53] K. P. Wabersich, A. J. Taylor, J. J. Choi, K. Sreenath, C. J. Tomlin, A. D. Ames, and M. N. Zeilinger, “Data-driven safety filters: Hamilton-Jacobi reachability, control barrier functions, and predictive methods for uncertain systems,” *IEEE Control Systems Magazine*, vol. 43, no. 5, pp. 137–177, 2023.
- [54] K.-C. Hsu, H. Hu, and J. F. Fisac, “The safety filter: A unified view of safety-critical control in autonomous systems,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 7, pp. 47–72, 2024.
- [55] L. Yang, B. Werner, M. de Sa, and A. D. Ames, “CBF-RL: Safety filtering reinforcement learning in training with control barrier functions,” *arXiv preprint arXiv:2510.14959*, 2025, enforces CBF safety filtering during RL training so the deployed policy needs no runtime filter.

- [56] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative Q-learning for offline reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1179–1191.
- [57] C. Khazoom, D. Gonzalez-Diaz, Y. Ding, and S. Kim, “Humanoid self-collision avoidance using whole-body control with control barrier functions,” in *IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*, 2022, pp. 558–565, arXiv:2207.00692.
- [58] D. Morton and M. Pavone, “Safe, task-consistent manipulation with operational space control barrier functions,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 187–194, arXiv:2503.06736.
- [59] R. M. Bena, G. Bahati, B. Werner, R. K. Cosner, L. Yang, and A. D. Ames, “Geometry-aware predictive safety filters on humanoids: From Poisson safety functions to CBF constrained MPC,” in *IEEE-RAS 24th International Conference on Humanoid Robots (Humanoids)*, 2025, pp. 1–8, arXiv:2508.11129.
- [60] Y. Sun, R. Chen, K. S. Yun, Y. Fang, S. Jung, F. Li, B. Li, W. Zhao, and C. Liu, “SPARK: Safe protective and assistive robot kit,” 2025, short version presented at the IFAC Symposium on Robotics, 2025.
- [61] W. Cai, J. Abanes, N. Evangeliou, and A. Tzes, “Safe human-to-humanoid motion imitation using control barrier functions,” 2026, arXiv preprint; simulation only.
- [62] K. Nakamura, L. Peters, and A. Bajcsy, “Generalizing safety beyond collision-avoidance via latent-space reachability analysis,” in *Robotics: Science and Systems (RSS)*, 2025, arXiv:2502.00935.
- [63] K. Nakamura, A. L. Bishop, S. Man, A. M. Johnson, Z. Manchester, and A. Bajcsy, “How to train your latent control barrier function: Smooth safety filtering under hard-to-model constraints,” 2025, arXiv preprint.
- [64] D. D. Oh, D. P. Nguyen, H. Hu, and J. F. Fisac, “Provably optimal reinforcement learning under safety filtering,” 2025, to appear in Proceedings of the International Association for Safe and Ethical AI (IASAI) 2026.
- [65] R. Tian, L. Sun, A. Bajcsy, M. Tomizuka, and A. D. Dragan, “Safety assurances for human–robot interaction via confidence-aware game-theoretic human models,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022, arXiv:2109.14700.
- [66] P. Trautman and A. Krause, “Unfreezing the robot: Navigation in dense, interacting crowds,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2010, pp. 797–803.
- [67] J. Zhang, Y. Xu, and V. Aggarwal, “Don’t freeze, don’t crash: Extending the safe operating range of neural navigation in dense crowds,” 2026, arXiv preprint.
- [68] H. Krasowski, J. Thumm, M. Müller, L. Schäfer, X. Wang, and M. Althoff, “Provably safe reinforcement learning: Conceptual analysis, survey, and benchmarking,” *Transactions on Machine Learning Research*, 2023, arXiv:2205.06750.
- [69] R. F. Prudencio, M. R. O. A. Maximo, and E. L. Colombini, “A survey on offline reinforcement learning: Taxonomy, review, and open problems,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 8, pp. 10 237–10 257, 2024.

- [70] H. Xu, X. Zhan, and X. Zhu, “Constraints penalized Q-learning for safe offline reinforcement learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 8, 2022, pp. 8753–8760.
- [71] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.
- [72] F. Muratore, F. Ramos, G. Turk, W. Yu, M. Gienger, and J. Peters, “Robot learning from randomized simulations: A review,” *Frontiers in Robotics and AI*, vol. 9, p. 799893, 2022.
- [73] S. Shin, J. Kim, E. Halilaj, and M. J. Black, “WHAM: Reconstructing world-grounded humans with accurate 3D motion,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 2070–2080, arXiv:2312.07531.
- [74] B. Zhang, Y. Zhang, J. Ji, Y. Lei, J. Dai, Y. Chen, and Y. Yang, “SafeVLA: Towards safety alignment of vision-language-action model via constrained learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2503.03480. Introduces the Safety-CHORES benchmark.
- [75] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, “RLBench: The robot learning benchmark & learning environment,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [76] Z. Erickson, V. Gangaram, A. Kapusta, C. K. Liu, and C. C. Kemp, “Assistive Gym: A physics simulation framework for assistive robotics,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 10 169–10 176, arXiv:1910.04700.
- [77] X. Puig, E. Undersander, A. Szot, M. Dallaire Cote, T.-Y. Yang, R. Partsey *et al.*, “Habitat 3.0: A co-habitat for humans, avatars, and robots,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2310.13724.
- [78] J. Romero, D. Tzionas, and M. J. Black, “Embodied hands: Modeling and capturing hands and bodies together,” *ACM Transactions on Graphics (Proc. SIGGRAPH Asia)*, vol. 36, no. 6, pp. 245:1–245:17, 2017.
- [79] Y. Tian, H. Zhang, Y. Liu, and L. Wang, “Recovering 3D human mesh from monocular images: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 15 406–15 425, 2023.
- [80] N. Mahmood, N. Ghorbani, N. F. Troje, G. Pons-Moll, and M. J. Black, “AMASS: Archive of motion capture as surface shapes,” in *International Conference on Computer Vision (ICCV)*, 2019, pp. 5442–5451.
- [81] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, “Real-world humanoid locomotion with reinforcement learning,” *Science Robotics*, vol. 9, no. 89, p. eadi9579, 2024.
- [82] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [83] B. Acosta, W. Yang, and M. Posa, “Validating robotics simulators on real-world impacts,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6471–6478, 2022.

- [84] R. Joseph and A. Dutta, “Contact force estimation for a single leg test setup with compliance in MuJoCo,” *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science*, vol. 240, no. 5, pp. 1569–1588, 2026.
- [85] A. Schlotzhauer, T. Stotz, R. Awad, and W. Kraus, “Virtual validation of power and force limiting setups in human-robot-collaboration,” *Procedia CIRP*, vol. 107, pp. 845–850, 2022.
- [86] D. Kim and S. Oh, “Efficient off-policy safe reinforcement learning using trust region conditional value at risk,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7644–7651, 2022.
- [87] D. Kim, K. Lee, and S. Oh, “Trust region-based safe distributional reinforcement learning for multiple constraints,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2301.10923.

A Hyperparameter Tables

A.1 Phase 2: Offline CQL Safety Critic

Table A.1: Phase 2 offline CQL safety-value-function hyperparameters. The dataset is collected under **noisy** perception because a runtime filter must learn the conditions it faces at deployment (Section 5.3).

Hyperparameter	Value	Note
Architecture	MLP [256, 256, 256] + scaled sigmoid	bounded value head
Optimiser	Adam, lr 3×10^{-4}	
CQL regulariser	$\alpha_{\text{CQL}} = 5.0$	ablated in E2.1
Target violation rate	0.30	matches τ_{prox} calibration
Discount	$\gamma = 0.99$	
Target-net Polyak	$\tau = 0.005$	
Dataset	v3: CQN-AS rollouts, <code>coworker_train</code> , noisy	labelled at $\tau_{\text{prox}}=0.3$ m ($\sim 16.8\%$ unsafe)
Sampler	violation-balanced, target 1:1	
Training / batch	200,000 steps / 512	

The four collection-versus-deployment faithfulness fixes that make this critic’s deployment decisions valid (action de-normalisation, execution-mode matching, the $500 \rightarrow 20$ Hz control-frequency root cause, and coworker-scenario alignment) are described in Section 4.9; the $R = 0$ sweep agreement ($0.286 \approx 0.296$) is the validation that the fixes succeeded.

A.2 Phase 3: Constrained Lagrangian Agent (Headline: Fixed λ)

Table A.2 gives the headline configuration. The headline multiplier is *fixed* at $\lambda = 0.1$; the PID gains below are used *only* for the E3.2 auto-tuning runs that establish the seed-instability finding (Section 5.12.2), and are set to zero for the headline. There is no active cost budget d in the headline, because pinning λ removes the dual-update loop entirely; $d = 0.3$ is the budget used in the PID-ablation runs.

Table A.2: Phase 3 headline hyperparameters (fixed- λ constrained CQN-AS). The cost critic is fully decoupled (its own optimiser and its own features, the RoboBase shared-encoder constraint) and cold-started.

Group	Value	Note
CQN-AS backbone	C51, 101 atoms, $\gamma = 0.99$	[4]
Value support	$v_{\min} = -6$, $v_{\max} = +2$	satisfies Prop. 4.1
Cost critic	mirrored C51, same support	decoupled optimiser & features; cold-started
Replay / batch / stack	1M / 256 / 4	
Demonstrations	num_demos = 36	
Multiplier (headline)	fixed $\lambda = 0.1$	dual loop removed
PID gains (E3.2 ablation only)	$K_I=10^{-3}$, $K_P=10^{-2}$, $K_D=0$	$\lambda_{\max}=100$; zero in headline
Cost budget (E3.2 ablation only)	$d = 0.3$	not active in headline
Workspace shaping	$\beta=0.05$, $r_{\text{ws}}=0.4$ m, $c_{\text{ws}}=1.0$ m	confound-controlled (E4.1)
Lagrangian fine-tune	up to 40,000 frames	on the stage-2 base
Checkpoint selection	safety-aware	lowest deploy proximity at success floor 0.75
Eval interval	2,000 frames, 20 eval ep	basin at $\sim 20\text{k}$ – 35k frames

A.3 Phase 4: Per-Regime Deployment Configurations

The deployed runtime configuration is regime-dependent (Section 5.15). On the persistent-co-location task, the both-axis configuration is the fixed- λ policy composed with *unconditional* graded speed-scaling ($d_{\text{slow}} = 0.40$ m); gating does not help there. On the intermittent-co-location tasks the deployment filter is *critic-gated* speed-scaling at the recommended per-task operating points. The learned SVF *veto* ($R = 2.25$, $\alpha_{\text{CQL}} = 5.0$) is a negative result everywhere (velocity backstop at best, OOD on the avoiding policy, coherence break on chunked policies) and is never the deployment filter; Table A.3 lists the configurations.

Table A.3: Runtime deployment configurations per regime. The veto and the model-based dodges are taxonomy points, not deployment configurations.

Task (regime)	Filter	Operating point	Critic
saucepan (persistent)	uncond. speed-scaling on fixed- λ policy	$d_{\text{slow}}=0.40$, $d_{\text{stop}}=0.15$ m	none (ungated)
dishwasher (intermittent)	critic-gated speed-scaling	$R=2.75$, $d_{\text{slow}}=0.50$, $d_{\text{stop}}=0.15$ m	svf_dish_drawers_v1
drawers (intermittent)	critic-gated speed-scaling	$R=3.0$, $d_{\text{slow}}=0.80$, $d_{\text{stop}}=0.25$ m	svf_dish_drawers_v1
drawers (near-free option)	critic-gated speed-scaling	$R=2.75$, $d_{\text{slow}}=0.80$ m	svf_dish_drawers_v1
(all tasks) SVF veto	negative result; not deployed	$R=2.25$	per-task SVF
(saucepan) model-based dodge	taxonomy point (flee); not deployed	$d_{\text{target}}=0.35$ m	none

Shared dish/drawers SVF critic. Same architecture and CQL recipe as Table A.1, trained once for both tasks: 52,794 transitions (random + on-policy curriculum rollouts under COWORKER), near-contact label at min-separation < 0.10 m, 200k training steps. Anticipatory-label variants (0.20/0.30 m) and a DAgger-style re-collection on filtered rollouts were trained and rejected by the ablations of Section 5.14.5.

WCSAC external baseline. Faithful reimplementation of [1] (the Gaussian variant; the journal extension adds a quantile-regression critic [48]): Gaussian distributional cost critic, CVaR $_{\alpha}$ constraint with $\alpha = 0.9$, budgets $d \in \{5, 15, 30\}$ on the discounted cost return, adaptive multiplier ($\lambda_{\text{max}} = 100$); trained from scratch (num_demos = 0), 150k frames, single seed per cell; evaluated with identity action-normalisation statistics (Section 4.4) over 30 episodes per cell under **oracle** perception.

B Reproducibility Checklist

Code

<https://github.com/ayushpatel4/safety-bigym-2> (MIT licence). Headline experiments were run from the `phase3` branch at commit `b3b2ce5`; in addition, the snapshot-evaluation harness stamps the exact git SHA into every benchmark CSV row, so each reported cell is traceable to the code that produced it.

Upstream assets

BiGYM [6] is consumed via a public fork at <https://github.com/ayushpatel4/bigym>, which carries the collision-channel and PD-actuator fixes of Section 3.4. AMASS [80] (CMU subset, downloaded 18 February 2026) drives the demonstration-replay human-state channel. CQN-AS [4] is vendored at upstream commit `8cf806e` (Section 4.8).

Hardware and software

A single NVIDIA A100 (40 GB), CUDA 12.8, PyTorch 2.10.0; MuJoCo via BiGYM. Every script has a CPU-only smoke gate (Section 5.6).

Seeds

Trained seeds $\{0, 1, 2\}$ for all three-seed cells; evaluation uses seeds $\{0, 1, 2\}$ with 20 episodes each (Section 5.6).

Headline recipe (R1)

(i) Train the three-stage curriculum (Table 4.5; ≈ 19 h). (ii) Fine-tune with fixed $\lambda = 0.1$ for up to 40k frames per seed (≈ 9 h each; Table A.2). (iii) Benchmark every saved checkpoint and select per seed by the safety-aware rule (lowest deployment proximity at success ≥ 0.75), which yields the headline checkpoints `lam0p1_seed{0,1,2}` at steps 30,546 / 8,225 / 32,696. (iv) Reproduce Table 5.7 by running `scripts/benchmark_policy.py` on those checkpoints (minutes per cell; no retraining).

Headline recipe (R2/R3)

All intermittent-co-location and boundary numbers are snapshot evaluations through the same harness: the budget sweep via `scripts/dispatch_budget_sweep.py` (artifacts under `results/budget_sweep_eval/`); the veto, unconditional, gated, and ablation sweeps under `results/{svf_sweep, speedscale, gated_sweep, improve, anticipatory_sweep, dagger_sweep}/`; the boundary sweep via `scripts/dispatch_gated_saucepan.py` (under `results/e4_1/gated_saucepan/`); WCSAC via `scripts/dispatch_wcsac.py` and `scripts/eval_wcsac.sh` (under `results/wcsac_eval/`). Figures regenerate via `scripts/make_report_figures.py` and `scripts/make_f7_gate_activity.py`.

Hyperparameters

Appendix A; the Hydra configs in the repository pin every value not listed there.

C Use of Generative AI

Generative AI tools (Anthropic Claude, OpenAI ChatGPT) were used during this project, and their use is declared here in full. Their role was assistive and reviewable:

- **Literature search and triage.** AI tools helped shortlist candidate work in safe RL, CBF filtering, shielding, and human-aware robotics. Every retained citation was then checked manually against a primary source (arXiv, IEEE/ACM/PMLR/AAAI/NeurIPS/JMLR, ISO catalogue, or publisher page). This verification step changed several entries; the audit is recorded in `REFERENCES_AUDIT.md`.
- **Code scaffolding.** AI tools drafted repetitive infrastructure such as data-loading helpers, wrapper boilerplate, and plotting scripts. Correctness-bearing code, including training loops, loss functions, cost signals, evaluation metrics, and aggregation logic, was reviewed, modified where necessary, and tested by the author before use.
- **Experiment-log summarisation.** AI tools helped summarise large collections of CSVs, JSONL logs, and draft result notes into candidate tables or prose. Every numerical claim in the report was checked back to an artifact in `results/`, a documented script, or a saved figure-data note before inclusion.
- **L^AT_EX and prose.** AI tools assisted with L^AT_EX formatting, restructuring, drafting, and proofreading. No AI-generated passage is included verbatim without author editing; the author chose the final framing, claims, section ordering, and wording.

The tools’ limitations shaped the workflow. They sometimes overstated ISO compliance or treated simulated SSM improvements as real-world certification, so all safety claims were edited back to the qualified SSM/proximity language of Section 6.2.1. They also surfaced inaccurate bibliographic metadata, which is why the reference audit was performed. They did not diagnose the C51 support-saturation failure (Section 4.4.5) or the CUDA `linspace` integer-overflow bug (Section 4.8); both required direct code, simulator, and log inspection. All experimental design decisions, all choices of what to run and measure, all quantitative results, and all scientific claims are the author’s responsibility.

D Ethical Considerations

The project involves no human-subjects research and no personal data. All human-motion data is from the publicly released AMASS dataset [80], used under its research-only licence. No real-robot deployment was conducted; the simulated humanoid–human contact forces are not safety-validated for real-world use. The work targets a safety standard (ISO 15066) and aims to improve safety outcomes; the principal ethical risk is over-claiming compliance and thereby prompting under-validated deployment. This is mitigated by the qualified, SSM-only compliance claim of Section 6.2.1, by the honest reporting of the safety–throughput trade rather than presenting the safe configuration as free, and by the explicit recommendation in Section 6.3 that any real-world deployment require independent validation and, for vulnerable users, stronger guarantees than this work provides.

E Sustainability

Total compute was roughly 450–500 GPU-hours on NVIDIA A100 (40 GB) GPUs (Section 5.7): ≈ 220 h for the persistent-co-location arc, reconstructed from run logs, plus ≈ 230 – 280 h for the intermittent-co-location arc and the WCSAC external baseline, of which the six 150k-frame from-scratch WCSAC runs dominate and are estimated rather than log-reconstructed. At an assumed mean board power of 0.3 kW and a data-centre power-usage effectiveness of 1.4, the midpoint (≈ 475 h) is approximately 200 kWh; at the 2024 UK grid carbon intensity of 0.21 kg CO₂e/kWh, approximately 40 kg CO₂e. Several practices kept this figure low: resumable training with checkpoint reuse; CPU smoke gates before every multi-hour training launch; deliberate seed-count selection (3–5) balancing statistical confidence against compute; warm-starting the constrained policy from the Phase 1 base-policy snapshot rather than retraining the task from scratch (Section 4.4.9); and snapshot-only Phase 2 dataset collection, which avoids a separate, slow random-rollout pass. The dominant cost is policy training, not evaluation; the snapshot harness makes re-evaluation effectively free relative to a retrain.

F Code and Data Availability

The code, including the `safety_bigym` benchmark, all runtime-filter implementations, the training entry points, and the snapshot-evaluation harness (`scripts/benchmark_policy.py`, Section 3.7), is open-source under the MIT licence at <https://github.com/ayushpatel4/safety-bigym-2>. The BiGYM simulator fork carrying the collision-channel and PD-actuator fixes of Section 3.4 is public at <https://github.com/ayushpatel4/bigym>. Pre-trained weights for the headline fixed- λ policy (the three selected checkpoints listed in Appendix B), the saucepan (v3) and shared dish/drawers (v1) SVF critics together with their dataset shards (37,883 and 52,794 transitions respectively), the WCSAC checkpoints, and the per-cell benchmark CSVs behind every results table (including the gated, ablation, and boundary sweeps) are archived on the project’s experiment workstation and are available from the author on request while a hosted release is prepared; every quantitative table in this report is regenerable from those snapshots with the harness, without any retraining. One exception is recorded for completeness: the three R1 basin checkpoints were lost to a storage cleanup after their benchmark CSVs were written (Section 5.15); their numbers remain reproducible from the archived CSVs, and regenerating the checkpoints themselves requires re-running the (deterministic-recipe but stochastic-outcome) fixed- λ fine-tune.